



PI Interface for OPC DA

2.7.2.20

© 2015-2026 AVEVA Group Limited and its subsidiaries. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, mechanical, photocopying, recording, or otherwise, without the prior written permission of AVEVA Group Limited. No liability is assumed with respect to the use of the information contained herein.

Although precaution has been taken in the preparation of this documentation, AVEVA assumes no responsibility for errors or omissions. The information in this documentation is subject to change without notice and does not represent a commitment on the part of AVEVA. The software described in this documentation is furnished under a license agreement. This software may be used or copied only in accordance with the terms of such license agreement. AVEVA, the AVEVA logo and logotype, OSIsoft, the OSIsoft logo and logotype, Archedra, Avantis, Citect, DYNsIM, eDNA, EYESIM, InBatch, InduSoft, InStep, IntelaTrac, InTouch, Managed PI, OASyS, OSIsoft Advanced Services, OSIsoft Cloud Services, OSIsoft Connected Services, OSIsoft EDS, PIPEPHASE, PI ACE, PI Advanced Computing Engine, PI AF SDK, PI API, PI Asset Framework, PI Audit Viewer, PI Builder, PI Cloud Connect, PI Connectors, PI Data Archive, PI DataLink, PI DataLink Server, PI Developers Club, PI Integrator for Business Analytics, PI Interfaces, PI JDBC Driver, PI Manual Logger, PI Notifications, PI ODBC Driver, PI OLEDB Enterprise, PI OLEDB Provider, PI OPC DA Server, PI OPC HDA Server, PI ProcessBook, PI SDK, PI Server, PI Square, PI System, PI System Access, PI Vision, PI Visualization Suite, PI Web API, PI WebParts, PI Web Services, PRISM, PRO/II, PROVISION, ROMEo, RLINK, RtReports, SIM4ME, SimCentral, SimSci, Skelta, SmartGlance, Spiral Software, WindowMaker, WindowViewer, and Wonderware are trademarks of AVEVA and/or its subsidiaries. All other brands may be trademarks of their respective owners.

U.S. GOVERNMENT RIGHTS

Use, duplication or disclosure by the U.S. Government is subject to restrictions set forth in the license agreement with AVEVA Group Limited or its subsidiaries and as provided in DFARS 227.7202, DFARS 252.227-7013, FAR 12-212, FAR 52.227-19, or their successors, as applicable.

AVEVA Legal Resources: <https://www.aveva.com/en/legal/>

AVEVA Third Party Software Notices and Licenses: <https://www.aveva.com/en/legal/third-party-software-license/>

Contents

PI Interface for OPC DA 2.7.2	7
Introduction to PI Interface for OPC DA	8
Hardware configurations	8
Operational overview	10
Related manuals	11
Related tools	12
Supported features and operating systems	13
DMZ Secure Link	15
Install the OPC DA interface	16
Installation prerequisites	16
Installation directory and file locations	16
Install the OPC DA interface on the interface node	16
Migration steps for existing instances of the read/write interface	17
Configure the OPC DA interface	18
Configuration checklist	18
Interface startup configuration	19
Create and configure the interface instance	19
Learn about server connection management	21
Security configuration	21
Best practices	21
Create trusts	22
Configure the Windows service	23
Manage the PI Interface for OPC DA service	24
DCOM configuration	25
DCOM security levels	25
DCOM clients and servers	25
Windows domains and users	25
DCOM user accounts	26
Firewalls and security	27
PI point configuration	27
Export OPC items to PI Builder	28
Create PI points manually	28
Configure point attributes	28
PI point attributes	29
Tag (PI point name)	30
PointSource (point source)	30
Data type attribute	31
Location1 (interface instance)	31

Data type handling attribute (location2)	31
Processing type attribute (location3)	33
Scan class attribute (location4)	33
OPC deadband (location5)	34
OPC ItemID attribute (instrumenttag)	35
Extended descriptor attribute (exdesc)	35
SourceTag	36
Scaling code attribute (totalcode)	36
Square root attribute (squareroot)	36
Conversion factor attribute (convers)	36
OPC array index attribute (userint1)	36
Event group attribute (userint2)	37
Scan	37
Shutdown	37
Exception reporting attributes	38
Scan class settings and update rates	38
Input points	40
Advise points	40
Event points	41
Polled points	43
Learn about output point configurations	44
Learn about allowlist file configurations	44
Learn about interface outputs to OPC servers	45
Data type compatibility	45
Time stamp adjustments	47
Transformations and scaling	49
Data quality information	51
Reading OPC quality into a digital PI point	54
OPC array item PI points	55
OPC arrays as event PI points	55
Failover configuration	56
UniInt failover modes	56
OPC server-level failover	58
Configure OPC server failover	59
Configure failover on OPC server state change	60
Server status and OPC watchdog points	60
Configure a single watchdog PI point	60
Configure multiple watchdog PI points	61
OPC item quality	61
Configure server-specific OPC watchdog points for efficient failover	61
Failover timing	62
Buffering for PI Interfaces	62
Buffering services	63
Buffering and collectives	63
Buffering configuration	63
Enable buffering	64
Learn about plug-ins (post-processing DLLs)	64

Operate the OPC DA Interface	66
Verify interface startup	66
OPC server data collection	66
Learn about PI ICU reference	67
OPC server settings	67
Advanced options settings	68
Data Handling settings	70
DCOM Security settings	72
Failover settings	72
Plug-in settings	75
Miscellaneous settings	75
Debug settings	75
Time stamp options	77
Troubleshooting the interface	78
Open PI OPC client tool	78
OPC server issues	78
Item browsing	78
OPC server time stamp options	78
Unreliable values	79
Access path	79
Problems with data returned by OPC server	79
OPC server operation	80
Interface specific messages	81
OPC refreshes	81
Shutdown	83
OPC server performance issues	83
Command-line parameters	86
Alphabetical list of parameters	86
Parameters by function	103
Review error and informational messages	106
Logging	106
Message Logs	106
System errors and PI system errors	116
UniInt failover-specific messages	116
Appendix A. Set ExcMax on advise and event points	120
Why set ExcMax for an advise point?	120
How does the interface handle ExcMax for advise points?	120
Why not just use the /TA parameter?	121
Why is the data sometimes out of order?	121

What happens if I ignore the fractional parts of timestamps?	122
What happens if I set ExcMax equal to the update rate?	122
Summary	123
 Glossary of interface terms	 124
buffering	124
ICU	124
ICU Control	124
n-way buffering	124
OCS	124
OMF	125
pigetmsg utility	125
PIHOME	125
PIHOME64	125
pipc.log file	125
point	125
PI API	126
PI Data Archive	126
PI Data Archive collective	126
PI Data Archive computer	126
PI ICU	126
PI Interface Configuration Utility	126
PI message log	127
PI Message Subsystem	127
PI point	127
PI point attribute	127
PI SDK	127
PI SMT	127
PI System Management Tools	128
tag attribute	128
 Release notes	 129

PI Interface for OPC DA 2.7.2

PI Interface for OPC DA is an AVEVA client application that reads and archives data from OPC data sources. This release includes support for OMF Health Messaging.

For details about this release, see the [Release notes](#).

Introduction to PI Interface for OPC DA

PI Interface for OPC DA is an AVEVA client application that communicates between an OPC server and Data Archive. PI Interface for OPC DA collects real-time data from the OPC server and writes values to PI points. Optionally, the interface can write data from PI points in Data Archive to the OPC server.

PI Interface for OPC DA is built on the AVEVA UniInt framework, which includes a standard set of common features. For example, the interface can be configured to run as an automatic service, use disconnected startup, and use buffering to prevent data loss. UniInt framework features are documented in the [PI Universal Interface \(UniInt\) Framework User Guide](#), which is a companion guide to this manual.

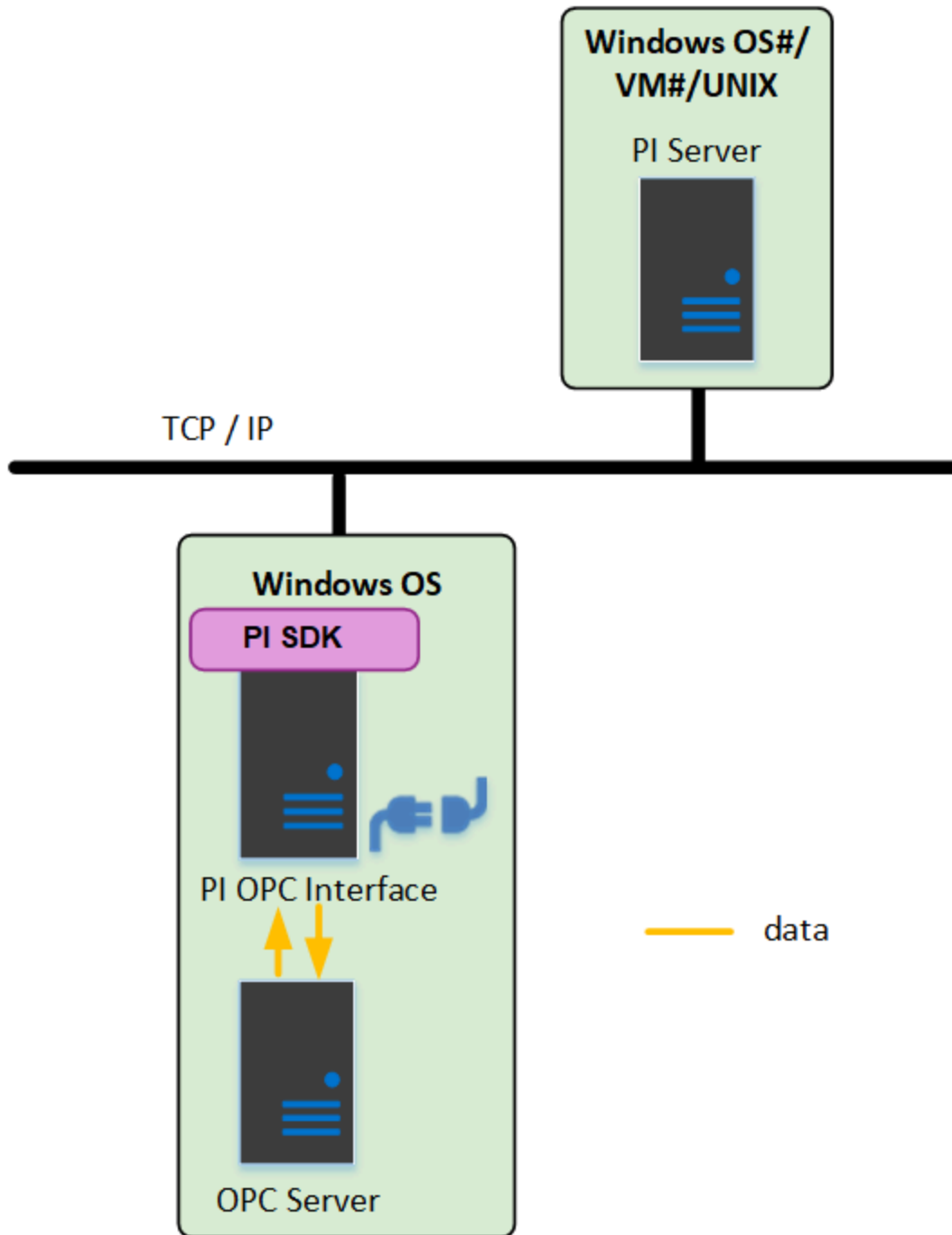
OPC is dependent on Microsoft COM and DCOM services, and the interface must run on Windows platforms. PI Interface for OPC DA supports versions 1.0a, 2.0, and 2.05 of the OPC Data Access standard.

Note: The OPC DA standard is designed for real-time data. For OPC XML-DA specification data, use PI Interface for OPC DA XML. To collect or synchronize historical OPC data using the OPC HDA standard, use PI Interface for OPC HDA. To collect alarm and event OPC data using the OPC AE standard, use PI Interface for OPC Alarms and Events.

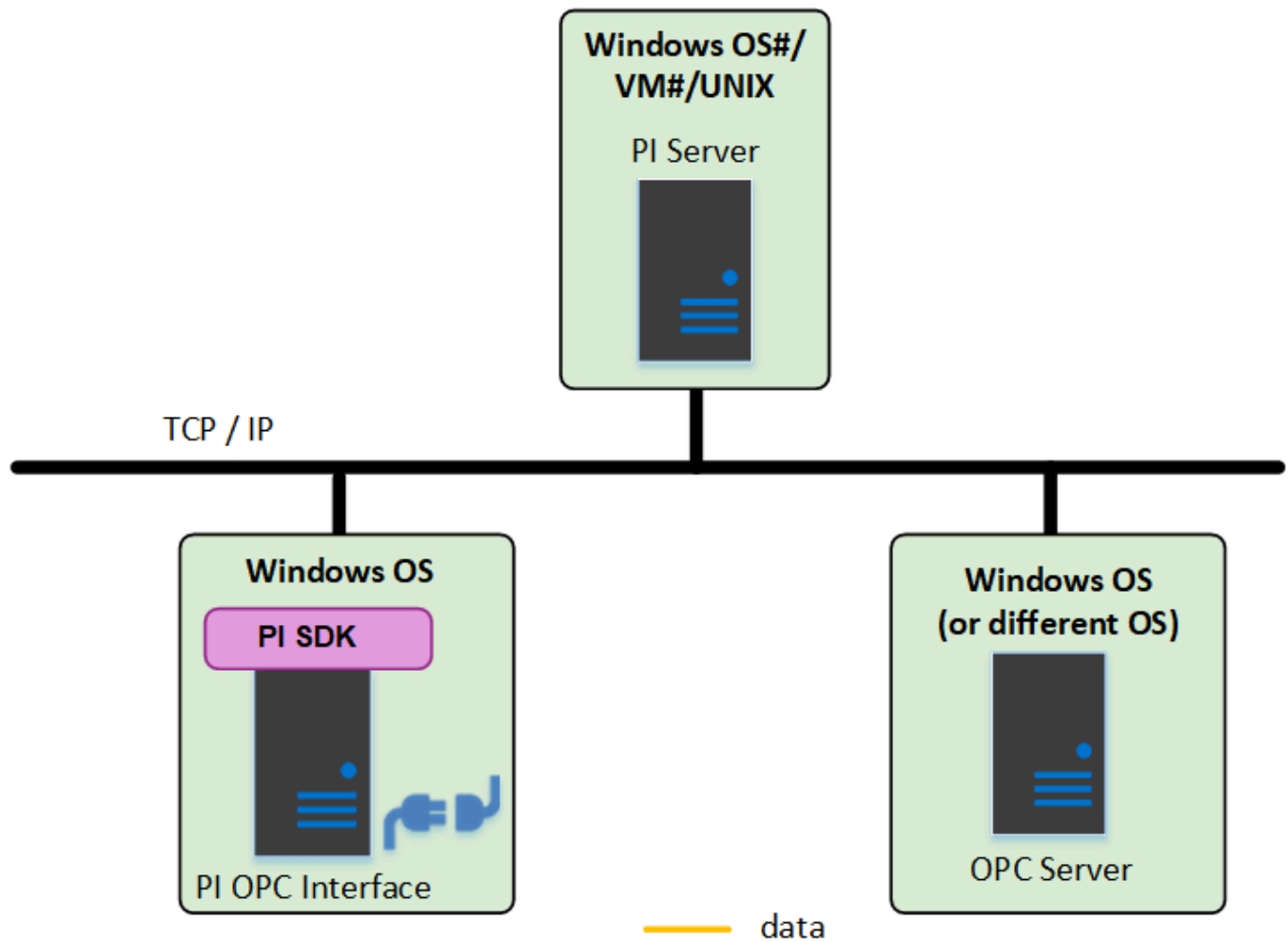
Hardware configurations

PI Interface for OPC DA can be configured to run on the OPC server machine, the Data Archive server machine, or on a separate machine. Each configuration requires a different set of settings to run the interface. To ensure that the interface does not compete with Data Archive for system resources, install the interface on a dedicated computer.

Simple configuration that permits data buffering on the interface node



Configuration that places the OPC server on its own computer



Operational overview

PI Interface for OPC DA starts by using a Windows batch (**.bat**) file that invokes the interface executable. The **.bat** file contains command-line parameters for the interface settings.

For normal startup, PI Interface for OPC DA performs the following operations:

1. Connects to Data Archive (unless disconnected startup is configured)
2. Gets PI points from Data Archive
3. Connects to OPC server
4. Creates OPC groups
5. Adds items to groups
6. Activates groups (unless groups are created as active)
7. Begins data collection

Once data collection starts, the interface performs the following operations:

- Services scheduled input points and process each scan class in turn

- Services output points as events arrive
- Services triggered input points as events arrive
- Checks the PI point database for points that are added, edited, and deleted
- Reloads new or changed points

The interface checks for point updates every 2 minutes. The interface processes 25 point updates at a time. If more than 25 points are added, edited, or deleted at one time, the interface processes the first 25 points, waits 30 seconds (default) or one-fourth of the time specified by the `Unlnt updateinterval` parameter, whichever is lower, processes the next 25 points, and so on. After all points have been processed, the interface resumes checking for updates at the update interval.

Related manuals

PI Interface for OPC DA is based on the Unlnt framework. Unlnt interfaces use and interact with many other components of the PI System. For related information, refer to the following documents:

- **PI Universal Interface (Unlnt) User Guide**
Describes configuration, operation, and troubleshooting for Unlnt-based interfaces. See the [PI Universal Interface \(Unlnt\) Framework User Guide](#).
- **Read-only versus Read-write FAQ**
Describes information on the differences between read-only and read-write interfaces, and instructions for migrating from the read-write to the read-only option. See the [Read-only versus Read-write FAQ](#).
- **DCOM Security and Configuration**
Describes security configuration for OPC-related PI System products. See [DCOM Security and Configuration](#).
- **PI OPC Client User Guide**
Includes information on using the PI OPC Client Tool. See the [PI OPC Client Tool User Guide](#).
- **Data Archive System Management Guide**
Documentation for managing Data Archive installations, interfaces, PI points, and digital state sets. See the [PI Data Archive System Management Guide](#).
- **Data Archive Security Configuration Guide**
Includes information about Data Archive security. See the [PI Data Security Configuration Guide](#).
- **High Availability Administrator Guide**
Describes how to configure high availability and manage Data Archive and interface installations when using PI collectives. See the [High Availability Administrator Guide](#).
- **AF SDK and PI API Online Help Files**
Documentation for the AF SDK and PI API libraries used by interfaces. The embedded help (**.chm**) files are located in the `%PIHOME%\PIPC\Help\` directory. **PIHOME** is an environment variable that contains the PI API installation path. See [AF SDK and PI API Online Help Files](#).
- **PI API Installation Instructions User Manual**
Details about installing supporting components, including buffering to prevent data loss if the connection to Data Archive is lost. See the [PI API Installation Instructions User Manual](#).
- **PI Buffering Manager Help Files**

Explains how to configure buffering using PI Buffer Subsystem 4.3 or later, including how to upgrade from API Buffer Server (bufserv) if needed. The embedded help files are located at **PIPC/HELP/BufferManager.chm**.

- PI Interface Configuration Utility User Guide

Describes how to configure interfaces using PI ICU. See the [PI Interface Configuration Utility User Guide](#).

- Interface logging

For more information about reading message logs, see the AVEVA Knowledge Base article [How to read new Unilnt 4.5.0.x and later Interface message logs](#).

- Interface release notes

Release-specific information about functional updates, bug fixes, and new features. Release notes are provided both for the Unilnt framework and for each interface.

- Data Access Custom Interface Standard v2.05a

The OPC Foundation documentation for v2.05a of OPC DA.

- Handling non-English characters or strings on the PI Server

See the AVEVA Knowledge Base article, [Can non-English characters be used in tag names, modules names, module database alias names, and properties names](#).

Related tools

PI Interface for OPC DA does not have its own graphical user interface (GUI). The following tools perform multiple functions related to setting up, configuring, running, and troubleshooting the interface and the OPC server:

- PI OPC Tools Set

Connects to OPC servers and displays the data in connected OPC servers, including groups and items.

- OPC to CSV tool

Provides access to OPC item properties and enables you to apply OPC server-specific or site-specific knowledge to revise or create PI point attributes before PI points are created or edited. Exports OPC items to a comma-separated values file that can be loaded into the PI Builder add-in for Excel.

- OPC Enum tool

Discovers OPC servers.

- Data Translation tools

These programs, which are run from the command line, are used for translating time stamps from refresh logs for the OPC server.

- opcsan.exe
- opcrefresh.exe
- opcresponse.exe

- PI Interface Configuration Utility (PI ICU)

PI ICU is used to create and configure instances of each interface. Once created, each instance is available from the drop-down menu in PI ICU.

PI Interface for OPC DA no longer includes PI OPC Tools in the self-extracting installation kit. PI OPC Tools can be downloaded from the AVEVA [Customer Portal Products page](#). Search for PI Interface for OPC DA and select the

download icon to locate the installation kit for PI OPC Tools Set (OPCTools) Install Kit.

Supported features and operating systems

Supported operating systems

PI Interface for OPC DA is supported on the following operating systems:

- Windows 11
- Windows 10
- Windows Server 2022
- Windows Server 2019
- Windows Server 2016
- Windows Server 2012

Supported features

This section lists the supported features for this interface:

Feature	Support
OPC Data Access Standard	1.0a / 2.0 / 2.05
Auto creates PI points	APS Connector
Point Builder utility	Yes
ICU control	Yes
PI point types	Int16 Int32 Float16 Float32 Float64 Digital String Timestamp
Sub-second time stamps	Yes
Sub-second scan classes	Yes
Automatically incorporates PI point attribute changes	Yes
Exception reporting	Interface: PI exceptions OPC server: Deadband
Outputs from Data Archive	Yes
Inputs to Data Archive	Scan-based unsolicited event tags
Outputs to data source	Yes
Read-only interface available*	Yes

Feature	Support
Supports questionable bit	Yes
Supports multi-character <i>pointsource</i> PI point attribute	Yes
Maximum point count	Unlimited
Uses PI SDK*	Yes
PINet string support	N/A
Source of time stamps	Interface or OPC server (configurable)
History recovery	No
UniInt-based*	Yes
Disconnected start-up*	Yes
<i>SetDeviceStatus</i>	Yes
Failover options	OPC server-level failover and UniInt Phase 2 Interface-level failover
Vendor software required on PI interface node	No
Vendor software required on data source	Yes
Vendor hardware required	No
Additional PI software included with interface	Yes
Device point types	• VT_I2 • VT_I4 • VT_R4 • VT_R8 • VT_BSTR • VT_DATE
Serial-based interface	No
OMF Health Messaging**	Yes

* - Additional documentation in the section below.

** - For more information, see the "OMF Health Messaging" topic in the [PI Universal Interface \(UniInt\) User Guide](#).

- **Read-only Interface available**

A read-only version of this interface is available. Read-only versions cannot update the data source (that is, control system). If outputs to the data source are not needed, use the read-only version of the interface.

Note: A read-only interface provides the best defense against accidental or malicious changes to the control system.

- **Uses AF SDK**

The AF SDK and the PI API are bundled together and must be installed on each interface node. This interface does not specifically make AF SDK calls.

- **Unilnt-based**

Unilnt stands for Universal Interface. Unilnt is not a separate product or file; it is an AVEVA-developed template used by developers and is integrated into many interfaces, including this interface. The purpose of Unilnt is to keep a consistent feature set and behavior across as many of AVEVA's interfaces as possible. It also allows for the very rapid development of new interfaces. In any Unilnt-based interface, the interface uses some of the Unilnt-supplied configuration parameters and some interface-specific parameters. Unilnt is constantly being upgraded with new options and features.

- **Disconnected Start-Up**

The PI OPC DA interface is built with a version of Unilnt that supports disconnected start-up. Disconnected start-up is the ability to start the interface without a connection to the PI Data Archive. This functionality is enabled by adding /cachemode to the list of start-up parameters or by enabling disconnected startup using the ICU.

DMZ Secure Link

DMZ Secure Link uses a transparent proxy where an allowlist, maintained by AVEVA, enables connection from on-premise System A to System B (CONNECT) easily and securely. This solution requires minimal configuration and no extra administration while offering the following advantages:

- DMZ Secure Link facilitates using AVEVA SaaS solutions in a segmented network architecture without exposing systems to the entire internet.
- DMZ Secure Link supports compliance and regulatory requirements.
- DMZ Secure Link can work with additional proxy servers to protect your network infrastructure.

To learn more about how to install and configure the product, see the [DMZ Secure Link](#) documentation.

Install the OPC DA interface

To install PI Interface for OPC DA, verify that the system meets all the installation prerequisites and then run the executable (.exe) file.

After installing the interface software, configure interface settings using PI ICU.

Installation prerequisites

Before installing and configuring the interface, take the following steps to ensure successful operation.

1. Confirm that your operating system, vendor hardware, and software support the requirements of the interface. For more information, see [Supported features and operating systems](#).
2. Use PI System Management Tools (PI SMT) to view the status of your connection to Data Archive.
The system must be connected to a running Data Archive node.
3. Use PI OPC Client Tool to view the OPC server connection and contents and verify that the system is connected to a running OPC server that is populated with points.
4. Verify that the time zone for the interface node is set correctly. Time zone settings are discussed in the [Configure the OPC DA interface](#) and other sections of this user guide.
5. Verify that an administrator account is being used to install the interface. For more information about security requirements for the interface, see [Security configuration](#).

Installation directory and file locations

The PI Interface for OPC DA installation directory contains all the files required to configure and run the interface. By default, the PI Interface for OPC DA is installed in the following location: `%PIHOME%\Interfaces\OPCInt\`.

The `%PIHOME%` root directory is an environmental variable under which AVEVA products are installed, and is defined by the `PIHOME` entry in the `pipc.ini` configuration file in the `%windir%` directory. To override the default locations, edit the `pipc.ini` configuration file.

Note: Restrict the Windows accounts that can create or write files in the `%PIHOME%` folder and subfolders. For more information about account security and restricting directory and file access, see the *PI Universal Interface (Unilnt) User Manual*.

The OPCEnum tool, which discovers OPC servers, is installed in the `%windir%\system32` directory, except on 64-bit systems, where it is installed in `%windir%\sysWOW64`. The OPC libraries, provided by the OPC Foundation and installed with the interface, are installed in the same directory as OPCEnum.

Install the OPC DA interface on the interface node

Install the software for the interface on the interface node.

Verify that the [Installation prerequisites](#) are met.

1. On the interface node, run the PI Interface for OPC DA setup program.
2. Test the connection to Data Archive from the interface node.
 - For PI API connections, open a command prompt, navigate to the **%PIPC%\bin** directory, and issue the `apisnap PISERVERNODE` command.
 - For AF SDK connections, from the Windows **Start** menu, select **All Programs > PI System > About PI-SDK**, and then select **File > Connections** to connect.

Configure the interface using PI ICU. See [Configure the OPC DA interface](#) for more details.

Migration steps for existing instances of the read/write interface

When upgrading from a previous read and write version of an interface to a read-only version, migration steps should be followed for existing interfaces. See the [Read-only versus Read-write FAQ](#).

Configure the OPC DA interface

After the interface software is installed, you must configure the following items:

- Startup parameters
- Security
- Windows service
- PI points
- Buffering

Additional configuration options include failover, disconnected startup, and interface performance and status points. For details about configuration for common UniInt interface features, refer to the *PI Universal Interface (UniInt) User Guide*.

Copies of the interface software, called interface instances, can be made on one or more interface nodes. Each interface instance requires its own startup (**.bat**) file. The **.bat** file contains parameters, sometimes called flags, which are configured to create the startup and runtime settings of the interface.

Use PI ICU to configure interface settings to ensure a correctly-formatted **.bat** file. Once the first interface instance is configured, you can use a copy of the configured **.bat** file and replace instance-specific parameters for each interface instance. The **.bat** files are located in the installation directory for the interface, which also includes a template batch file (**opcint.bat_new**) that you can use for reference and testing.

The **.bat** file parameters can also be run from the command line for testing and troubleshooting. For a complete list of parameters, open a command window, navigate to the interface installation directory, and issue the following command: `opcint -help`.

Configuration checklist

After the interface software is installed, use PI ICU to configure the interface instance. A standard configuration would include the following steps. For more information about standard UniInt interface configurations, see the *PI Universal Interface (UniInt) User Guide*. For more information on using PI ICU, see the *PI Interface Configuration Utility (PI ICU) User Guide*.

1. Create the interface instance on the interface node.
2. Configure security.
 - a. [Configure the Windows service](#).
 - b. Restrict access for service accounts.
 - c. Create and configure a PI identity, PI trust, and mappings for the interface (application name: *OPCpE*), the buffering subsystem, and the PI ICU applications.
 - d. Configure DCOM settings.
3. Configure interface startup batch (**.bat**) file parameters with the minimal settings:
 - **Point source**
Use OPC or an unused point source of your choice.

- **Scan classes**
Set as desired. Scan class 1 is reserved for advise points.
 - **Interface ID**
Use 1 or any unused numeric ID.
4. Configure the connection to an OPC server.
From the **OPC Server** tab in the **OPCInt** tab in PI ICU, select the **List Available Servers** button, then select the server. If the server is on another machine, specify that machine or IP address in the **Server Node** field.
 5. Use the PI OPC Client tool to verify that the OPC server is connected, running, and populated with points.
 6. See [Operate the OPC DA Interface](#) on how to verify the interface startup and information about OPC server data collection.
 7. Define the digital state sets for digital PI points.
 8. Build input points and (optional) output points for the interface, and verify that data appears in Data Archive.
For details on point attributes, see [PI point configuration](#).
 9. Configure buffering.
Stop the interface, then configure and start buffering. Restart the interface and confirm that the service and buffering restart.
 10. Configure the following optional features:
 - **Performance and health points**
Performance and health points are PI points that track the status and performance of the interface.

Note: PI Interface for OPC DA does not support scan class performance points.

 - **Disconnected startup**
Disconnected startup allows the interface to start even if Data Archive is not available.
 - **Unilnt failover**
Unilnt failover minimizes data loss by enabling a backup interface instance to collect data if the primary interface instance fails.

Interface startup configuration

PI Interface for OPC DA is normally run as a Windows service. The interface uses a Windows batch (**.bat**) file which specifies settings using command-line parameters.

To ensure a correctly-formatted **.bat** file, use PI ICU. Do not edit the file manually.

For a complete list of Unilnt startup parameters, see the *PI Universal Interface (Unilnt) User Guide*. This document describes interface-specific parameters.

Create and configure the interface instance

For each OPC server intended to exchange data with the PI System, you must create at least one instance of the PI OPC interface. For each instance you create, settings are stored in a separate Windows command file (a **.bat** file) in the PI OPC interface installation directory.

Tip: Configure multiple instances of the PI Interface for OPC DA with unique interface ID (id) and point source

(ps) startup parameters to more easily identify where the data written to Data Archive originated, and trace any problems that arise.

1. Launch PI ICU and select **Interface > New from BAT file**.
2. Browse to the interface installation directory (default %PIPC%\Interfaces\OPCInt), select **OPCInt.bat_new**, and then select **Open**.

The Select PI Host Server window opens.

3. Specify the Data Archive and select **OK**.
PI ICU displays the settings of the new instance of the interface.
4. Configure the basic settings as follows:

- **General tab**

Point source	ID	Scan Class
Use OPC or a point source not already in use.	Use 1 or a numeric ID not already in use by another instance of the interface.	Set to desired scan frequency. Scan class 1 is reserved for advise tags. When defining multiple scan classes, you can spread the server workload using offsets.

- **OPCInt page**

Select the **List Available Servers** button, then select your OPC server from the drop-down list of servers. If the server resides on another machine, specify the node name or IP address in the **Server Node** field before listing the available servers.

Note: Certain OPC servers may refuse remote connection attempts. If the **Server Node** field is blank or set to LocalHost (case insensitive), the interface will treat the server as though it is local. If anything else is used in the **Server Node** field, the connection will be treated as remote.

- **Security Parameters tab**

Most OPC servers do not support OPC security. Verify that your OPC server supports security before enabling this option. If your OPC server requires clients to use OPC security, enable OPC security and select **NT security** or **Private OPC security**, then select **OPC Security Credentials**. This opens a separate file with a .cred extension. Fill in the corresponding OPCUser and OPCPassword fields and save the file. Upon startup of the interface the OPCUser and OPCPassword fields are replaced with asterisks. A corresponding Windows Credential is created for the OPC Security Credentials. The Windows Credential is linked to the account that the interface is running under. The OPC Security Credentials file path corresponds to the parameter */credfilepath*.

Note: Previously set credentials will no longer be valid, they must be reentered using the OPC Security Credentials file.

If your OPC server supports both NT security and private OPC security, AVEVA recommends using NT security. This option authenticates the interface to the server using your Windows domain account and avoids transmitting a password over the network. If you must use private OPC security, configure DCOM using the Packet Privacy authentication level to ensure the password is encrypted in transit. See the *DCOM Security and Configuration Guide* for more details.

Learn about server connection management

Each instance of the PI Interface for OPC DA connects to a single OPC server. To handle multiple OPC servers, run multiple instances of the PI OPC interface. Multiple instances of the interface can be configured to connect to the same OPC server. To enable the interface to collect data without a connection to Data Archive, configure disconnected startup for the interface.

If the interface cannot connect to the OPC server during startup, it logs the problem and retries the connection every five seconds until it reconnects. When the OPC server reports that it is running, the interface connects to it and starts creating groups and adding items. If the PI OPC interface loses the connection to the OPC server after the initial connection, it tries to re-establish the connection. To ensure that no data from the OPC server is lost if Data Archive is disconnected, configure buffering on the interface node.

For more information about buffering, see the *PI Buffering Manager Help Files*.

Security configuration

Sending data across networks and through individual machines within those networks can pose security risks. Therefore, AVEVA recommends you configure your interface service accounts using the most secure methods possible that meet your requirements.

For general information about service account types, restricting access for security accounts and securing read-write interfaces, see the *PI Universal Interface (Unilnt) User Guide*.

For more information about configuring DCOM security, see the *DCOM Security and Configuration Guide*.

Best practices

For more information about security for OPC, see [DCOM Security and Configuration](#).

For more information about configuring security for Data Archive, see *PI Server Configuring PI Server Security*.

Interface installation requires an administrator account for the following tasks:

- Installing the interface software
- Creating the interface service account
- Creating, editing, and removing the interface service
- Adding, updating, and removing performance counters

Security considerations service accounts for PI Interfaces based on the Unilnt framework are discussed in the *PI Universal Interface (Unilnt) User Guide*.

Note: In order to assure that PI buffering functions properly, the user specified in the "Log on as" portion of the **Service** tab must be a member of the "PI Buffering Administrators", or the "PI Buffer Writers" user groups. A virtual service account can be added to those local groups, just like with any local/domain account. Failure to add a user to at least one of these user groups could result in a failure to buffer data. For more details, see the [PI Buffer Subsystem](#) user guide.

For more information about managing Data Archive security and trusts, see the *PI Server System Management Guide*.

Authentication and authorization

In order for the interface to connect to Data Archive, the interface service account must be authenticated and authorized for communications. This requires configuring security for the applications that connect to Data Archive. For more information about authentication and authorization for interfaces, see the PI Universal Interface (UniInt) User Guide.

Note: When configuring PI users and PI groups for the interface, avoid using the piadmin super-user or the piadmins group. These built-in users and groups have high-level privileges that can pose security risks.

PI point security and allowlist files

PI point security can be configured for user access to the point attributes using the *ptsecurity* attribute, and user access to the point data using the *datasecurity* attribute.

For output points, the *zero* and *span* attributes can be used to specify the minimum and maximum values allowed by the output point.

For all interfaces that can write to the data source, AVEVA recommends configuring a file of allowed output points from Data Archive to the data source. Create a *allowlist* of output points to specify authorized point updates to the data source. When using an allowlist, the interface verifies an output point against the list before updating the data source. The allowlist file is a comma-separated values (**.csv**) file that contains a list of valid output points and any attributes necessary to specify the output points and their intended location within the data source. Use an empty allowlist file if you do not need outputs to the data source. Enable use of the allowlist using the `/whitelist=path/filename` startup parameter.

For more information about configuring the allowlist file and the startup file parameter to enable the allowlist file, see [Learn about allowlist file configurations](#) or the *PI Universal Interface (UniInt) User Guide*.

DCOM security

OPC server and client applications are based on Microsoft's COM/DCOM communication model. For an overview, see [DCOM configuration](#).

For more information about DCOM security for PI OPC products, see the *DCOM Security and Configuration Guide*.

Create trusts

When creating trusts, you have many options. Following is a simple and secure approach, creating a trust for the following applications:

- PI Interface for OPC DA
- PI Interface Configuration Utility (PI ICU)
- Buffering

Note: AVEVA discourages using highly-privileged identities, users, or groups in PI trusts for interfaces. Avoid using the piadmin super-user and piadmins group. The recommended best practice for Data Archive security is to create an identity, user, or group that has only the access rights which are necessary for the interface, PI ICU, or buffering to operate.

To create each of these trusts using PI System Management Tools, connect to the Data Archive server and perform the following steps:

1. Select **Security** and choose **Mappings & Trusts**.
2. On the **Trusts** tab, right-click and choose **New Trust**.
The Add Trust wizard opens.
3. Specify a meaningful name and description for the trust.
4. Configure settings as follows:

Trust	Type	Application Name	Network Path	PI User
PI Interface for OPC DA	PI API application	OPCpE	Name of the interface node or IP address plus netmask 255.255.255.255	Identity with access rights to the PI points for the interface. Enabled by the <i>datasecurity</i> attribute.
PI ICU	PI API application	PI-ICU.exe	Name of the interface node or IP address plus netmask 255.255.255.255	Dedicated PI identity with the precise permissions required (database read access, read ptsecurity and read-write permission for OPC points)
Buffering	PI API application	Bufserv: APIBE PIBufss: Pibufss.exe	Name of the interface node or IP address, plus netmask 255.255.255.255	Buffering service identity with necessary access rights for the PI points for all interfaces on the interface node.

Configure the Windows service

To ensure that PI Interface for OPC DA restarts when the interface node is restarted, configure it as a Windows service.

Note: As a security defense, AVEVA recommends running the interface service under an account with minimum privileges, such as a Windows built-in service virtual account, the built-in Network Service account, or a non-administrative account that you create. For more information about security, see [Best practices](#).

Use PI ICU to install the interface as a service.

1. Launch PI ICU and select the **Service** tab.
2. Set the fields as described in the following table.

Field	Description
Service name	Descriptive name for the interface service.
ID	Numeric ID of the interface instance. Must be unique for each instance.
Display name	The service name displayed in the Windows Services control panel. The default display name is the service name with a PI- prefix. You can override the default. To ensure that AVEVA-related services are sorted together in the Services control panel, retain the PI- prefix.
Log on as	The Windows user account associated with the service. The user must have DCOM permissions configured on the OPC server. Set password expiration to Never.
Password	Password, if any, for the preceding user.
Dependencies	Any services that the interface depends on. The only dependency is the TCP/IP service, which is pre-configured. If buffering is enabled, you are prompted to create a dependency on the buffering service.
Startup type	Specifies whether the interface service starts automatically when the interface node is restarted. Generally, interface services are set to start automatically.

3. To create the service, select **Create**.
4. To start the service, select the  button.

Manage the PI Interface for OPC DA service

After you install the interface as a Windows service, use the procedures in this section to manage the service.

- To verify that the service is running, run **services.msc** from the Windows **Start** menu.
- To start and stop the service, use PI ICU.
- To remove the service using PI ICU, stop it and select **Remove**.
- To start the service interactively, use PI ICU and select **Interface > Start Interactive**.

Note: Interactive startup is usually done only for debugging.

DCOM configuration

Classic OPC server and client applications are based on Microsoft's COM/DCOM communication model. The type of communication depends on the network configuration:

- **Component Object Model (COM)**

Provides a set of interfaces that enable software components to communicate on a single computer.

- **Distributed Component Object Model (DCOM)**

Extends COM to enable software components to communicate between network nodes.

DCOM enables a process on one computer to request another computer to execute programs on its behalf, so permissions must be granted carefully to maintain security.

Note: DCOM permissions must be granted to the accounts under which the OPC server and PI Interface for OPC DA run, even if you are using the same node for both the OPC server and the OPC client.

For details, see [DCOM Security and Configuration](#).

DCOM security levels

Access to a COM server is governed by Windows security and controlled by access control lists (ACLs), which grant specific users or groups permission to use that server. In addition, system-level policies and settings determine how users are authenticated and how permissions are granted.

DCOM security is implemented on several levels:

- System-level ACLs, settings and policies define the minimum level of security for all DCOM components (the *Edit Limits* ACLs in **dcomcnfg** utility).
- Default ACLs and security levels are used if a DCOM component does not explicitly set a security level (the *Edit Defaults* ACLs in **dcomcnfg** utility).
- Custom ACLs and security levels can be specified for individual DCOM servers using the Windows **dcomcnfg** utility.
- Custom security can be implemented in code by the DCOM server (*CoInitializeSecurity*).

DCOM clients and servers

OPC servers are DCOM servers, and OPC clients are DCOM clients, but the roles are not fixed.

PI Interface for OPC DA retrieves data from the OPC server using asynchronous callbacks. During a callback, PI Interface for OPC DA acts as a DCOM server, while the OPC server acts as a DCOM client. For this reason, DCOM security on the interface node must be configured to allow access by the account associated with the OPC server.

Windows domains and users

A Windows domain is a network of computers with a common security database. If the PI Interface for OPC DA node and the OPC server are members of the same domain (including domains with bi-directional trusts), domain users can be used in DCOM access control lists. If the interface and server reside in different domains,

you must configure identical local accounts on both machines and use those accounts in ACLs. These accounts must have the same user name and password, and password expiration for these accounts must be disabled.

If the interface and the OPC server are on the same computer, any account that is recognized by that computer can be used, including the SYSTEM built-in account. Some OPC servers implement custom security. In this case, the account must be granted access to the OPC server, apart from any DCOM permissions.

Computers in Windows workgroup configuration have default policies that interfere with OPC server/client communications, requiring you to disable simple file sharing.

DCOM user accounts

If the OPC server vendor has prescribed DCOM settings, do not change them unless it is not possible to communicate with the server using those settings. DCOM settings do not retain a history of the changes that were made. To ensure that settings can be reverted back their original settings, you should record the current settings before making any changes. Adjust the client settings to accommodate the server, rather than the other way around.

Access to DCOM servers is controlled by ACLs that specify user accounts and their associated permissions. When the client calls the server, the user account associated with the client process is authenticated, then the ACL is evaluated to determine if that account has permission to use the server.

For interactive clients such as PI OPC Client or PI Interface for OPC DA to run interactively, the effective user is the account that was used to log on to the computer. An interactive process can be run under a different account by using the *RunAs* command.

For client programs running as Windows services, the user is the account specified in the **Logon** tab in the Services control panel.

For the OPC server, the user is the account specified in the **Identity** tab in the DCOM configuration for the server. Windows provides several options for the identity used by a DCOM server:

- **Interactive user**

The account that is logged on to the console of the computer where the server is running. This setting is problematic for OPC communications: if no one is logged on to the console or the user logged on does not have DCOM permissions, the client cannot connect to the OPC server.

- **Launching user**

The server process runs under the same account as the calling client. Do not use this setting if multiple clients running under different accounts need to access the same OPC server, because a new instance of the OPC server is launched for each user. Note that the user ID of the calling client might not have permission to connect to the server, because many servers implement their own user authentication aside from DCOM permissions.

- **This user**

Recommended, unless the OPC server vendor specifies a different setting. Include the specified user in the default DCOM ACLs on the interface node. If the OPC server runs as a Windows service, use the same account as the logon account for the service.

- **System account (services only)**

Use only if the interface runs on the same computer as the OPC server.

Firewalls and security

DCOM relies on dynamically-assigned TCP ports. When an OPC client connects to an OPC server, it connects to port 135 (the RPC port mapper), which assigns one TCP port and one UDP port to the component. Communication between client and server is directed to those ports. Because of these limitations, it is difficult to configure DCOM to work through a conventional firewall.

Third-party vendors offer products that address these limitations. OPC tunnelers use a specialized OPC client that mirrors data to a specialized OPC server through an encrypted channel. OPC-aware firewalls enable secure communication with OPC servers with minimal configuration.

If third-party solutions are not desirable, secure OPC through configuration as follows:

- If the OPC server vendor supports it, install PI Interface for OPC DA on the machine running the OPC server. The local COM connection permits you to disregard firewall issues between client and server.
- If a separate interface node is required, locate the interface on the OPC server's subnet. It is much easier to open a single firewall exception to port 5450 on the Data Archive server than to configure DCOM to work through a conventional firewall.
- Configure DCOM permissions on a "least privilege" basis, by including only specific service accounts in DCOM ACLs. See [DCOM Security and Configuration](#) for more information.
- Use the built-in Windows firewall that is included in the Windows version you are using.

PI point configuration

The PI point is a time-stamped record of a single set of measurements (for example, tank temperature). Configure PI points to allow PI Interface for OPC DA to transmit the data from OPC server to Data Archive.

Note: To populate Data Archive with the points that are defined in your OPC server, use PI OPC Client Tool or OPCtoCSV to export OPC items to an Excel file (.csv), then use the PI Builder Microsoft Excel add-in to load the tags into the Data Archive. For details, see [Export OPC items to PI Builder](#).

If outputs to the OPC server are not needed, configure the interface instance to disable outputs from the Data Archive. If outputs are needed, configure an output point allowlist for the interface instance.

Note: Disabling outputs or configuring an output point allowlist defends against accidental or malicious changes to the control system. For more information, see [Best practices](#).

PI points for PI Interface for OPC DA require the following minimum point attributes to be configured:

- Point name (*tag*)
- Point source (*pointsource*)
- Data type (*pointtype*)
- Interface instance (*location1*)
- PI point type: advised, event, or polled (*location3*)
- Scan class (*location4*)
- OPC ItemID (*instrumenttag*)

Depending on the type of point you are creating, additional attribute configurations might be required.

Export OPC items to PI Builder

To populate Data Archive with the points that are defined in your OPC server, use PI OPC Client tool or OPCToCSV (installed in %PIPC%\PI OPC Tools\PI_OPCToCSV) to export OPC items to an Excel file (.csv), then use the PI Builder add-in to Microsoft Excel to load the points into Data Archive.

PI Interface for OPC DA no longer includes PI OPC Tools in the self-extracting installation kit. PI OPC Tools can be downloaded from the [AVEVA Customer Portal Products page](#). Search for PI OPC Tools Set (OPCTools) Install Kit.

Note: To permit PI Builder to create PI points, you must define a trust or configure permissions that enable Microsoft Excel to write to Data Archive.

To export OPC points and create the corresponding PI points, perform the following steps:

1. Start PI OPC Client tool and connect to your OPC server.
2. To select the OPC items you want to export, create a group (select ) and add the desired items to it.
3. Choose **File > Save As** and specify the name and location of the export file.
4. Select **Save**.
PI OPC Client tool creates a .csv file containing the OPC items you selected.
5. In Microsoft Excel, open the .csv file that contains the exported OPC items.
6. Start Microsoft Excel and select the PI Builder menu item.
7. Select the Data Archive using the dropdown next to the **Data Server:** Ribbon item.
8. Examine the generated entries to ensure that the desired points are listed. If any entries have Unknown in the **PointType** column, specify the desired data type for the point.
9. To generate the PI points, select the **Publish** Ribbon item.
The **Publish Options** dialog box opens.
10. Verify that the **Edit Mode** is Create and Edit, and then select **OK**.
The PI points will be created.

Create PI points manually

To build individual PI points manually, use Point Builder in PI System Management Tools (SMT). To start Point Builder, choose **Points > Point Builder**.

Configure point attributes

Configure input and optional output PI points for PI Interface for OPC DA using the following minimum attributes.

Ensure the following PI point attributes are configured:

- *pointsource*
Identifies all points that belong to this instance of the PI OPC interface. Specify the same **Point source** entered on the PI ICU **General** tab.
- *location1*

Specifies the OPC interface instance ID, which is displayed on the PI ICU **General** tab.

- *location2*

To enable handling for OPC servers that do not return certain numeric types in their native format, set *location2* to 1. Numeric data is returned as a string.

- *location3*

Specifies the point type. Options are: 0 (polled/event), 1 (advise), 2 (output), 3 (polled WatchDog), 4 (advise WatchDog).

Note: See [Input points](#) for a procedure for configuring the "0 (polled/event)" tag so that *location3* = 0 can be interpreted as either polled or event.

- *location4*

Specifies the scan class.

- *location5*

Optional deadband value for advise points.

- *exdesc*

Specifies event points, Long ItemID, Dzero for scaled points, or ItemID to get the time stamp for an output value.

- *instrumenttag*

OPC ItemID that corresponds to the PI point you are defining. Case-sensitive. To display OPC server items, use PI OPC Client Tool.

- *datasecurity*

For every output PI Point the interface instance services, the access control list in the *datasecurity* attribute must grant read access to the PI Identity in the PI trust that authenticates the interface instance. If no output PI Points are used, no read access is required in the *datasecurity* attribute.

- If the interface is used without a buffering application, write access must be granted.
- If the interface is used with a buffering application, the buffering application requires write access but the interface does not.

Note: When buffering is configured, the *datasecurity* attribute must permit write access for the buffering application's PI trust or mapping. The *datasecurity* write permission for the interface's PI trust is required only when buffering is not configured.

- *ptsecurity*

The access control list in the *ptsecurity* attribute of every PI point that the interface instance services must grant read access to the PI identity in the PI trust that authenticates the interface instance.

PI point attributes

Some PI point attribute configurations are specific to PI Interface for OPC DA points, while others are common to all points. For additional information about PI point configurations, see the *PI Universal Interface (Unint) User Guide*, and the *PI Server System Management Guide*.

Note: Data loss can occur for Event Points (*ExDesc* contains event='trigger_tag_name' condition) if the connection to the PI Server is lost. The interface signs up for events from Update Manager for the source points configured for the Event Points. When there is no connection, the source point data cannot be sent to PI, nor can updates be received from the Update Manager for the Event Points. Thus, Event Points will not be updated

during the time period when connectivity is lost, nor can data be recovered during the period of lost connectivity

Tag (PI point name)

The *Tag* attribute (or tag name) is the name for a point. There is a one-to-one correspondence between the name of a point and the point itself. Because of this relationship, Data Archive documentation uses the terms "*tag*" and "*point*" interchangeably.

Follow these rules for naming PI points:

- The name must be unique in the Data Archive.
- The first character must be alphanumeric, the underscore (_), or the percent sign (%).
- Control characters such as linefeeds or tabs are illegal.
- The following characters also are illegal: * ' ? ; { } [] | \ ` " "

Tag length

Depending on the version of the PI API and the Data Archive, this interface supports *Tag* attributes whose length is at most 255 or 1023 characters. The following table indicates the maximum length of this attribute for all the different combinations of PI API and Data Archive versions.

Supported tag length

PI API	Data Archive	Maximum Length
1.6.0.2 or later	3.4.370.x or later	1023
1.6.0.2 or later	Earlier than 3.4.370.x	255
Earlier than 1.6.0.2	3.4.370.x or later	255
Earlier than 1.6.0.2	Earlier than 3.4.370.x	255

PointSource (point source)

PointSource is an identifier that associates a PI point with a PI interface instance, enabling the interface to query the for the points that it updates. This field is not case- sensitive. In the interface (batch) startup file, the point source is specified using the */PS* command-line parameter.

The following point sources are reserved. Do not configure them for interface instances.

Reserved point sources

Point Source	Reserved By
T	Totalizer Subsystem
G and @	Alarm subsystem
R	Random interface

Point Source	Reserved By
9	RampSoak interface
C	Performance equations subsystem

Data type attribute

The data type (*pointtype*) attribute specifies what type of data is stored in the PI point.

Typically, OPC item data types do not need to match PI point data types exactly, but the data types must be compatible. For example, integer values from a device can be sent to floating-point or digital PI points. Similarly, a floating-point value from the device can be sent to integer or digital PI points, although the values might be truncated.

PI Interface for OPC DA supports all PI point types except BLOB.

Note: Some OPC servers lack support for the full range of PI point types. To determine which PI point types are supported by your OPC server, refer to the vendor-supplied documentation.

If the point type defined in Data Archive does not match the required data type defined in the OPC server, the interface attempts to translate the data. To determine whether the point can be read as the required type, use PI OPC Client Tool to try to read the point directly from the OPC server. For more information on data type compatibility, see [Data type compatibility](#).

Location1 (interface instance)

Location1 specifies the instance of the interface to which the PI point belongs. The value of the attribute must match the */id* command-line parameter if */id > 0*. If */id = 0*, the *Location1* parameter is not used and the interface will load all points matching the *PointSource*. This setting plus *PointSource* identify the interface instance that writes to a particular point. (*/id*).

Data type handling attribute (location2)

The *location2* attribute configures handling of data types for PI Interface for OPC DA.

Valid settings for *location2* are as follows:

Value	Description
0	Normal processing; no special handling is used.
1	Read and write value as a string. For digital points, the strings read from the OPC server must match the strings in the digital state set used by the PI point. For integer and real points, the interface requests a string value, and translates it to a number.
2	Read value as a Boolean. Boolean values are either zero or nonzero. For numeric points, any value but 0 (False) is set to -1

Value	Description
	(True). Use this option to correctly convert an OPC server Boolean value into the PI System digital state, which prevents the PI point from receiving Bad quality values for a Boolean value that is True.
3	Read value as a four-byte integer. This setting accommodates servers that cannot send the value as a two-byte integer, which is how digital points are normally read.
4	Stores the quality of the item rather than the value.
5	Request real points as VT_R8 items (eight-byte real). By default, the interface requests real points as VT_R4 items (four-byte real). For float32 points (including all PI2 Real points), values that cannot fit in a 32-bit floating-point number lose precision. This setting is included to support servers that do not translate VT_R8 data to VT_R4 data and to permit the use of float32 points where the benefit of greater precision is not worth the overhead of using float64 points.
6	Read time stamps from the OPC server as strings and transform them into seconds. The PI point can be an int or a float. The format of the time stamp string is specified in the interface batch file using the <i>tf</i> parameter.
7	Read time stamps from the OPC server as VT_DATE variables. These values can be translated into time stamp strings or passed to Data Archive as a number of seconds, suitable for use in computations. If the value is translated into a string, the format of the time stamp string is specified in the interface batch file using the <i>tf</i> parameter.
8	Directs the OPC server to send the canonical data type. The interface tries to transform the value into the proper data type for the PI point. Use with caution, because the transformation can fail if the source data type is not compatible with the PI point data type, or if the value cannot be represented using the PI point data type.

Value	Description
>= 1024	When a post-processing DLL is used with PI Interface for OPC DA, directs the data to be processed by the DLL. Adding any of the above settings (1-8) to 1024 enables those options to be used during processing. The Location2 tag attribute specifies what special handling should be done with the data, such as asking for a different data type than would normally be requested for a given tag type. When using certain OPC-DA plug-ins the value 1024 is added to whatever value Location2 already has. This allows most standard functionality of the interface to be maintained, and allows the OPC-DA plugin code to apply additional rules. Thus, if the tag is a Float32, but the OPC server can only provide the data if it has a VT_R8 (64-bit floating point number), and the interface is using the Time Array plug-in, Location2 would ordinarily be set to 5. To do the same thing for a tag that will receive an array of time-series data, set Location2 = 5 + 1024 = 1029.

Processing type attribute (location3)

The *location3* attribute specifies whether this PI point is a polled, advise, event, or output point. For definitions of these input points, refer to [Input points](#).

Location3	Description
0	Polled or event
1	Advise
2	Output
3	Polled watchdog used with server-level failover
4	Advise watchdog used with server-level failover

For an advise point, PI Interface for OPC DA registers for updates with the OPC server, and the OPC server sends new values to the interface. The update rate from the server does not exceed the update rate for the group.

Scan class attribute (location4)

The *location4* attribute configures the scan class for the PI point. The scan class determines the frequency at which input tags are scanned for new values.

For trigger-based points, set *location4* to zero. For all other points, the *location4* attribute must be a positive

number, representing the scan class number (scan class #) to which the PI Point should be assigned. The configuration of the scan classes is done on the interface itself.

For output tags, *location4* configures the output class. When necessary for load balancing, the interface distributes tags in scan class 1 across multiple OPC groups. Scan classes other than scan class 1 are assigned to separate groups for load balancing.

You can assign multiple groups to the same scan classes for PI Interface for OPC DA. For each scan class (with the exception of scan class 1), you can assign at most one polled group, one advise group, and one output group. The limits are as follows:

Tag	Maximum number of groups ¹
Polled	200
Advise	600
Event	199
Output	No maximum

¹ Refers to the actual maximum number of groups and not the maximum number of tags per group.

Note: We do not recommend mixing two groups within a single scan class. For scan class and grouping guidelines, see Knowledge Article 000001178 (<https://customers.osisoft.com/s/knowledgearticle?knowledgeArticleUrl=2700OSI8>)

The OPC standard does not guarantee that it can scan data at the rate that you specify for a scan class. If the OPC server does not support the requested scan frequency, the frequency assigned to the class is logged in the **pipc.log** file. If the interface workload is heavy, scans can occur late or be skipped. For more information on skipped scans, see the *PI Universal Interface (Unint) User Guide*.

For more information about setting the scan class, scan offset, and update rate, see [Scan class settings and update rates](#).

OPC deadband (location5)

Under the OPC standard, deadband processing is optional for servers. If the OPC server does not support deadband processing, the PI point is updated for all value changes to the point, depending on the exception attributes specified for the PI point.

Use a deadband to reduce the amount of network traffic from the OPC server to PI Interface for OPC DA. If the change between the last value read and the new value is less than the deadband, the OPC server does not send the value to the interface. For advise points, the *location5* attribute specifies a deadband value for analog OPC items.

Note: Before attempting to configure advise points, be sure that your OPC server supports deadband processing.

OPC server deadband processing is not the same as exception deadband processing that takes place between the interface and Data Archive.

To configure the OPC server deadband, specify a percentage of the exception deadband range, multiplied by 100. OPC server deadband only applies to *dweUType* analog items, which use the EU Low and EU High properties to calculate the range for the item. These OPC item properties correspond to the *zero* and *span* attributes in the PI point definition. For more information about OPC server deadband, refer to the OPC DA 2.05 specification.

For example, if the OPC server item is defined as analog with an EU Low of -10 and an EU High of 10, and

location5 is set to 2500 (25%), data is sent to the interface only when the difference between the new value and the old value is at least 5 (25% of 20 = 5). Exception deadband processing continues to be applied to the values received by the interface. The OPC deadband attribute (*location5*) setting only affects the values sent by the OPC server.

OPC ItemID attribute (*instrumenttag*)

The *instrumenttag* attribute maps the PI point to an OPC item. This field must exactly match the item name as defined on the OPC server, including any punctuation, spaces, and case.

To verify an ItemID, use the PI OPC Client tool. If the OPC server supports browsing, choose **List Server's Tags** to see a list of defined ItemID values. To display the full ItemID required for *instrumenttag* field, double-click the ItemID in the list

The maximum length of the *instrumenttag* attribute depends on the versions of Data Archive and PI API in use. If PI API is version 1.6.0.2 or later and Data Archive is 3.4.370.x or later, the maximum length is 1023. For all earlier versions, the maximum is 32. If you are running earlier versions and require more than 32 characters to specify the ItemID, you must enable the AF SDK or use the extended descriptor (*exdesc* attribute) to specify the OPC ItemID.

Extended descriptor attribute (*exdesc*)

The extended descriptor (*exdesc*) attribute is a multi-purpose field.

If you use the *exdesc* attribute to specify more than one setting, put a comma between the definitions.

By default, leading and trailing spaces are stripped from entries in the *exdesc* attribute. To preserve leading and trailing spaces, enclose your entry in double quotes.

PI Interface for OPC DA uses the *exdesc* attribute in the following ways:

- Event-based data collection

To define an event tag, set this attribute to *event=tag*. When the specified tag has an exception event, the tags for which it is the trigger are read from the OPC server.

- Dzero for scaled tags

When the device returns values that must be scaled to fit the range of values stored by the tag, store the device zero in the *exdesc* attribute. The format for specifying the device zero is *Dzero=nnnnn.nnn*. To specify the device span, use the *convers* attribute.

- Target OPC item for output tag time stamp

To direct the time stamp of an output point to an OPC item, specify the target ItemID in the *exdesc* attribute. The format of the time stamp depends on the data type of the ItemID that is to receive the time stamp, as follows:

- Tim=ItemID

Setting the Tim option causes the time stamp to be written as a string (*VT_BSTR*), formatted for the interface instance using the time stamp format (*tf*) parameter in the batch file. The time stamp comes from the Data Archive server and is not adjusted for differences in time zone or daylight savings time setting.

- Dat=ItemID

Setting the Dat causes the time stamp to be written as a *VT_DATE*. *VT_DATE* is an automation type that

does not specify time zone. *VT_DATE* values are usually treated as local time.

In error messages related to this time stamp ItemID, the interface reports a generated tag name of the form *TS:pointname*, where *pointname* is the name (*tag* attribute) of the output PI point.

- **OPC ItemID**

Because of limitations to the maximum length of the *instrumenttag* attribute, you might need to specify the OPC ItemID in the *exdesc* attribute.

Use the syntax *instr=ItemID*, where *ItemID* exactly matches the ItemID defined on the OPC server. If the *ItemID* contains a comma or space, enclose it in double quotes.

If PI API is version 1.6.0.2 or later and Data Archive is 3.4.370.x or later, the maximum length of the *instrumenttag* attribute is 1023 characters. For all earlier versions, the maximum is 32. If you are running earlier versions and require more than 32 characters to specify the ItemID, you must enable the AF SDK or use the *exdesc* attribute to specify the OPC ItemID.

SourceTag

For output tags (points that write data to the point source), this attribute specifies the PI tag that triggers query execution and from which the data is read (unless the output tag's query uses placeholders to derive the data from other tags).

Scaling code attribute (totalcode)

The *totalcode* attribute contains a code that specifies how the value is to be scaled.

The *totalcode* attribute is used in conjunction with the *squareroot*, *convers*, and *exdesc* attributes. For more information, see [Transformations and scaling](#).

Square root attribute (squareroot)

The *squareroot* attribute specifies that the square or square root of the value is to be used.

For more information, see [Transformations and scaling](#).

Conversion factor attribute (convers)

The *convers* attribute applies an offset or multiplier to a point value.

For scaled tags, the *convers* attribute contains the device span. For more information, see [Transformations and scaling](#).

The OPC item can have deadband values that correspond to the *zero* and *span* attributes, which define the actual span of values that the OPC server sends. PI Interface for OPC DA can use these OPC deadband values to translate the units used by the device to the units defined for the PI point. For more information, see [OPC deadband \(location5\)](#).

OPC array index attribute (userint1)

The *userint1* attribute maps a PI point value to an element in an OPC array item. Ensure that all PI points that are mapped to the same OPC array have identical settings for *instrumenttag*, *exdesc*, and all location attributes.

For PI points that are not mapped to an OPC array, set *userint1* to 0 (zero).

An OPC array item contains multiple values plus a single time stamp and a quality field. These items can be identified by using the PI OPC Client tool to read the item and examining the data type returned by the OPC server. If it is an array item, the type of the value is *VT_ARRAY* / *VT_other*, where *VT_other* is a data type such as *VT_R4* or *VT_I2*. The values in the array are sent as one data item and they all have the same data type.

Data Archive does not support PI points with an array type, so values must be assigned to a number of individual PI points. The first value in the array maps to the PI point that has *userint1* set to 1, the second to the tag with *userint1* set to 2, and so on.

If these values need to be processed as different data types, use the *location2* attribute for the PI point with *userint1*=1 and the settings for scaling and transformation for each individual point to configure how PI Interface for OPC DA handles the individual value.

PI Interface for OPC DA receives the data using the data type specified by the *location2* value for the point with *userint1*=1, then processes the value according to how the individual point is configured. Note that some servers cannot provide array data using any data type other than the canonical, or native, data type (the one displayed in the PI OPC Client tool if you omit data type). For those servers, you must either use a PI point with the correct data type, or set *location2* to 8 to configure the interface to ask for the canonical data type. For maximum efficiency, always request the canonical data type.

Event group attribute (*userint2*)

The *userint2* attribute assigns an event group to an event point. For points that are not event points, set *userint2* to 0 (zero).

For more information, see [Event points](#).

Scan

By default, the *Scan* attribute has a value of 1, which means that scanning is turned on for the point. Setting the *Scan* attribute to 0 turns scanning off. If the *Scan* attribute is 0 when the interface starts, a message is written to the log and the point is not loaded by the interface.

There is one exception to the previous statement.

If any PI point is removed from the interface while the interface is running (including setting the *Scan* attribute to 0), Scan Off is written to the PI point regardless of the value of the *Scan* attribute. Two examples of actions that would remove a PI point from an interface are to change the point source or set the *Scan* attribute to 0. If an interface-specific attribute is changed that causes the point to be rejected by the interface, Scan Off is written to the PI point.

Shutdown

The *Shutdown* attribute is 1 (true) by default. The default behavior of the PI Shutdown Subsystem is to write the *SHUTDOWN* digital state to all PI points when PI is started. The timestamp that is used for the *SHUTDOWN* events is retrieved from a file that is updated by the Snapshot Subsystem. The timestamp is usually updated every 15 minutes, which means that the timestamp for the *SHUTDOWN* events will be accurate to within 15 minutes in the event of a power failure. For additional information on shutdown events, refer to Data Archive manuals.

Note: The *SHUTDOWN* events that are written by the PI Shutdown Subsystem are independent of the

SHUTDOWN events that are written by the interface when the following command-line parameter is specified:
/stopstat=Shutdown

SHUTDOWN events can be disabled from being written to PI points when the Data Archive is restarted by setting the *Shutdown* attribute to 0 for each point. Alternatively, the default behavior of the PI Shutdown Subsystem can be changed to write *SHUTDOWN* events only for PI points that have their *Shutdown* attribute set to 0. To change the default behavior, edit the **Shutdown.dat** file, as discussed in the Data Archive manuals.

Bufserv and PIBufss

It is undesirable to write shutdown events when buffering is being used. Bufserv and PIBufss are utility programs that provide the capability to store and forward events to a Data Archive, allowing continuous data collection when the Data Archive is down for maintenance, upgrades, backups and unexpected failures. That is, when the Data Archive is shutdown, Bufserv or PIBufss will continue to collect data for the interface, making it undesirable to write *SHUTDOWN* events to the PI points for this interface. Disabling Shutdown is recommended when sending data to a Highly Available Data Archive collective. Refer to the Bufserv or PIBufss manuals for additional information.

Exception reporting attributes

To minimize the data that is stored and transmitted over the network, you can use exception reporting to filter out value changes that are not meaningful. Filtering takes place at the interface node before data is sent to Data Archive.

- *Maximum exception reporting interval (excmax)*
This attribute configures the maximum time period allowed between sending values to Data Archive.
This setting applies to both advise and polled tags. For advise tags, if PI Interface for OPC DA does not receive a value after the specified number of seconds and does not detect a dropped connection, it sends the last value received to the Data Archive with the time stamp set to the last time stamp plus *excmax* time. For polled tags, the interface sends a value to Data Archive if it has not sent one in the last *excmax* seconds, even if the new value does not pass *excdev* tests.
- *Minimum exception reporting interval (excmin)*
This attribute configures the minimum time period between values sent to Data Archive.
- *Exception deviation (excdev)*
This attribute configures the minimum change from the last value sent to Data Archive required for PI Interface for OPC DA to send a new value.

Users can disable exception handling altogether for a given point by setting *ExcMax=0*. For advise points in particular, this setting prevents the interface from re-sending old data with a new timestamp.

For more detailed information about *ExcMax* for advise tags, see [Appendix A. Set ExcMax on advise and event points](#).

Scan class settings and update rates

Scan classes specify how often a data source is checked for new data. The *f* parameter in the **.bat** file defines one or more scan classes.

Syntax: `/f=scan-interval`

Specify scan interval and optional offset using the following format: *time value,time offset,L*

Scan class time and offset time values use the following format: *HH:MM:SS.##*, where:

- *HH* is hours
- *MM* is minutes
- *SS* is seconds
- *##* is hundredths of seconds (01 to 99)

If you omit *HH* and *MM*, the scan period is assumed to be in seconds. For example, `/f=00:01:00,00:00:05` is equivalent to `/f=60,5`.

The *L* value associates the time unit value with the local computer time-of-day settings. Time-of-day scans use a 24-hour clock and are automatically adjusted for daylight saving time. This is also known as wall clock scheduling. The following example uses a scan frequency of 24 hours (once a day), an offset of eight hours from midnight (8 AM), and enables time-of-day scanning by specifying the *L* value: `/f=24:00:00,08:00:00,L`

The scan class frequency determines how often to scan for data. The offset specifies how long after the frequency's time value to scan for data. To balance the scanning workload, specify an offset for scan classes that have the same interval so that they are not scanned simultaneously.

For each scan class that you want to define, specify the *f* parameter. Scan classes are numbered by the order in which you define them. Scan class 1 is the first defined *f* parameter, and scan class 2 is the second *f* parameter you define.

Examples:

- Scan class 1 has a scanning frequency of one minute with an offset of five seconds, and scan class 2 has a scanning frequency of seven seconds with no offset:

`/f=00:01:00,00:00:05 /f=00:00:07`

- Two scan classes, defined using only seconds:

`/f=60,5 /f=7`

- Sub-second scan classes defined as decimal values:

`/f=0.5 /f=00:00:00.1`

Note: Deleting scan classes or changing their order can adversely affect the operation of existing PI points, which are closely related to scan rates. Scan classes should be adjusted only by PI System administrators who are fully aware of the PI System configuration and the effects of any such changes.

For more information about scan classes, see the *PI Interface Configuration Utility (PI ICU) User Guide*.

Scanning offsets

To mitigate the interface and OPC server workload, you can use the offset to stagger scanning. If an offset is specified, scan time is calculated from midnight on the day that the interface was started, applying any offset specified. In the first scan class in the above example, if the interface was started at 05:06:06, the first scan occurs at 05:07:05, the second scan at 05:08:05, and so on. If offset is omitted, scanning is performed at the specified interval, regardless of clock time.

Offsets determine when the interface asks the OPC server for the current values for polled classes. They do not

control the behavior of the OPC server, and have no effect on advise classes unless the *ga* parameter is specified to stagger the activation of groups. In this case, the offsets are used to time the activation of all groups except for scan class 1, which is reserved for advise tags.

Update rates

The OPC server reads data from the device according to the update rate for the group in which the item resides. By default, the update rate is the same as the scan rate.

To override the default using PI ICU, browse to the **OPCInt > OPC Server > Advanced Options** page and enter your update rate value for the *ur* parameter in the **Update Rates** area.

For polled groups, configuring an update rate that is shorter than the scan period ensures that the interface is receiving current data. For example, if the scan period is five seconds but the update rate is two seconds, the data is no more than two seconds old when it is read. However, note that a faster update rate increases the OPC server workload.

For advise groups, assign identical update and scan rates, except in cases where phase 1 Unilnt failover is configured for the interface. In this case, to ensure that the interface sees new values for failover heartbeat tags as soon as possible, set the update rate to half the scan period. This configuration reduces the risk of control needlessly switching back and forth. Dedicate a scan class with faster update rate to the failover heartbeat points.

Note: AVEVA recommends using phase 2 Unilnt failover. For more information about converting from phase 1 to phase 2 Unilnt failover, see the *PI Universal Interface (Unilnt) User Guide*.

Input points

PI Interface for OPC DA uses the following three types of input points for collecting data from OPC DA data sources:

- Advise points

Input is collected and stored in PI points when the OPC server updates item values in the server cache, based on the scan class configured for the PI point. The interface updates PI points when advised that a value changed.

- Event points

Input is stored in PI points when the interface is notified of an updated trigger point. The interface reads the event points, which can be associated with an event group, from the OPC server.

- Polled points

Input is collected and stored in PI points based on the scan class configured for the point and the update rate of the OPC server. The interface polls the OPC server at a regular rate.

All three types of points are received asynchronously by the interface.

Advise points

Advise points are sent to PI Interface for OPC DA by the OPC server only when a new value is read into the OPC server cache. Scan class 1 is reserved for advise points, and you can create additional scan classes for advise points as required. Ensure that the scan class rate is fast enough to capture all the changes from the data source.

If assigned to scan class 1 (Location4 =1) , the interface assigns the tags to OPC groups with a default of 800 tags per OPC group. For best performance, ensure that your OPC group size does not exceed 800 items per group.

Note: Your server might perform better with smaller group sizes; a limit of 200 points per group has proven effective with a number of OPC servers.

To change the default limit for advise-based PI points in a scan class, use PI ICU to set the **Number of Tags** in the **advise group** field on the **OPCInt > Data Handling** page.

Configure the following PI point attributes to create advise points:

- *location3*
Set to 1 for advise points.
- *location4*
Assigns the scan class for a point; scan class 1 is reserved for advise points.

Note: Do not assign the same scan class to both advise and polled points. Use a separate class for each point type.

Event points

Event points are read by the PI OPC interface when it receives notification that a trigger point has a new event. Event PI points are configured with a trigger PI point. When the trigger point receives a value, the event point is read.

Note: Data loss can occur for Event Points (*ExDesc* contains event='trigger_tag_name' condition) if the connection to the PI Server is lost. The interface signs up for events from Update Manager for the source points configured for the Event Points. When there is no connection, the source point data cannot be sent to PI, nor can updates be received from the Update Manager for the Event Points. Thus, Event Points will not be updated during the time period when connectivity is lost, nor can data be recovered during the period of lost connectivity

Frequent device reads can impair the performance of the OPC server. By default, the server is requested to update its cache every second for every event point defined. You can set reads to different update rates depending on the OPC server version:

- OPC v2.0 servers always read event points from the device, not the cache. To reduce system resource overhead from cache updates to the OPC server, set the event rate (*er*) parameter to a high value, such as eight hours.
- For OPC v1.0a servers, asynchronous reads come from the cache. The internal cache update interval needs to be fast enough to ensure that data in the cache is not stale.

For any asynchronous read, the OPC server is required to return all of the values together, which can delay the return of new values to Data Archive if the OPC server encounters a delay in reading the values. To improve performance in this case, group points according to the device where the data originates.

Configure the following PI Point attributes to create event points.

- *location3*
Set to 0 for event points.
- *location4*
Assigns the scan class for a point. Set to 0 for event points.

- *userint2*

For each OPC event group, configure this attribute as the same integer for each PI point in the group. Assigning PI points to OPC event groups ensures that the points are read together. For example, a plug-in DLL that post-processes data might require the data to be sent in a single group.

- *exdesc*

For each OPC event group, configure the *exdesc* attribute using the same name of the triggering point. Use the syntax *TRIG='trigger_point_name' event_condition*. Use single quotes. To treat all changes as triggering events, omit *event_condition*.

See the following table for a list of event descriptions:

Event Condition	Description
Anychange	Trigger on any change as long as the value of the current event is different than the value of the previous event. System digital states also trigger events. For example, an event will be triggered on a value change from 0 to "Bad Input," and an event will be triggered on a value change from "Bad Input" to 0.
Increment	Trigger on any increase in value. System digital states do not trigger events. For example, an event will be triggered on a value change from 0 to 1, but an event will not be triggered on a value change from "Pt Created" to 0. Likewise, an event will not be triggered on a value change from 0 to "Bad Input."
Decrement	Trigger on any decrease in value. System digital states do not trigger events. For example, an event will be triggered on a value change from 1 to 0, but an event will not be triggered on a value change from "Pt Created" to 0. Likewise, an event will not be triggered on a value change from 0 to "Bad Input."
Nonzero	Trigger on any non-zero value. Events are not triggered when a system digital state is written to the trigger tag. For example, an event is triggered on a value change from "Pt Created" to 1, but an event is not triggered on a value change from 1 to "Bad Input."

The PI point that triggers the read is a separate input point than the event points. When a new event for a trigger point is sent to the PI Snapshot, the PI System notifies the interface, which then reads the values for all the associated event points from the OPC server.

For v1.0a servers, an asynchronous read is sent to the server's cache. For v2.0 servers, the interface performs an asynchronous read from the device.

For more information about configuring input trigger points, see the *PI Universal Interface (Unilnt) User Guide*.

For efficiency with v1.0a servers, separate event points into groups based on the triggering event. For OPC v2.0 servers, separate event points according to the data source. The OPC v2.0 standard requires that all asynchronous reads originate from the device rather than from the server's cache, so set the cache update rate high and do not group values that come from different devices.

The following table illustrates PI points configured for separate event points.

Tag	ExDesc	Instrume ntTag	Location 1	Location 2	Location 3	Location 4	Location 5	UserInt1	UserInt2
PM1_Temp.PV	TRIG=PM1_Trigger	ItemID1	1	0	0	0	0	0	1
PM1_Rate.PV	TRIG=PM1_Trigger	ItemID2	1	0	0	0	0	0	1
PM2_Temp.PV	TRIG=PM2_Trigger	ItemID3	1	0	0	0	0	0	2

In the preceding example, PM1_Trigger and PM2_Trigger are points that are updated either by this interface instance, another interface, or by manual entry. When PM1_Trigger gets a new event in the snapshot, the interface sends the OPC server a *read* command that requests data for both PM1_Temp.PV and PM1_Rate.PV. Both values are returned in a single call. Likewise, when PM2_Trigger gets an event in the snapshot, the interface requests a value for PM2_Temp.PV.

Polled points

Polled PI points are grouped by scan class. If possible, the interface reads groups using the scan class rate configured for the point. Configure scan class rates using PI ICU.

Note: The OPC server does not guarantee that scan rates match between the OPC server and the interface. PI Interface for OPC DA sends a request to the OPC server to use an update rate that matches the scan class, but the OPC server determines its own update rate for scanning its data sources. The scan class offset has no effect on the OPC server, unless the interface is configured for staggered group activation and the OPC server uses the activation of the group to initiate the scanning cycle.

For more information about polled points, see the OPC Foundation document *Data Access Custom Interface Standard v2.05a*.

Configure the following PI point attributes to create polled points:

- *location3*
Set to 0 for polled points.
- *location4*
Assigns the scan class for a point.

Note: Do not assign the same scan class to both advise and polled points. Use a separate class for each point type.

Learn about output point configurations

Output points send data from Data Archive to the OPC server. Note that only good values can be sent. System digital states are not sent to OPC items.

To configure an output point, edit the point using PI Point Builder and specify the following settings:

- Set *location1* to the *id* parameter value of the interface instance.
- Set *location3* to 2.
- Specify the ItemID (the OPC item to be written) for the *instrumenttag* attribute.
- Optional: Specify the source point (the PI point that contains the value to be written to the OPC server) in the *sourcetag* attribute. Not required if you intend to send the value directly to the output item without copying the values and time stamps to a PI point.
- Optional: Set *location4* to the desired output group. Output points with *location4* set to 0 are distributed across output groups for load balancing.

You can trigger an output to the OPC server from the interface using the input point that stores OPC server data, or from a separate PI point configured to collect output values and the time stamp for the write. To configure output points, use one of the following methods:

- To use the same PI point as the source and the output point, leave the *sourcetag* attribute blank. Any means of updating the snapshot value of the output point is acceptable. Any new value is sent to the OPC item.
- To trigger the output to the target OPC item, the time stamp must be more recent than the previous time stamp, regardless of whether the value changes. The value that you enter into the output point's snapshot is written to the target item on the OPC server.
- To configure the output point using a separate source point, set the *sourcetag* attribute to the name of another PI point that will contain the values that you want written to the OPC item.

When the source point is successfully updated, the new value is written to the target OPC item. If the PI OPC interface succeeds in updating the OPC item, it writes the value and time stamp to the output point. If the interface does not succeed in updating the OPC item, it writes a digital state that describes the error to the output point. For output points, a success status indicates that the OPC server item has been updated, but there is no guarantee that the corresponding data source has been updated. To verify that the data source has been updated, create a corresponding input point and add logic to ensure that the values of the input and output points match.

The *pointsource* attribute of the output point must match the point source (*ps*) value of the interface instance, but the source point can be associated with any point source. The data type of the source point must be compatible with that of the output point.

Learn about allowlist file configurations

For enhanced security, create an allowlist of output points and enable the *allowlist* startup parameter for the interface. When this feature is enabled, the interface verifies that an output point is in the allowlist before writing to an OPC data source.

The allowlist file is a comma-separated values (.csv) file that contains a list of approved output PI points and the attributes that associate the PI point with a writable value in an OPC device.

For this interface, include the following required attributes in the allowlist file to secure the output points:

- tag
- instrumenttag
- location3
- userint1

The allowlist feature also supports range checking of output values. Include the *zero* and *span* attributes to specify a secure range of values to send to the OPC data source. These attributes specify the minimum (*zero*) and maximum (sum of *zero* and *span*) allowed for the output value.

For additional information on creating and configuring and enabling an output point allowlist file, see the *PI Universal Interface (Unilnt) User Guide*.

Learn about interface outputs to OPC servers

When the interface writes a value to the OPC server, the interface waits for an acknowledgement (ACK) from the server, and monitors acknowledgements of writes.

To speed up processing of outputs, you can control output processing in the following ways:

- Configure multiple output groups
- Specify the number of outstanding writes a group is permitted
- Specify the amount of data to be sent in each write

To assign an output point to an output group, set its *location4* attribute to the group number. For load balancing, output points with *location4* set to 0 are distributed across output groups, including groups to which output points are explicitly assigned.

The OPC server is not required to accept more than one write at a time from any group, but many servers permit multiple writes to be sent without waiting for the first one to be acknowledged.

Even if the server accepts only one write at a time, defining multiple output groups can improve throughput. If you specify more outstanding writes than the OPC server can accept, PI Interface for OPC DA reduces its setting to the OPC server maximum.

You can specify how long to wait for the OPC server to acknowledge a write. If no acknowledgement is received in the specified period, the interface cancels the write and reissues it.

If your OPC server does not acknowledge writes, you can create an alert using the Device Status health point. Configure the alert to detect a desired number of queued writes. When the specified level is reached, the alert sets an alarm state and drops a specified number of values, oldest or newest.

Data type compatibility

PI Interface for OPC DA supports all PI point data types collected in your PI Data Archive, including string. The only data type the interface does not support is BLOB.

The data type of a PI point must be compatible with the data type of the corresponding OPC item. For example, if a string value from the OPC server is to be put into an Int32 PI point, the string must contain a number. If a 64-bit floating-point value is to be put into an Int16 point, its value must not overflow the target data type.

PI Interface for OPC DA specifies the desired data type when requesting information from the OPC server and the OPC server is responsible for delivering the requested data type if it can.

PI Interface for OPC DA normally requests values using the following default data types:

PI System data type (pointtype)	OPC data type
Digital	Two-byte Integer (VT_I2)
Int16	Two-byte Integer (VT_I2)
Int32	Four-byte Integer (VT_I4)
Float32	Four-byte Float (VT_R4)
Float64	Eight-byte Float (VT_R8)
String	String (VT_BSTR)

For additional details on the PointType values, see [Data Archive Administration](#).

If your OPC server does not permit clients to specify a data type, set *location2* to 8 for all your OPC-based PI points to configure the interface to request the canonical, or native, data type from the OPC server.

Note: PI Interface for OPC DA might receive data for which no reasonable conversion is possible. Where possible, always specify the OPC data type that matches the PI point.

Int16 and Int32 point types

If you use a Int16 point type and expect negative numbers from the OPC server, this does not occur because this point type is an unsigned 15 bit integer. You should use the Int32 data type instead.

Additionally, the Int32 point type does not support the full range of Int32 values. The lowest 32,767 values of the 32-bit range are reserved. If you expect values in the reserved range (-2147483647 to -2147450880) from the OPC server, you should use a float point type instead.

OPC numeric strings

Some OPC servers return certain numeric data types only as strings. To interpret string-formatted Int16, Int32, Float32, and Float64 values, set *location2* to 1. PI Interface for OPC DA requests the data as a string (VT_BSTR) and interprets the data as a number.

Digital PI points contain integer values that correspond to specific strings in the digital state table in the point's digital set property. Some devices read and write the string value rather than the integer value. To read and write digital points as string values, set *location2* to 1. Make sure that the strings used by the OPC server are identical to the strings in the digital set, including punctuation and spaces. For optimal performance, read digital points as numbers whenever possible.

Boolean values

Some OPC servers send Boolean values as 0 and -1 when read as integers. This approach creates a problem when reading that data into a digital PI point, because "-1" is not the value that must be stored. To handle the data from such servers, the interface uses the absolute value of any integer or real values read for digital points.

Because digital point values are actually offsets into the digital set for the point, and a negative offset has no functional meaning, this issue does not cause problems for properly-written servers.

PI Interface for OPC DA can also request the item as a Boolean (VT_BOOL). This approach only works for items that have two possible states, because any non-zero value is interpreted as 1. To have items read and written as though they were Boolean values, set *location2* to 2.

Four-byte integers

If your OPC server does not support the two-byte integer (VT_I2) data type, you can configure PI Interface for OPC DA to request the data as a four-byte integer (VT_I4) by setting *location2* to 3.

Float64 values

To handle eight-byte floating-point numbers (VT_R8), set the *location2* of the target point to 5. Data Archive stores the value as a four-byte floating-point number, with possible loss of precision. If the number is too large to fit in the point, a status of BAD INPUT is stored.

Time stamp adjustments

PI Interface for OPC DA does not adjust the time stamps it receives, regardless of the time zone settings or *ts* parameter specified in the batch file. Any scaling or transformation is performed after the string has been translated into seconds, which enables a wide range of values to be handled.

There are two different types of time stamps:

- Windows type time stamp
This time stamp type accompanies item values and is defined as a UTC time.
- VT_DATE or other variant type (VT_*)
This is an item value type that can represent a date, time, or combination of the two. Items that contain a VT_DATE value are interpreted as local time with no daylight savings adjustments. Item values that contain a VT_BSTR (string) representation of a date or time have no specification for the presumed time zone; the OPC server has the option of local time or UTC.

Time stamp to seconds

To store a time stamp string (VT_BSTR) as seconds, set *location2* to 6.

If the PI point is an integer, PI Interface for OPC DA attempts to translate the time stamp into whole seconds. Because Int16 points can only hold numbers up to 32767, use Int32 points for time spans longer than nine hours.

If the PI point has a floating-point data type, the time stamp is translated into seconds and stored as a floating-point number.

Time stamps as VT_DATE data types

The OPC standard allows the VT_DATE data type, which is an internal representation of a time stamp. PI Interface for OPC DA translates between VT_DATE and integer, float, or string points. The interface does not adjust the time stamps received, regardless of the time zone settings.

To configure PI Interface for OPC DA to use the VT_DATE data type for reading the value from the OPC server or for writing the value to output points, set *location2* to 7.

For string points, the time stamp format of the string must be specified using the *tf* command-line parameter.

Time stamp strings

To configure the format of the time stamp sent by the OPC server using PI ICU, go to the **OPCInt > Data Handling** page and specify the format in the **Format of Timestamp Strings** field using the following tokens:

Time stamp token	Description
<i>cc</i>	Two-digit century
<i>yy</i>	Two-digit year
<i>mn</i>	Two-digit month
<i>mon</i>	Three-character month (Jan Feb Mar, etc.)
<i>dd</i>	Two-digit day
<i>hh</i>	Two-digit hour from 0 to 23
<i>hr</i>	Two-digit hour from 0 to 12
<i>mm</i>	Two-digit minute
<i>ss</i>	Two-digit second
<i>000</i>	Three-digit milliseconds
<i>XM</i>	AM or PM

The position of the tokens and delimiters must specify the format of the time stamp string precisely. Examples:

Format String	Result
<i>ccyy/mn/dd hh:mm:ss.000</i>	1998/11/29 15:32:19.391
<i>dd mon, ccyy hr:mm:ss XM</i>	29 Nov, 1998 03:32:19 PM
<i>mn-dd-ccyy hh:mm:ss</i>	11-29-1998 15:32:19
<i>hh:mm:ss.000</i>	15:32:19.482

Only one format string can be specified for each instance of PI Interface for OPC DA. If more than one format of time stamp needs to be processed, configure additional instances of the interface with the required time stamp format string.

If you omit elements of the format strings, the defaults are as follows ("current" values are UTC):

Omitted format string element	Default
Day	Current day
Month	Current month
Year	Current year
Century	Current century

Note: If you specify only hours, minutes and seconds, the date defaults to January 1, 1970. To ensure accurate time stamps, be sure to specify all the elements of the time stamp format. If the OPC server returns a zero value for the day, month or year element, the interface applies the defaults described above, regardless of the format string you specify.

Transformations and scaling

You can configure PI points so that PI Interface for OPC DA performs transformations and scaling. Transformation and scaling are applied before the value is compared to the exception parameters for the point, so that the exception parameters are applied to the value that is to be sent to Data Archive rather than the raw value.

Transformation

To transform the value to another scale of measurement, to apply an offset or conversion factor, or to perform bit masking, configure the settings as shown in the following table.

If *squareroot* is set to 1 or 2, the square root or square of the value is calculated first, then the formula is applied.

Convers	TotalCode	SquareRoot	Dzero	Operation Input points	Operation Output points
0	0	1	No effect	$(\text{Value})^2$	$(\text{Value})^{0.5}$
		2	No effect	$(\text{Value})^{0.5}$	$(\text{Value})^2$
Non-zero	1	0	Defined	$[(\text{Value} - \text{Dzero}) / \text{Convers}] * \text{Span} + \text{Zero}$	$[(\text{Value} - \text{Zero}) / \text{Span}] * \text{Convers} + \text{Dzero}$
		1	Defined	$[(\text{Value})^2 - \text{Dzero}) / \text{Convers}] * \text{Span} + \text{Zero}$	$[(\text{Value})^{0.5} - \text{Zero}) / \text{Span}] * \text{Convers} + \text{Dzero}$
		2	Defined	$[(\text{Value})^{0.5} - \text{Dzero}) / \text{Convers}] * \text{Span} + \text{Zero}$	$[(\text{Value})^2 - \text{Zero}) / \text{Span}] * \text{Convers} + \text{Dzero}$
	2	0	No effect	$\text{Value} * \text{Convers}$	$\text{Value} / \text{Convers}$

Convers	TotalCode	SquareRoot	Dzero	Operation Input points	Operation Output points
		1	No effect	$(\text{Value})^2 * \text{Convers}$	$(\text{Value})^{0.5} / \text{Convers}$
		2	No effect	$(\text{Value})^{0.5} * \text{Convers}$	$(\text{Value})^2 / \text{Convers}$
	3	0	Defined	$(\text{Value} / \text{Convers}) - \text{Dzero}$	$(\text{Value} + \text{Dzero}) * \text{Convers}$
		1	Defined	$((\text{Value})^2 / \text{Convers}) - \text{Dzero}$	$((\text{Value})^{0.5} + \text{Dzero}) * \text{Convers}$
		2	Defined	$((\text{Value})^{0.5} / \text{Convers}) - \text{Dzero}$	$((\text{Value})^2 + \text{Dzero}) * \text{Convers}$
	4	0	Defined	$(\text{Value} - \text{Dzero}) / \text{Convers}$	$(\text{Value} * \text{Convers}) + \text{Dzero}$
		1	Defined	$((\text{Value})^2 - \text{Dzero}) / \text{Convers}$	$((\text{Value})^{0.5} * \text{Convers}) + \text{Dzero}$
		2	Defined	$((\text{Value})^{0.5} - \text{Dzero}) / \text{Convers}$	$((\text{Value})^2 * \text{Convers}) + \text{Dzero}$
	5	0	No effect	$\text{Value} + \text{Convers}$	$\text{Value} - \text{Convers}$
		1	No effect	$(\text{Value})^2 + \text{Convers}$	$(\text{Value})^{0.5} - \text{Convers}$
		2	No effect	$(\text{Value})^{0.5} + \text{Convers}$	$(\text{Value})^2 - \text{Convers}$
	6	No effect	No effect	Value AND Convers	Value AND Convers
	7	No effect	No effect	Value OR Convers	Value OR Convers
	8	No effect	No effect	Value = Value XOR Convers	Value = Value XOR Convers

Scaling

To configure scaling for an OPC-based PI point, set the *totalcode* and *squareroot* attributes of the point. The *convers* attribute specifies the span of the device. The *exdesc* attribute specifies the device zero (Dzero), where

the format is "Dzero=nnnnn.nn", including the quotes, where nnnnn.nn is a number, e.g. "Dzero=20". Using these values, the interface can translate a value from the scale of the device to the scale of the PI point. Scaling is only supported for numeric points.

For simple square/square root scaling, set *totalcode* and *convers* to zero. To configure how the value is stored, set *squareroot* as follows:

- To square a value before sending it to Data Archive, set *squareroot* to 1. For output values, the square root is calculated before it is written to the device.
- To send the square root to Data Archive and the square to the device, set *squareroot* to 2.

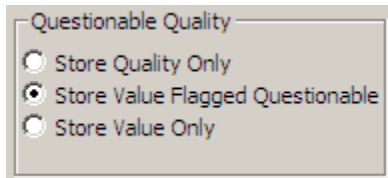
Data quality information

The OPC Data Access standard specifies a set of quality flags. PI Interface for OPC DA translates the quality flags to the closest approximation in the default System digital state set. The low eight bits of the quality flags are arranged into three fields, quality (QQ), sub-status (SSSS) and limit status (LL), arranged as follows: QQSSSLL.

Handling data of questionable quality

PI Interface for OPC DA stores either the quality or the value in a PI point, whereas the OPC server returns value and quality in separate fields. If a value is good, it is stored in the point. If a value is bad, a digital state that describes the quality is stored. For questionable-quality data, you can configure the interface to treat the values as good and store them, or treat them as bad and store a digital state. You cannot configure the interface to store a bad-quality value.

To configure handling of questionable-quality data using PI ICU, go to the **OPCInt > OPC Server** page and enable the desired option, as shown in the following figure.



Storing both values and quality information

To record both the values reported and the quality information returned with the values, store the quality in a separate PI point. To configure a point to store the quality for the associated ItemID, set *location2* to 4. Because OPC qualities are unsigned 16-bit integers, the PI Interface for OPC DA requires an Int32 tag to store them. The qualities are stored in Data Archive without any change, and their status is always *GOOD*. For details about OPC quality values, download the *OPC Data Access specification* from the [OPC Foundation](#) website or consult the *PI_OPCCClient User's Guide*.

Quality states

Quality data is composed of three sub-fields. The following tables list the values that are returned.

Good quality

Quality	OPC Definition	PI System Status
11SSSSLL	Non-specific	<i>Good</i>
Except: 110110LL	Local Override	<i>_SUBStituted *</i>

* These values will be marked as questionable, unless you use the following switches to ignore questionable values.

- If you use /SG=S, then "Local Override" is treated as good. However, it also suppresses bad and questionable values.
- /SQ=I writes questionable values but suppresses the questionable flag, so that they appear in PI as if they were good.

Not used by OPC

Quality	OPC Definition	PI System Status
10SSSSLL	Invalid	<i>Bad Input</i>

Questionable quality

Quality	OPC Definition	PI System Status
010110 LL	Sub-Normal	<i>Bad_Quality</i>
010101LL	Engineering Units Exceeded	
LL=01	Low Limited	<i>Under LCL</i>
LL=10	High Limited	<i>Over UCL</i>
Otherwise		<i>Inp OutRange</i>
010100LL	Sensor Not Accurate	
LL=01	Low Limited	<i>Under Range</i>
LL=10	High Limited	<i>Over Range</i>
Otherwise	Out of calibration (if not under or over range)	<i>Invalid Data</i>
010011LL	Invalid	<i>Bad Input</i>
010010LL	Invalid	<i>Bad Input</i>
010001LL	Last Usable Value	<i>No_Sample</i>

Quality	OPC Definition	PI System Status
010000LL	Non-specific	<i>Doubtful</i>

Bad quality (PI version 3.3 and higher)

Quality	OPC Definition	PI System Status
000111LL	Out of Service	<i>Out of Service</i>
000110LL	Comm Failure	<i>Comm Fail</i>
000101LL	Last Known Value	<i>Scan Timeout</i>
000100LL	Sensor Failure	<i>Equip Fail</i>
000011LL	Device Failure	<i>Unit Down</i>
000010LL	Not Connect	<i>Not Connect</i>
000001LL	Configuration Error	<i>Configure</i>
000000LL	Non-specific	<i>Bad</i>

Custom digital states

To replace the default PI System digital states with custom states using PI ICU, go to the OPCIntData Handling page and set the Alternate Digital State for Questionable/Bad Qualities field. To override the default states, you must specify the full set of replacements, and the numeric values must be contiguous.

The following table lists the digital states and PI statuses that you can override.

Order After Marker State	Default PI status
1	Bad_Quality
2	Under LCL
3	Over UCL
4	Inp OutRange
5	Under Range
6	Over Range
7	Invalid Data
8	Bad Input
9	No_Sample
10	Doubtful

Order After Marker State	Default PI status
11	Out of Service
12	Comm Fail
13	Scan Timeout
14	Equip Fail
15	Unit Down
16	Not Connect
17	Configure
18	Bad

Reading OPC quality into a digital PI point

To store OPC quality in a digital PI point, use transformations and scaling to translate quality to a digital state set of Bad Value, Questionable Value, Invalid Value, or Good Value.

To define such a PI point, set *location2* to 4 to read the quality of the item rather than its value, then define a mathematical transformation that translates the quality values to an integer from 0 to 3. Divide the quality number by a conversion factor that produces the proper number.

OPC quality is returned in ranges of values, as follows:

Range	Description
Less than 0x40	Bad Value
Greater than or equal to 0x40 and less than 0x80	Questionable Value
Greater than or equal to 0x80 and less than 0xc0	Not used by OPC
Greater than or equal to 0xc0	Good Value

Because each range has the same size (decimal 64), you can use a simple conversion to obtain the corresponding digital state, as follows:

Convers	TotalCode	SquareRoot	Dzero	Operation
Not 0	3	0	Defined	Input points: Value = (Value / <i>convers</i>) – Dzero Output points: Value = (Value + Dzero) * <i>convers</i>

Define the point attributes as follows:

Attribute	Setting
<i>convers</i>	64
<i>totalcode</i>	3
<i>squareroot</i>	0
<i>exdesc</i>	"Dzero=0"

OPC array item PI points

OPC servers can contain arrays of data, which are composed of multiple values of the same data type plus a single quality and time stamp. Because Data Archive does not support array data types, you must configure one PI point for each array element that you want to store, or use the TimeArray plug-in. For additional information, download and review the [PI Interface for OPC DA - Plug-In DLLs](#) documentation.

To define a PI point to contain an element from an OPC array, specify the ItemID of the array item in the *instrumenttag* attribute and set *userint1* to the index number of the element in the array. You must define a PI point for the first array element (*userint1* = 1), even if you do not require its data. However, you do not need to define points for all array elements, only for the first array element and any individual elements of interest.

If an array tag from the OPC server is read into multiple PI points, each PI point receives the value of the array element indexed by the *userint1* setting and the same time stamp and quality, because an array contains multiple values but only one time stamp and quality. To read an OPC array tag into a single PI point, you must use the TimeArray plug-in, which stores an array of values into a single PI point as successive data values, incrementing the time stamp that came with the array by a configured interval for each value. For details, refer to the TimeArray plug-in manual.

All PI points configured for an OPC array must have identical settings for *pointsource*, *location1*, and *location4* (and *userint2*, if they are event points). For advise points, set *location3* to 1. For polled points, set *location3* to 0.

When configuring PI points to read OPC arrays, note the following:

- You must define a point that reads the first array element.
- Assign the points to the same scan class.
- To optimize CPU usage, do not use the same scan class to read more than one OPC array.
- If you need to read the same OPC array element into more than one point, you must assign the points to different scan classes.

OPC arrays as event PI points

Multiple scan classes can have the same scan period, and event classes are a logical grouping of PI points. For efficiency, put event arrays into their own scan classes with any other points that need to be read with the array.

Configuring arrays that are read as event tags is complex: because only the first array item (with *userint1* = 1) causes a read, you must create a dummy trigger PI point to use with the rest of the array items. That PI point must have a *pointsource* that is either unused or used for manual entry points (lab data usually is entered manually, so L is often used as the *pointsource* for manual entry PI points).

In the following example, the trigger PI point is called TriggerTag and the dummy trigger PI point is called

DummyTrigger.

Tag	ExDesc	Instrume ntTag	Location 1	Location 2	Location 3	Location 4	Location 5	UserInt1	UserInt2
Array000 1.PV	TRIG=Tri ggerTag	Data.Arr ay	1	0	0	0	0	1	1
Array000 2.PV	TRIG=Du mmyTrig ger	Data.Arr ay	1	0	0	0	0	2	1
Array000 3.PV	TRIG=Du mmyTrig ger	Data.Arr ay	1	0	0	0	0	3	1

Because all the tags in an array must belong to the same group, even if the OPC server is v2.0 and some part of the array data comes from a different device than the rest of the array data, all the array tags must be configured to be in the same event group.

Failover configuration

PI Interface for OPC DA provides two methods for configuring failover, to ensure that data collection continues if either the interface or the OPC server fails:

- Unilnt failover

If one instance of PI Interface for OPC DA fails, another instance can take over data collection.

Note: Unilnt failover is a feature common to Unilnt-based interfaces. For more information about configuring Unilnt failover, see the [Introduction to PI Universal Interface](#).

- OPC server-level failover

If PI Interface for OPC DA stops receiving data from the currently-connected OPC server, it can switch to another OPC server and resume data collection.

If you are configuring both types of failover, configure failover in the following order:

1. Configure and verify Unilnt failover.
2. Disable Unilnt failover.
3. Configure and verify OPC server-level failover.
4. Re-enable Unilnt failover.

Windows Cluster failover is no longer recommended.

Unilnt failover modes

The failover mode specifies how the backup PI Interface for OPC DA instance handles connecting to an OPC server, creating groups, and adding points when failover occurs.

The faster the backup interface instance can take over data collection, the less data is lost. However, the ongoing activities required to maximize the readiness of the backup interface instance increase OPC server load and system resources. To determine which mode to use, consider how long failover takes and how much workload your system can handle. Be prepared to experiment, and consult your OPC server documentation and vendor as needed.

PI Interface for OPC DA provides five levels of failover, from cold to hot. Higher (“hotter”) levels ensure that more data is preserved in the event of failover, but impose increasing workload on the system. The highest level, hot failover, is lossless unless both the primary and backup interface instance nodes fail together.

Hot failover

Hot failover is the most resource-intensive mode. Both the primary and backup interface instances are collecting data, possibly from the same OPC server. No data is lost during failover, but the OPC server carries a double workload, or, if two servers are used, the back-end system must support both OPC servers.

Warm failover

There are three options for warm failover:

- No groups on backup interface instance (**Warm_1**)

The backup instance connects to the OPC server every 30 seconds and checks its status, but does not create groups or add items. Because the interface pre-loads point information from Data Archive, this option is faster than cold failover, but when the backup becomes the primary instance, it must create groups, add items to them, activate them, and then advise them. Because of the time required, data can be lost during failover. This option is for OPC servers that cannot support groups when they are not the active OPC server. The command line equivalent is `/UFO_TYPE=HOT /FM=1`.

- Inactive groups on backup interface instance (**Warm_2**)

The backup instance connects to the OPC server, creates inactive groups, and adds items to the groups, but does not activate the groups. For most OPC servers, this approach reduces workload, because the server does not need to maintain the current values for inactive groups. When the interface instance becomes primary, it activates the groups and then advises them. This approach is faster than option 1. The command line equivalent is `/UFO_TYPE=HOT /FM=2`.

- Active groups on backup interface instance (**Warm_3**)

The backup interface instance connects to the OPC server, creates groups, adds items, and activate the groups, but does not advise the groups. The OPC server must maintain its cache of current values for all the items, but does not send the values to the backup interface instance.

If both interface instances are connected to the same OPC server, and the server maintains one central cache for data, this approach might impose very little load on the server, because the cache must be updated for the primary interface instance.

For an OPC server that does not use a centralized cache or a configuration in which the interface instances connect to different OPC servers, this approach can impose a considerable load on an OPC server or the data source system. When the backup interface instance becomes primary, all it needs to do to start collecting data is to advise the groups, making this the fastest warm failover option. The command line equivalent is `/UFO_TYPE=HOT /FM=3`.

Cold failover

Cold failover is required if an OPC server can support only one client, or if you are using redundant OPC servers and the backup OPC server cannot accept connections. The backup instance does not connect with the OPC server until it becomes primary. At that time, it must create groups, add items to groups, and advise the groups. This delay almost always causes some data loss, but imposes no load at all on the OPC server or data source system.

Tip: In addition to Unilnt based failover, PI Interface for OPC DA also supports OPC server-level failover. See the topic [Configure OPC server failover](#) and its sub-topics for detailed descriptions of the options available. OPC server-level failover is based on information received from the OPC Server; there is no PI point to control which OPC Server is active or which connection is preferred.

In brief, OPC server-level failover utilizes OPC watchdog points to control OPC server-level failover. Watchdog points enable the interface to detect when its OPC server is unable to adequately serve data and failover to another OPC server if that other OPC server is better able to collect data. This approach is intended for OPC servers that are data aggregators, collecting data from multiple PLCs. If one point on each PLC is designated as a watchdog point, the interface can be instructed to failover if less than a specified number of those points are readable. This approach enables the benefits of redundancy to be applied at the data collection level.

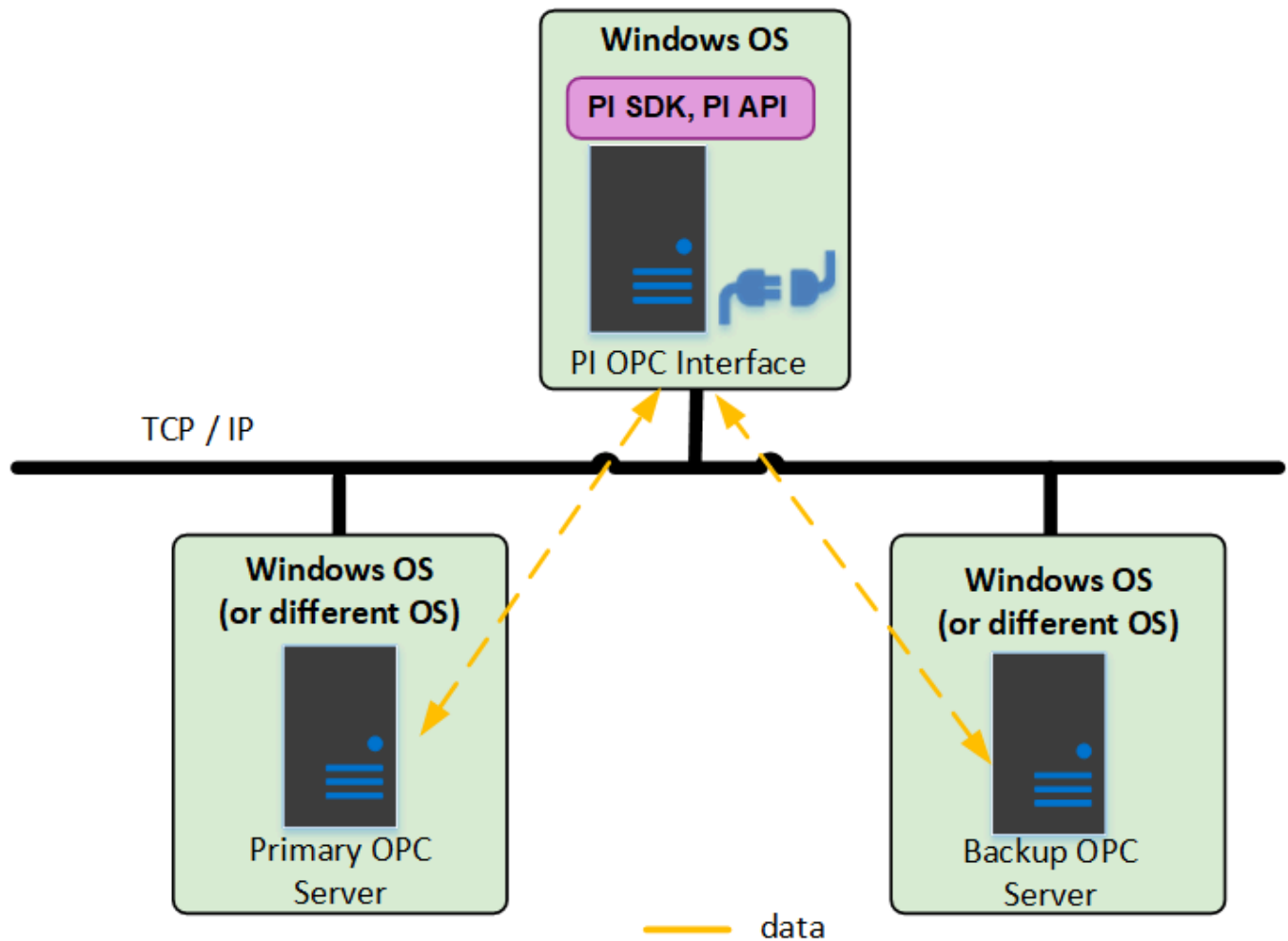
OPC server-level failover

To ensure that data continues to flow from the OPC server to PI Interface for OPC DA, the interface can be configured to switch to another OPC server under the following conditions:

- Loss of connectivity to current OPC server
- The OPC server state changes to a value other than "RUNNING"
- Specified OPC server items indicate that the OPC server is unavailable, either by a specific value or by the quality

OPC servers can be fully redundant (multiple servers active at the same time) or configured for failover, where one server at a time is active. Note that OPC servers vary widely in their approach to tracking and reporting status, so consult your OPC server documentation to determine what options are supported.

The following diagram illustrates a basic OPC server-level failover configuration.



Configure OPC server failover

Configure PI Interface for OPC DA to fail over to another OPC server in the event it loses connection to the original OPC server.

1. In PI ICU, go to the **OPCInt Failover > Server Level** page and enter the node and name of the other OPC server.
This basic configuration triggers failover only when PI Interface for OPC DA loses connectivity to the OPC server.
2. Optionally, create a string PI point to track the active OPC server. Assign the PI point to an unused point source.
In PI ICU, go to the **OPCInt > Failover > Server Level** page and enter the PI point name in the **Current Active Server Tag** field. To display the value of the point, launch PI SMT and use the **Data > Current Values** feature.
When failover occurs, the value of this point changes to the name of the currently connected OPC server. The archive of these changes enables you to view failover history.
3. To verify that failover occurs when connectivity is lost:
 - a. Start both OPC servers, then start the interface.

- b. Use PI SMT or the pigetmsg utility to check the AF SDK log on the Data Archive node for messages that verify successful startup.
- c. Stop the currently active OPC server and check the SDK log to confirm that the interface has switched to the other OPC server.
- d. To switch back to the first OPC server, restart it, stop the second server, and check the SDK log or the value of the PI active server point, if defined, to verify that the PI OPC interface has switched back to the first OPC server.

Configure failover on OPC server state change

You can configure failover for cases where the OPC server leaves RUNNING state.

Navigate to the **OPCInt > Failover > Server Level** page, and check **Failover if Server Leaves RUNNING State**.

To configure a waiting period, set the **Wait For RUNNING State** field to the desired number of seconds. If the OPC server has not entered the RUNNING state before the specified period expires, PI Interface for OPC DA attempts to connect to the other server. If the other server has not entered the RUNNING state before the specified period expires, the interface retries the first server, alternating between the two until one is detected to be running again.

Server status and OPC watchdog points

To enable the interface to track the availability of OPC servers, use PI ICU to configure watchdog PI points that are mapped to OPC items that reliably reflect the state of the OPC server.

The OPC items must contain a positive integer if the OPC server is running or 0 if the OPC server is unavailable, and OPC watchdog points must have an integer data type.

When choosing the OPC items that you map to the OPC watchdog points, consider which ones are most reliable and representative of the state of the OPC server. For example, in a manufacturing context, an item that counts number of units manufactured might make sense. To ensure that the primary and backup OPC servers report status consistently, choose an OPC item based on the data source itself (as opposed to an item that originates in the OPC server).

To ensure that the values in OPC watchdog points are valid (0 or positive integer), use scaling and transformation for PI points as required.

To reduce system workload, configure the OPC watchdog points as advise points if the OPC server supports advise points. If not, assign them to a scan class with a short scan period.

If more than one instance of the interface is running, either on the same or different nodes, you must create separate OPC watchdog points for each instance of the interface. When the instance ID is not used, each instance scans only those PI points with *pointsource* attributes that matches the interface instance *ps* parameter. When the instance ID is used, the *pointsource* attribute must match the interface *ps* parameter and the *location1* attribute must match the interface *id* parameter.

Configure a single watchdog PI point

If you can determine the availability of the OPC server based on a single OPC item, create a single watchdog PI point. When the value of the OPC item is 0, PI Interface for OPC DA attempts to connect to the other server. If it cannot connect successfully to the other server within the specified connection timeout period, it attempts to reconnect to the first OPC server again.

To create and configure an OPC watchdog point:

1. Create a PI point. Map the point to an OPC item that you consider a reliable indicator of server status.
The OPC item to which the point is mapped must be defined identically in both the primary and backup OPC servers, while possibly having different values on the two servers.
2. Indicate the watchdog point for the primary and backup OPC servers: Using PI ICU, go to the **OPCInt > Failover > Server Level** page and configure the **Primary Server Watchdog Tag** and **Backup Server Watchdog Tag** fields.
3. Verify that the OPC item triggers failover:
 - a. Start the OPC servers and verify that the watchdog item is non-zero on at least one of the servers. Start the interface.
 - b. Manually set the OPC item to 0 on the currently-connected server.
 - c. Examine the AF SDK log or check the Active Server point to determine whether the interface failed over to the other OPC server.

Configure multiple watchdog PI points

If you must assess several OPC items to determine availability (such as when the OPC server can report on the availability of the back-end data sources), configure multiple OPC watchdog PI points. The sum of the values of the OPC watchdog points determines whether the server is considered active. The interface initially assigns each watchdog point a value of 1 and recalculates the total whenever it receives a new value for one of the points. If the sum goes below the specified minimum, failover is triggered.

1. Create PI points and map them to the OPC items that you consider reliable indicators of OPC server status. For each point, set *location3* to 3 for polled points or 4 for advise points.
2. Using PI ICU, go to the **OPCInt > Failover > Server Level** page and set the **Multiple Watchdog Tags Trigger Sum** field to the minimum acceptable total of the values of the watchdog points.
3. Verify that failover is triggered if the total of the values drops below the specified minimum:
 - a. Start the OPC servers and the interface.
 - b. Manually set the values of the OPC items.
 - c. Examine the AF SDK log or check the Active Server point to determine whether the interface failed over to the backup OPC server.

OPC item quality

If you have multiple watchdog PI points, you can configure failover to occur when a specified number of watchdog points are read with BAD quality. To configure this setting, go to the **OPCInt > Failover > Server Level** page and enter the desired maximum in the **Maximum number of Watchdog Tags which can have Bad Quality or Any Error without triggering Failover** field.

Configure server-specific OPC watchdog points for efficient failover

OPC servers track their own states (isolated mode). To enable PI Interface for OPC DA to determine the state of an OPC server before attempting to failover to it, configure both OPC servers to track each other's state as well (server-specific mode). This configuration enables the OPC server to determine the state of both servers without

the overhead of creating a second connection.

Note: The method by which an OPC server tracks its state is highly vendor-dependent and implementations vary. For details, consult your OPC server documentation.

Use PI ICU to configure OPC watchdog points.

1. In both OPC servers, create identical items that track the status of each server.
If an OPC server is active, the OPC item must contain a positive value. If an OPC server is unable to serve data, the item value must be zero. Implement any logic required to ensure that both servers correctly detect and maintain the status of the other server and that, in both OPC servers, the values are identical.
2. Configure the OPC servers so that, during normal operation, one server sends data to PI Interface for OPC DA and the other waits until the primary server fails.
Status for the primary server must be positive, and for the backup server, status can be zero. If failover occurs, the primary server status must be set to zero and the backup server status to a positive value.
3. In Data Archive, create a watchdog PI point for each OPC server, mapped to the OPC items that track server status.
4. Using PI ICU, go to the **OPCInt > Failover > Server Level** page and set the **Primary Server Watchdog Tag** and **Backup Server Watchdog Tag** fields to the names of the watchdog PI points you created in the previous step.
In the interface batch startup file, these settings are specified by the *wd1* and *wd2* parameters.

If both watchdog points are zero, data collection stops until one watchdog point becomes positive. If both watchdog points are positive, the interface remains connected to the server that is currently serving data to it.

Failover timing

When failover is triggered, PI Interface for OPC DA must quickly recognize that the current OPC server is no longer available and determine whether the backup OPC server is available. To configure failover efficiency in the PI ICU, you can adjust the following settings on the **OPCInt > Failover > Server Level** page:

- **Switch to Backup Delay (/FT=#)**
Specifies in seconds how long the OPC interface tries to reconnect to the current server, before failing over to the other server and, if less than 30, how often the interface checks the server state.
- **Number of Interfaces on this Node (/NI=#)**
/ni=# (Optional) Specifies the number of instances of the interface running on this node. This is used in conjunction with the *FT* parameter to determine how long to wait before initiating server-level failover.
- **Wait for Running State (/SW=#)**
Specifies how many seconds the interface waits for the server to enter the RUNNING state before failing over to the other server. By default, the interface waits indefinitely for the server to enter the RUNNING state.
Note that OPC servers vary significantly in the time required to enter the RUNNING state.

Buffering for PI Interfaces

The interface node uses buffering to prevent data loss when Data Archive is not available. Unilnt interfaces can buffer data to store point values when network communication to the Data Archive is unavailable. Unilnt disconnected startup requires buffering, and it is highly recommended for failover. Buffering for interfaces is

configured and enabled through PI ICU.

Buffering services

The PI System offers two services to implement buffering at interfaces:

- PI Buffer Subsystem (PIBufss)
- API Buffer Server (Bufserv)

PI Buffer Subsystem is the best option for most environments.

Use API Buffer Server only if one or more of the following conditions is true:

- The version of Data Archive receiving the buffered data is earlier than 3.4.375
- Your interfaces run on a non-Windows platform

If any of the above conditions apply to you, see the *PI Buffering Manager Help (PIPC/HELP/BufferManager.chm)* documentation for PI Server. Otherwise, use PI Buffer Subsystem.

Buffering and collectives

PI Buffer Subsystem 4.3 and later can buffer data to multiple independent servers, including those configured as PI Server collectives. For interfaces to use PI Buffer Subsystem with PI Server collectives, the Data Archive servers must be running Data Archive version 3.4.375 or later.

Caution: API Buffer Server does not detect and validate the PI Server collective configuration. As a result, API Buffer Server requires manual configuration whenever a collective changes. Also, because API Buffer Server results in compression at the Data Archive machine, archives at different servers in the collective could contain different records.

Buffering configuration

Use PI Interface Configuration Utility (PI ICU) to configure interface buffering.

The **Tools > Buffering** option helps you configure buffering. Depending on your current configuration, this option does one of the following:

- If this computer is configured to buffer data using PI Buffer Subsystem 4.3 or later, the Buffering Manager window opens and shows a buffering dashboard. The dashboard shows information about the status of buffering on this computer.
- If this computer is not currently configured to buffer data, and PI Buffer Subsystem 4.3 or later is installed, you are prompted to configure PI Buffer Subsystem. If you select Yes, the Buffering Manager window opens and shows the installation wizard, which helps you configure PI Buffer Subsystem.
- If this computer is configured to buffer data using API Buffer Server (Bufserv), and PI Buffer Subsystem 4.3 or later is installed, you are prompted to convert to and configure PI Buffer Subsystem. If you select Yes at both prompts, the Buffering Manager window opens and shows the upgrade wizard, which helps you upgrade from API Buffer Server to PI Buffer Subsystem.

- If PI Buffer Subsystem 4.3 is not yet installed, the Buffering window opens for API Buffering or the earlier version of PI Buffer Subsystem.

For PI Buffer Subsystem 4.3, when configuring an interface to buffer data to a Data Archive server which has not been added to the buffered server list you must enable buffering. Select the **Enable** button in the **Buffering Status** box on the interface General page. To verify that the buffering status is On, exit PI ICU, then restart and select the interface.

You can use Buffering Manager to configure, monitor, and troubleshoot buffering using PI Buffer Subsystem. PI Buffer Subsystem is recommended for applications connecting to Data Archive 3.4.375 or later. Older versions of PI Data Archive require API Buffer Server, as do some sites with custom solutions. See *Buffering Manager help* ([PIPC/HELP/BufferManager.chm](#)) for more information.

Enable buffering

These instructions apply only if PI Buffer Subsystem version 4.3 or later is installed on this computer.

To determine the version that you have installed, select **ToolsBuffering...** from the PI ICU toolbar. If you have PI Buffer Subsystem version 4.3 or later installed, the Buffering Manager window displays. In that case, you would use the following procedure to start buffering.

If version 3.4.380.79 or earlier is installed, an alert displays indicating that PI Buffer Subsystem is not configured.

1. Select **Tools > Buffering...**
2. When the dialog box displays asking if you want to configure PI Buffer Subsystem, select **Yes**.

Note: If you currently use API Buffer Server (bufserv), you may need to confirm more than one prompt.

The Buffering Manager - New Install Wizard window opens. This indicates one of two things:

- The computer is configured to buffer data using API Buffer Server (bufserv). In this case, and before continuing, review the "Install or upgrade to PI Buffer Subsystem 4.3 or later" topic found in the Help system. From the Buffering Manager - New Install Wizard window, select Help to navigate to the topic, which also contains information on upgrades from API Buffer Server.
 - The computer is not configured to use any form of buffering.
3. To configure buffering, follow the instructions in the Buffering Manager - New Install Wizard window.
 4. After you finish, return to the PI Interface Configuration Utility window. Select each interface, and on the General page under **PI Host Information**, look at the **Buffering Status** setting:
 - If **Buffering Status** is On, the buffering configuration for the server to which this interface sends data is complete. (This is the server specified in the **API Hostname** field for this interface.)
 - If **Buffering Status** is Off, you need to configure the server specified in the **API Hostname** field to receive buffered data from this interface. To add the server to the buffering configuration, select the **Enable** button and follow the instructions on the Buffering Manager screen.

Note: If existing data flows have unbuffered interfaces, stop all interfaces services configured for buffering and restart the **PI Buffer Subsystem** to send data through the Buffer.

Learn about plug-ins (post-processing DLLs)

PI Interface for OPC DA can be configured to use plug-ins, which are DLLs that contain libraries of routines that

perform application-specific data processing before the data is sent to Data Archive or the OPC server. The DLLs and their accompanying files and documentation are included in the PI Interface for OPC DA installation kit and are installed into the **Plug-ins** sub-directory under the installation directory.

Operate the OPC DA Interface

During operation of PI Interface for OPC DA, you can verify interface startup, confirm data is being collected from the OPC server, and troubleshoot configuration and operational problems.

Verify interface startup

You can use the message log to check for proper startup of the interface.

1. Display the message log. To do this, open PI System Management Tools and select **Operation > Message Logs**.
2. Start the interface. To do this, open PI ICU, navigate to the interface instance, and then select **Interface > Start Interactive**.
PI ICU opens a command window and runs the startup batch (.bat) file. The interface logs messages as it attempts to initialize and run.
3. View the log to identify messages indicating success or errors.
4. To stop the interface, close the command window.

OPC server data collection

Data is read from OPC servers in groups, not as individual items. PI Interface for OPC DA creates OPC groups for PI System scan classes. (For advise PI points in scan class 1, multiple groups might be created.) The OPC server caches the most recent data.

By default, PI Interface for OPC DA reads from the cache of an OPC server. When the interface creates a group, it specifies how often the cache values for the points in that group are to be updated. The requested update rate is usually the same as the scan rate for the points. The OPC server might decline the requested rate and return the update rate that it supports for that group. The update rate to which the OPC server agrees is recorded in the local PI Message Log file.

Learn about PI ICU reference

PI Interface Configuration Utility (PI ICU) is a graphical user interface for configuring interfaces. PI ICU ensures that the configuration information stored in the startup batch file and the PI Module Database is generated and updated correctly, eliminating the need to edit it manually.

PI ICU requires Data Archive 3.3 or later. For more information, see the [PI Interface Configuration Utility \(PI ICU\) User Manual](#).

OPC server settings

OPC Server Node Name

The name or IP address of the OPC server node (**/SERVER=node::name**). Leave blank or set to "localhost" (case insensitive) if PI Interface for OPC DA and OPC server are on same node. If anything else is used for the node name, the interface will treat the OPC server as remote. This can impact certain OPC servers that may refuse "remote" connection attempts.

OPC Server Name

Registered name of the OPC server on the OPC server node.

Force v1.0a Protocol

By default, the interface tries to connect to the OPC server using the v2.0 OPC Data Access specification (**/vn=2**). If the attempt fails, it uses the v1.0a protocol. To force the interface to use only the v1.0a OPC Data Access specification, enable this option.

Timestamps

PI Server Provides All Timestamps: The OPC interface provides a time stamp when the data is received (**/ts=N**).

Original Timestamps Adjusted for PI Server: The OPC interface uses the data time stamps provided by the OPC server and accounts for the offset between the OPC server and the PI Data Archive node (**/ts=Y**).

Original Timestamps for Advise tags Only: The OPC server provides time stamps only for advise tags, and the interface accounts for the offset between the OPC server and the Data Archive node. For all other tags, the interface provides a time stamp when the data is received (**/ts=A**).

Original Timestamps: The OPC server provides the time stamps for all data, and the interface will not apply any time offset to these values. Data loss will occur if a value is received from OPC with timestamp 10 minutes or more past current time of the Data Archive node. (**/ts=U**).

Questionable Quality

Store Quality Data Only: If data has other than GOOD quality, store the quality information rather than the value

(/sq=Y).

Store Value Flagged Questionable: Store the value and flag as questionable (/sq=N, default).

Store Value Only: The interface treats questionable quality as good. (/sq=I).

Bad quality data is different than questionable quality data, and is always stored as a system digital state.

Advanced options settings

Time delay before reading OPC Tags (sec)

Specify a delay (in seconds) before reading from or writing to the OPC server. If this parameter is configured, the interface connects to an OPC server and waits the specified amount of time before attempting to read data. (/sd=#).

Event Tags Source

For v1.0a OPC servers, specifies whether event tags are read from the OPC server's cache or directly from the device. For v2.0 servers, it has no effect, because all event tag reads are from the device. (/es=CACHE or DEVICE).

Advise Groups on Creation

Some OPC servers do not return an initial value when an advise PI point is created. The resulting symptom is that, for a value that does not change often, PI Interface for OPC DA does not write a value to Data Archive when the interface starts. To determine whether your OPC server has this problem, use PI OPCClient tool to create a group, add tags, and then advise the group. If a value is not immediately returned for your tags, but adding a tag to the scan class causes a value to be returned, enable this setting. (/af=Y).

Note: Be cautioned that setting this attribute when the OPC server is returning an initial value upon advise PI point creation will lead to data loss.

Enable Mass Tag Adding

The PI OPC DA interface tells the OPC server which values it is interested in before the interface is allowed to request data for them. The interface does so by creating "groups" on the server and then adding items to the groups. The groups correspond roughly to PI scan classes and the items correspond to PI tags. The /ma parameter configures how the interface adds items to each group:

- In groups of up to 500 items at a time - Mass Tag Adding enabled (/ma=y), requiring one round-trip to the server for each group
- One at a time - Mass Tag Adding disabled (/ma=n), requiring one round-trip to the server for each item

If you have configured thousands of PI tags and notice that the interface takes a long time to start, try enabling Mass Tag Adding to reduce the time required to load points.

Note: Some OPC servers reject all the items in a group if even a single one of them is invalid. For those servers, set /ma=n to make sure the valid items are not rejected.

Depending on the network, the OPC server, and (most importantly) the point count, setting /ma=n can result in

significantly slower interface startup. However, if the OPC DA interface is installed on the OPC server then the startup time may not be significantly affected.

The default setting for PI OPC DA interface is `/ma=n`.

GlobalLocking Not Valid

If the local PI Message Log file contains *OnDataChange: Invalid group ID* messages, enable this option. If the problem is resolved, the OPC server does not follow the OPC specifications. In this case, e-mail details (including OPC server vendor and version) to techsupport@osisoft.com. This flag is only meaningful for version 1.0a OPC DA. (`/gl=N`).

Ignore Group Status

If you see *OnDataChange: Header status:* in the local PI Message Log file, the group status sent by the server is invalid. To ignore the group status, enable this option. This flag is only meaningful for version 1.0a of OPC DA. (`/gs=N`).

Ignore Server Status

If the OPC server does not go to `OPC_STATUS_RUNNING` state when it is ready to send data, enable this option to direct the interface to attempt to communicate with it anyway (`/is=Y`).

Ignore OPC Server Access Rights

If you see “Invalid read/write mode requested” messages in the local PI Message Log file, enable this option. (`/ar=N`).

Use Honeywell Plantscape Failover Error Codes

Enable checking for error codes that are specific to the Honeywell Plantscape system for server-level failover. Configures the interface to fail over if it receives an error code of `0xE00483FD` or `0xE00483FC` on any tag. This is obsolete because Honeywell stopped using these codes after only one release. (`/hwps`)

Reconnect to Server Delay (sec)

Specifies how long to wait, in seconds, before attempting to reconnect to the OPC server if the connection is broken. (`/rd=#`).

Update Rates

Specifies the requested update rate, if different from the scan period. Select a scan class from the drop down list, enter the rate in the box to the right of the scan class, and select . The scan class, scan rate, and update rate appear in the box below the period. Only scan classes that have update rates are listed.

This option is useful when the server must have a recent value for the items but the interface does not read it very often, for example, if PI Interface for OPC DA polls for the value every 30 minutes, but the value itself must

be no more than one minute old. This situation imposes more load on the OPC server than if the update rate and the scan period are the same, but it can reduce latency of values for items that need to be read less frequently. (/ur=period).

Data Handling settings

Staggered Group Activation

This option directs PI Interface for OPC DA to inactivate all groups on startup and to stagger the activation of the groups based upon the offsets specified for the group's scan period. This feature does not affect the operation of all OPC servers. It is intended to help level the workload by spreading out updates for groups that have the same scan period. (/ga).

Inactivate Groups on Startup

Inactivate all groups on startup. After the groups are built, they are activated. This option helps reduce the load on the OPC server during startup. (/gi).

Update Snapshot

If the current snapshot is a system digital state (such as *I/O timeout*, *Shutdown*, and so forth) and the interface reads in a new value that is older than the snapshot, the interface sends the new value one second after the snapshot time stamp of the system digital state. This check is omitted if the current snapshot is a good value. This is useful for setpoints that rarely change. (/us).

Ignore First Value

If the OPC server sends data before it reads its data source, it is likely to transmit zeros or erroneous values. This parameter directs the interface to ignore the first value that it receives after startup for each tag. (/if=Y).

Ignore Subsecond Timestamps

If the millisecond portion of the time stamp is not required, it can be truncated, which can speed up processing on Data Archive (/it=Y).

No Timeout

Direct PI Interface for OPC DA never to write *I/O timeout* errors, even if it loses its connection with the OPC server. Set this when configuring failover. (/nt=Y).

Disable Callbacks

Reduce the load on the OPC server by disabling call backs for polled groups. By default, polled groups have call backs enabled, but these call backs are not used by the interface. This option has no effect on advise groups. (/dc).

Write Status to Tags on Shutdown

This parameter specifies the digital state to be written to all PI points when the interface is shut down (/opcstopstat=state).

Note: In a *Unilnt* failover configuration, the digital state is not written to points when an interface instance is stopped, because the other interface instance takes over writing data to the point.

Alternate Digital State for Questionable/Bad Qualities

Assign alternate digital states for questionable and bad qualities. To use this option, create a contiguous set of digital states in the system digital state set that corresponds to the set of states listed in the OPC manual, then assign the first digital state of the set to the command line option. (/as=system digital state). To view system digital states using PI System Management Tools, go to **Points > Digital States**.

Format of Timestamp Strings

Sets the format for time stamp strings read from or written to the OPC server. (/tf=format). For more information, see [Time stamp adjustments](#).

Number of Tags in Advise Group

Configure the maximum number of tags for each advise group created with scan class 1. The recommended maximum is 800 tags per group, which is the default. Adjust this number according to the OPC server requirements. (/am=#).

Time Offset

If the OPC server node is set to a time zone other than the local time zone, this option directs the interface to adjust all the time stamps by the specified amount. To specify the offset, use the format [-]HH:MM:SS. (/to=offset).

Event Update Rate

Specify the requested update rate for the event class group. All event-based tags belong to the same group and the default update rate for the group is one second. If the OPC server's data cache for event-based tags does not need to be updated every second, you can reduce load on the OPC server by setting this parameter to a higher value (a lower rate). For v2.0 servers, all events are read from the device, so this value can be set quite high unless there is some other reason to update the cache. Use the format /ER=HH:MM:SS.

Trend Advise

For advise PI points, send the value from the preceding scan if the new value's timestamp is greater than the number of scan periods (configured by the /ta parameter). Enabling this setting causes advise tags to behave as if the *step* attribute is enabled.

The log file will print a message indicating that TA is enabled.

Note: Enabling Trend Advise will disable ExcMax processing for Advise points. See [Appendix A. Set ExcMax on advise and event points](#) for details on ExcMax and Advise/event points.

DCOM Security settings

For more information about configuring DCOM security, see the [DCOM Security and Configuration](#) guide.

Default Authentication Level

Set the DCOM security authentication level (*/da*) to one of the following:

- DEFAULT
- NONE
- CONNECT
- CALL
- PKT
- PKT_INTEGRITY (default)
- PKT_PRIVACY

Default Impersonation Level

Set the DCOM security Impersonation level (*/di*) to one of the following:

- ANONYMOUS
- IDENTIFY (default)
- IMPERSONATE
- DELEGATE

Failover settings

Unilnt-Interface Level Failover

The following three options are enabled only if warm failover is enabled on the **Unilnt > Failover** page:

- **Warm_1:** Do not create groups on the server (*/fm=1*)
- **Warm_2:** Create inactive groups and add PI points (*/fm=2*)
- **Warm_3:** Create active groups; do not advise groups (default) (*/fm=3*)

Note: For more information on Unilnt-Interface level failover modes see [Unilnt failover modes](#).

- **Percent of tags accepted by OPC Server as valid**

Specify the percentage of points that are required to be accepted by the OPC server as valid. If less than this percentage is accepted, PI Interface for OPC DA sets its device status to *Connected/No Data*, which triggers

failover if Unilnt failover is configured. (/rp).

- **Maximum number of Watchdog Tags which can have Bad Quality or Any Error without triggering Failover**

Specify the maximum number of watchdog PI points that can have an error or bad quality before failover is triggered. You can configure watchdog PI points to control failover when the interface is unable to read some or all of the items, or when the items have bad quality. This feature enables you to trigger failover when a data source loses the connection to one OPC server, but is able to serve data to the other. To configure watchdog PI points, set *location3*. For a watchdog point that is in an advise group, set *location3* to 4. For a watchdog point that is in a polled group, set *location3* to 3. (/uwq).

Cluster Interface Failover

To make selections in this option, first enable it by selecting the **Enable Cluster Interface Failover** check box. Note this tab is only available when **Unilnt Failover** is not selected.

Setting	Description
This node is the	Specify whether this node is primary (/pr=1) or backup (/pr=2).
Failover Mode	Warm_1: Do not create groups on the server (/fm=1). Warm_2l: Create inactive groups, and add points (/fm=2). Warm_3: Create active groups, do not advise groups (default) (/fm=3).
Cluster Mode	Configure behavior of the backup interface. Primary Bias: This node is the preferred primary. (/cm=0). No Bias: No node is preferred. The active PI OPC interface stays active until the cluster resource fails over, either as the result of a failure or through human intervention. (/cm=1).
Resource Number for APIONline	Identify the apionline instance that goes with this interface instance. For example, to configure the interface to depend on an instance named apionline2, set this field to 2. To configure the interface to depend on an instance named apionline (no resource number), set this field to -1. (/rn=#).
Active Interface Node Tag	Specify the string point that contains the name of the currently active OPC interface node. (/cn).
Health Tag ID	This parameter is used to filter Unilnt health points by <i>location3</i> . The parameter must be unique for each interface – failover member parameter. If this parameter has an invalid value or is not set, the

Setting	Description
	default value of 0 is used for the <i>location3</i> attribute when creating Unilnt health points. (/uht_id).

Server Level Failover

Setting	Description
Backup OPC Server Node Name	The name or IP address of the backup OPC server node (/backup).
Backup OPC Server Name	The registered name of the backup OPC server (/backup).
Number of Interfaces on this Node	The number of instances of the interface that are running on this node (/ni=#).
Switch to Backup Delay (sec)	The number of seconds to try to connect before switching to the backup server (/ft=#).
Wait for RUNNING State (sec)	The number of seconds to wait for RUNNING state before switching to the backup server (/sw=#).
Current Active Server Tag	(Optional) String PI point that contains the name of the currently active server. If set, the interface writes the name of the OPC server to this point whenever it connects. Useful for debugging server-level failover. (/cs=tag).
Primary Server Watchdog Tag	Watchdog point for the primary server (/wd1=tag).
Backup Server Watchdog Tag	Watchdog point for the backup server (/wd2=tag).
Multiple Watchdog Tag Trigger Sum	The minimum total value of the watchdog points. Failover is triggered if the sum of the value of these points drops below the specified value. (/wd=#).

Setting	Description
Maximum number of Watchdog Tags which can have Bad Quality or Any Error without triggering Failover	Default=0 if only one watchdog point. Cannot exceed the number of watchdog points defined. (/wq=#).
Failover if Server Leaves RUNNING State	Triggers failover if the server state changes to anything other than RUNNING. (/ws=1).

Plug-in settings

- Post-Processing DLL**
 Enter the DLL name and path to the post-processing DLL, for example, /DLL="Interfaces\OPCInt\plugins\exampldll.dll"
- Plug-In Configuration File**
 Enter the name of the post-processing DLL configuration file. This text box is displayed only if the post-processing DLL requires a configuration file.

Miscellaneous settings

Caution: Do not modify these settings unless directed to do so by AVEVA Technical Support.

Debug settings

To enable debugging options using PI ICU, go to the **OPCInt > Debug** tab. In general, enable debug options for a short period of time, as they can bloat log files and reduce performance. For options marked "Technical Support only," enable only at the direction of AVEVA Technical Support.

For details about other command-line parameters, refer to the [PI Universal Interface \(UniInt\) User Guide](#).

Option	Description	Value
Internal Testing Only	For AVEVA internal testing only.	/db=1
Log of Startup	Logs startup information for each PI point, including <i>instrumenttag</i> and <i>exdesc</i>	/db=2
Log Write Op's and Acks for Tag	Logs PI Interface for OPC DA writes, ACKs from the OPC server, and writes queued to the "pending write" queue. Can be configured to log values sent for a specific point if	/db=4

Option	Description	Value
	the Debug Tag field is specified.	
Log Timestamps of refresh	For use by AVEVA Technical Support only.	<i>/db=8</i>
Log Information for ExcMax	Log information about exception reporting	<i>/db=16</i>
Log Timestamp and Data (All Tags)	For each data value that the interface receives, logs the time stamp with the data, the adjusted time stamp, the PI time, the scan class, and transaction ID.	<i>/db=32</i>
Log Timestamp and Data for Tag	For use by AVEVA Technical Support only.	<i>/db=64</i> <i>/dt=tagname</i>
Logging of Event Tags	Logs the name of each PI point into the local PI Message Log file as it receives data for the point.	<i>/db=256</i>
Logging of Array Tags	Logs information about the array PI points	<i>/db=512</i>
Logging of OPC List Pointers	For AVEVA internal testing only.	<i>/db=1024</i>
Log TS, Data and Quality for Tag	For the point that is specified in the Debug Tag field, logs time stamps, values, and qualities in the local PI Message Log. If there is no point specified, the first point for which a value is received is logged. This option is verbose and can bloat the log file.	<i>/db=4096</i>
Log debugging info for /US command	Provide debugging information for the Update Snapshot (<i>/us</i>) option, if enabled.	<i>/db=8192</i>
Log client, server, and group handles	Log the addresses used by the interface and server for each tag added to the interface. Enable only under direction of AVEVA Technical Support.	<i>/db=16384</i>

Enter any additional parameters that are not available through PI ICU, (for example, */dbuniint=0x0400*). Separate parameters with one or more spaces. If a parameter argument contains embedded spaces, enclose the argument in double quotes.

Time stamp options

PI Interface for OPC DA can use the time stamps provided by the OPC server or create its own time stamps at the time that the data is received. Windows type time stamps are defined as Coordinated Universal Time (UTC), and items with VT_* variant values are interpreted as local time without daylight savings time adjustments.

If the OPC server provides time stamps, you can use PI ICU to configure the behavior of the interface as follows:

Option	Description	Timestamp Offset Applied
Interface Provides Time stamp (/ts=N)	(Default) The interface time stamps each value as it is received. Choose this option if the OPC server cannot provide accurate time stamps or you do not want to use the time stamps returned by the OPC server.	Difference between Data Archive node and interface node.
OPC Server Provides Time stamp (/ts=Y)	The interface uses the UTC time stamp provided by the OPC server.	Difference between Data Archive node and OPC server node.
Time stamp for Advise Tags Only (/TS=A)	For polled reads, some OPC servers return the time that the value last changed rather than the time of the read. This option configures the interface to use the advise time stamps but provides time stamps for the polled values. For more details about advise and polled points, see Input points .	For advise data, difference between Data Archive node and OPC server node. For all other data, difference between Data Archive node and interface node.
OPC Server Provides Timestamp, no Offset (/ts=U)	The interface uses the UTC timestamp provided by the OPC server and does not apply any offset to the time stamps. Caution: Use this setting with extreme caution, as data loss occurs if the OPC server sends a value with a timestamp that is 10 minutes or more later than the current time of Data Archive.	None.

For details about reading and writing time stamps from a PI point when the time stamp is the value of the point, see [Time stamp adjustments](#).

Troubleshooting the interface

Refer to the following sections when troubleshooting interface or OPC server problems.

Open PI OPC client tool

To connect to and view the contents of your OPC server, use PI OPC Client Tool. You can test data exchange procedures such as sync read, refresh, advise, and outputs using this tool.

PI Interface for OPC DA no longer includes PI OPC Tools in the self-extracting installation kit. PI OPC Tools can be downloaded from the AVEVA Customer Portal Products page [AVEVA Customer Portal Products Page](#). Search for PI OPC Tools Set (OPCTools) Install Kit.

- Open PI OPC Client Tool in one of two ways:
 - Locate and double-click the **OPCClient.exe** executable file.
 - From the Windows Start menu, select **All Programs > PI System > PI OPCClient**.
- See the PI OPC Client Tool User Guide for detailed information.

OPC server issues

The OPC specification permits a great deal of flexibility in how OPC servers are designed and in what features they support. These variations in OPC servers can affect users of PI Interface for OPC DA.

Item browsing

To be able to map PI points to OPC items, you must have access to OPC item names. However, OPC servers are not required to support item browsing. If browsing is supported, you can use the PI OPC Client tool to display the points that the OPC server recognizes.

OPC server time stamp options

OPC servers send time stamps in one of two ways:

- Send the time stamp for the last time that the data value and quality were read from the device. In this case, the time stamp changes even if the value does not.
- Send the time stamp of the last change to the data value or quality. In this case, if the data remains the same, the time stamp does not change.

You must set the way time stamps are recorded by configuring the time stamp (*ts*) parameter using PI ICU.

Unreliable values

Some OPC servers return a value when a client connects to a point, even if the server does not yet have a valid value for the point. Some servers send an unreliable value with a status of GOOD, which results in this value being sent to Data Archive.

To screen out these unreliable values, use PI ICU to enable the **Ignore First Value** option on the Data Handling page (/if=Y).

Access path

In OPC items, the access path suggests how the server can access the data. The OPC standard states that it is valid for servers to require path information to access a value, but not for them to require that it be sent in the access path field. According to the standard, the OPC server can ignore it, but some non-compliant OPC servers require the access path.

For example, RSLinx requires path information in the access path or as part of the ItemID, in the following format: *[accesspath]itemid*.

If your OPC server requires an access path, contact your OPC server vendor to determine how best to configure the server with PI Interface for OPC DA.

Problems with data returned by OPC server

The *Unpack2* error messages in the local PI Message Log can indicate invalid data. In some cases, PI Interface for OPC DA might be sending data to Data Archive and also logging *Unpack2* errors for the PI point, because the OPC server sends good values when it can and error codes when it cannot. Intermittent failures between the OPC server and the data source result in a combination of errors and values.

The following generic error messages are commonly logged:

```
In UnPack2 Tag MyPV.pv returns error 80020005: 80020005(Type mismatch)
In UnPack2 Tag MyPV2.pv returns error: The operation failed (80004005)
```

The following error messages indicate that the data received from the OPC server contained errors and the OPC server did not return a text explanation of the error:

```
In UnPack2 Tag MyPV3.pv returns error : Unknown error(800482d2)
In UnPack2 Tag MyPV4.pv returns error E004823E: Unknown error (e004823e)
In UnPack2 Tag MyPV5.pv returns error E241205C: Unknown error (e241205c)
In UnPack2 Tag MyPV6.pv returns error E2412029: Unknown error (e2412029)
```

To troubleshoot such data-related issues, consider the following causes and solutions:

- If you see *Unknown* errors, check with the OPC server vendor and have them look up the error code displayed in the error message. OPC servers can generate vendor-specific error codes, and only the OPC server vendor can explain what they mean.
- Restarting the OPC server might resolve the issue.
- *Type mismatch* errors indicate incompatible data types. Check for a mismatch between the Data Archive data type and the OPC item type. Check *location2* settings. To avoid cache issues after data types are changed, restart the interface.
- Verify that the data type of the PI point can accommodate the range of values being sent by the OPC server.

For example, if a PI point is defined as a two-byte integer and the OPC server sends values that are too large for it to accommodate, the point overflows.

- Make sure the data type of the OPC item and PI point are compatible.
- The data source might be sending corrupt data to the OPC server. Check for network issues that might corrupt the data packets.
- Check the size of the OPC server group. If the scan class contains more points than permitted in the OPC server group, *Unpack2* errors might result. Consult the OPC server documentation for group size limits.
- If the point is digital, and the data can be read into a string PI point, and the underlying control system is Honeywell, the digital state strings in Data Archive might need to exactly match the string reported by the DCS. To determine the digital states, go to Honeywell Universal Station or GUS to look at each controller block (data source).

OPC server operation

PI Interface for OPC DA can log OPC server interactions in the **opcresponse.log**, **opcscan.log**, and **opcrefresh.log** files. To interpret the information in these files, you must understand the basic interface architecture. PI Interface for OPC DA has two threads of operation:

- PI thread
Interacts with Data Archive.
- COM thread
Interacts with the OPC server.

Polled PI points

For polled PI points, the interface notifies the PI thread when it's time to scan. The PI thread starts the data collection process and logs the time, group number, and current flag value in **opcscan.log**, then sets the flag. (If the flag in **opcscan.log** is non-zero, the last call made to the server did not return before the interface initiated another poll, and data might have been missed as a result.)

When the COM thread detects that the flag is set, it logs the time, group number and transaction ID in the **opcrefresh.log** file and makes a refresh call to the OPC server. When it receives the synchronous response from the OPC server, it clears the flag.

Now the OPC server can send data at any time, in an asynchronous manner. When the OPC server sends data to the interface COM thread, the time, group number and transaction ID are logged in **opcresponse.log**.

Advise PI points

For advise PI points, the COM thread receives callbacks only when the data from OPC server changes value. Therefore, advise points do not generate entries in the **opcscan.log** or **opcrefresh.log** files, and only the data callbacks are logged in the **opcresponse.log** file. Advise points can be identified in the **opcresponse.log** file by group numbers that range from 200 to 800.

Interface specific messages

For OPC DA specifically, all interface-specific states have a device status value of 70. One way to troubleshoot communication problems between the OPC Server and the interface is to configure the Unlnt health tag for device status. OPCDA sends the following interface specific device status messages.

- *70 | Too many invalid item IDs*

The percentage of OPC items that the interface has successfully registered to read data from (i.e., added to groups) is below the threshold configured with the */RP* parameter in the interface's .bat file. By default, */RP=80*, meaning that the interface sets this device status if it can add fewer than 80 percent of the target OPC items to groups successfully. This error often indicates that the item IDs configured in the Instrument Tag attribute of the interface's PI points are either syntactically invalid or refer to OPC items that do not exist. Check the PI Message log on the interface machine for details about why the interface cannot add specific points.

- *70 | Error on activate group(s)*

The interface was able to create a group but was unable to activate it. Check the PI Message Log on the interface machine for more details about the circumstances that led to this status.

- *70 | Error on Refresh call(s)*

The interface was unable to scan for data for at least one polled group.

- *70 | One or more groups missing scan(s)*

The OPC server is not acknowledging scan requests for polled groups quickly enough. Specifically, the interface has attempted to scan at least one polled group at least ten times since the OPC server last acknowledged a request.

- *70 | One or more groups not getting data*

The OPC server has not sent data for one or more polled groups in the last */RT* scan intervals, where */RT* is configurable from the ICU. By default, */RT* is set to 10, indicating that the interface will wait 10 scan intervals for data from a polled group before setting this status.

- *70 | One or more groups hit output count warning*

The OPC server is not responding quickly enough to write requests from the interface, and the output queue (whose maximum capacity is configurable from the ICU using the */OD* parameter) has reached the warning level, configured from the ICU using the */OW* parameter.

- *70 | One or more output groups dropping data*

The OPC server is not responding quickly enough to write requests from the interface, and the output queue is now full. The interface will either drop the newest output events or will purge the oldest events from the queue, depending on the sign of */OD*. As a result, some output events will not be written to the OPC server.

OPC refreshes

Logging refreshes

To log OPC refreshes, enable debug option 8, which causes PI Interface for OPC DA to create three log files: **opcscan.log**, **opcrefresh.log**, and **opcresponse.log**. If the interface is running as a service, the files are located in the **%windows%/system32** directory (**%windows%/sysWOW64** for 64-bit systems). Otherwise, the files reside in the working directory for the interface process. The working directory does not need to be the same as the

directory containing the **.exe** file.

When the interface sets the flag for a scan, it logs the current time, the number of the scan class, and the current value of the scan flag in the **opcscan.log** file. The time stamp is in UTC (Greenwich time zone, daylight savings time is not observed), structured as a FILETIME structure written as a 64-bit hexadecimal field. The lower and upper halves of the number are transposed and the actual number is a count of the interval since January 1, 1601, measured in 10E-7 seconds.

After logging the data, the interface sets the scan flag for the group, then the COM thread takes its turn. When the interface cycles around to perform the poll, it logs the time, the scan class, and the TransID used in the **opcrefresh.log** file. For v1.0a server, the TransID logged is the TransID that was returned from the last poll of the group. For v2.0 servers, it is the actual TransID returned from the server.

When the interface receives data from the OPC server, it logs the time, the scan class, and the TransID received in the **opcresponse.log** file. For advise points, no entries are logged in the **opcrefresh.log** and **opcscan.log** files. Only the **opcresponse.log** file is updated.

Time stamps in PI Interface for OPC DA logs are stored in their native format, which is hard to read. To translate the time stamps to a readily readable format, use the following programs, which are installed into the **Tools** sub-directory below the interface directory:

- **opcscan.exe**
- **opcrefresh.exe**
- **opcresponse.exe**

To run one of these programs from the command line, specify the input and output file names. Examples:

```
> opcscan.exe opcscan.log scan.log
> opcrefresh c:\pipc\Interfaces\OPCInt\opcrefresh.log c:\temp\refresh.log
> tools\opcresponse opcresponse.log response.log
```

The utilities display the UTC time stamp that came with the data, both raw and translated, the time stamp translated into local time, both raw and translated, and the PI time sent to Data Archive. For example:

```
response.log 126054824424850000 2000/06/14 18:54:02.485 126054680424850000
2000/06/14 14:54:02.485 960994309.485001 2 1db8
```

To check the time stamp returned from the OPC server, consult these log files. The time stamp is based on 1 January 1600 UTC, so if you see a date around 1600, it indicates that the server is not sending valid time stamps. To configure the interface to create time stamps when it gets the data, use PI ICU to enable the **Interface Provides Timestamp** option on the **OPCInt** page (or edit the batch file and specify the **/ts=N** parameter).

If the interface is running with debugging options 32 or 64 enabled, the log file contains entries for individual data items that were received by the COM thread. For advise points, the group number in the **opcresponse.log** file might not be correct for entries generated by debugging options 32 or 64, although the shorter entries generated by debugging option 8 correspond to the correct group number.

By looking at the log files, you can see when the interface polled for data, when it made the call, and when the data came in. If the flag in **opcscan.log** is non-zero, the last call made to the server did not return by the time the interface started another poll. If you find non-zero flags in the log file, contact your server vendor and have them contact AVEVA.

No OPC server response to refresh calls

To determine whether the OPC server is responding to the refresh calls made by the interface, check the local PI Message Log file for the following message:

The OPC server did not respond to the last refresh call for scan class 2, and has not responded to the previous 100 refresh call(s).

This message indicates that the OPC server failed to respond to a refresh call. This problem occurs when the OPC server cannot keep up with the update rates or has suspended operation due to a bug. The message is repeated for each additional 100 refresh calls that receive responses from the OPC server for each scan class. If these messages appear in your local PI Message Log, data loss might be occurring. Contact your OPC server vendor immediately, and consider the following adjustments to reduce load on the OPC server:

- Move points into the Advise scan class 1.
- Reduce the total number of scan classes for the interface.

Shutdown

The *Shutdown* attribute is 1 (true) by default. The default behavior of the PI Shutdown Subsystem is to write the *SHUTDOWN* digital state to all PI points when PI is started. The timestamp that is used for the *SHUTDOWN* events is retrieved from a file that is updated by the Snapshot Subsystem. The timestamp is usually updated every 15 minutes, which means that the timestamp for the *SHUTDOWN* events will be accurate to within 15 minutes in the event of a power failure. For additional information on shutdown events, refer to Data Archive manuals.

Note: The *SHUTDOWN* events that are written by the PI Shutdown Subsystem are independent of the *SHUTDOWN* events that are written by the interface when the following command-line parameter is specified: **/stopstat=Shutdown**

SHUTDOWN events can be disabled from being written to PI points when the Data Archive is restarted by setting the *Shutdown* attribute to 0 for each point. Alternatively, the default behavior of the PI Shutdown Subsystem can be changed to write *SHUTDOWN* events only for PI points that have their *Shutdown* attribute set to 0. To change the default behavior, edit the **Shutdown.dat** file, as discussed in the Data Archive manuals.

Bufserv and PIBufss

It is undesirable to write shutdown events when buffering is being used. Bufserv and PIBufss are utility programs that provide the capability to store and forward events to a Data Archive, allowing continuous data collection when the Data Archive is down for maintenance, upgrades, backups and unexpected failures. That is, when the Data Archive is shutdown, Bufserv or PIBufss will continue to collect data for the interface, making it undesirable to write *SHUTDOWN* events to the PI points for this interface. Disabling Shutdown is recommended when sending data to a Highly Available Data Archive collective. Refer to the Bufserv or PIBufss manuals for additional information.

OPC server performance issues

Issue

An OPC server with performance issues will cause sporadic PI OPC tags to not update or otherwise appear stale. When the OPC server is having performance issues, the PI Interface for OPC DA version 2.2.2.0 or later reports the following error message in the **pipc.log** file:

The OPC server did not respond to the last refresh call for scanclass N, and has not

```
responded to the previous nnn  
refresh call(s).
```

When the PI OPC interface makes a refresh call to the OPC server, the OPC server should return something back before the interface makes another call. If the OPC Server does not respond before the next call, the interface prints out the message above. When this happens, it is likely (but not always the case) that data is lost for that call.

Solution

Optimize the load on the OPC Server

Depending on the circumstances and preferences, suggestions 1, 2a, and 3 are reasonable starting points.

1. Move tags into Advise scan class (#1) if they are advise tags. Do not mix polled and advise tags.
2. Adjust the size and number of groups. OPC servers handle data in groups. The group size can be adjusted from the OPC interface side in one of two ways:

For tags in scan class 1, the tags will be split into smaller groups according to the setting in ICU under **OPCInt > Data Handling > Number of tags** in advise group. This setting will subdivide all tags in scan class 1 into separate groups.

The second way is to manually create separate groups by placing tags into a separate scan class. For scan classes other than scan class 1, the number of tags in that scan class is the size of the group. This adjustment requires balancing two opposing factors: making each group smaller but also not creating too many groups in total. Contact your OPC Server vendor to find out the maximum number of groups in total the particular OPC server can handle.

- a. Adjusting the size of groups: Create smaller groups to divide data collection into more manageable chunks for the OPC server. For tags in scan class 1, this can be done by going to **OPCInt > Data Handling > Number of tags** in advise group and entering a value between 200~400. For tags not in scan class 1, check that there are no more than 200~400 tags per scan class.
 - b. Adjusting the number of groups: Reduce the total number of groups for the interface. Each scan class causes the creation of a group on the OPC server. If there are any unused scan classes, they should be removed from the interface start-up configuration file. The interface will not ask the OPC Server to perform work for scan classes that do not contain tags; however, what the OPC Server does on its own with that group could impact performance. The number of groups can also be reduced by increasing the number of tags per group in scan class 1. This is accomplished by adjusting **OPCInt > Data Handling > Number of tags** in advise group.
3. Increase the time for each scan rate. The scan rate is sent to the OPC server, and the OPC server makes a good faith effort to keep data in a cache as fresh as the scan rate. However, whether the OPC server is able to reach out to all of the end devices to do so is not guaranteed. Increasing the scan rate provides the OPC server more time to gather data from the data sources before going back out for another device scan.
 4. Create a separate instance, especially if the number of tags on 1 instance is over 10,000. Every OPC server responds differently, and some OPC servers struggle at as low as 5,000 tags. 10,000 tags tends to be a soft limit after which there is increased risk of performance issues for any OPC server. Note that the tags must be placed onto a separate instance, not just a separate scan. Creating separate scans within the same instance is irrelevant if the root cause is an overloaded instance.

Other than the above suggestions, not much more can be done from the PI Interface for OPC DA to help when the OPC server is having issues, and the OPC server may not provide additional clues to the OPC interface beyond not responding to the last refresh call. However, additional solutions are often possible on

the OPC server side. You should contact your OPC server vendor for further assistance if the problem continues.

Command-line parameters

There are two tables that list the command-line parameters used in the interface startup batch file to configure settings. One is organized alphabetically and the other is organized by functionality. These parameters are provided for debugging purposes only, to help you read the file.

To ensure a correctly-formatted file, use [PI Interface Configuration Utility \(PI ICU\)](#) to configure the interface.

Alphabetical list of parameters

The following table contains an alphabetical list of the .bat parameters for the OPC DA interface.

Parameter and syntax	Description
/af=Option	Optional. Configures data handing for advise points. Enable if advise points do not receive a current value on startup. Do not enable this setting if you are using Windows cluster failover, because it causes OPC groups to be advised as soon as they are created. Default: N Options: Y or N Note: Enabling the /af flag when pointing to a Conforming OPC-DA server will result in data loss for Advise tags. This flag specifically targets NON-Conforming OPC-DA servers, which don't give initial values when a group is advised.
/ag=group_name	Optional. Used to configure custom advise group names. To name a particular group, use /ag#=name. For example, /ag1=MyAdvise and /ag2=NextAdvise will instruct the interface to use the name "MyAdvise" for the first advise group and the name "NextAdvise" for the second advise group. To set a base name for advise groups, use /ag=name. If a name is not specified for a particular group using /ag#=name, then that group's name will be the base name with the group number appended to it. For example, if /ag=AdviseGroup and there is no /ag3 specified, the third advise group will be called "AdviseGroup3". The default base name for advise groups is "Advise". If advise group names are not configured, the advise groups will be named "Advise1", "Advise2", "Advise3",

Parameter and syntax	Description
	and so on. Default: /ag=Advise
/am=#	Optional. Specifies the number of points in each OPC advise group created for scan class 1. Intended for managing OPC server workload. Default: 800
/ar=Option	Optional. Enable/disable use of access rights property on items added to a group. If the interface logs the error <i>“Invalid read/write mode requested”</i> , try disabling access rights by setting /AR=N Default: Y Options: Y or N
/as=system_digital	Optional. Assign alternate digital states for questionable and bad qualities. To use this option, it is necessary to create a digital state in the system digital state set that corresponds to the <i>system_digital</i> option of the command line option. Then, any digital states that follow the <i>system_digital</i> argument are used to map digital states to PI points when data with questionable or bad qualities are received from the OPC server, overriding the default digital states. For more information, see Data quality information .
/at=#	How long to wait for write acknowledgement from OPC server (milliseconds). When this time has elapsed, the interface cancels the write and resends it. Minimum: 2000 milliseconds. To disable monitoring for stale writes and attempting to cancel and resend the data then set /at=0. Default: 2000 msec (two seconds)
/backup=hostname::OPCservername	For server-level failover, specifies the name of the backup OPC server. If the OPC server is on the local machine, omit <i>hostname</i> . If your server name has embedded spaces, enclose the name in double quotes.
/cachemode	Enable disconnected startup.
/cachepath=filepath	(Optional) Specifies the directory where point caching files are created for disconnected startup. The directory must already exist on the target machine. By default, the files are created in the same location as

Parameter and syntax	Description
	the interface executable. If the path contains any spaces, enclose the path in quotes. Examples: • /cachepath=D:\PIPC\Interfaces\CacheFiles • /cachepath="D:\Program Files\PIPC\MyFiles"
/cachesync=#	(Optional) Specifies the time slice period in milliseconds (ms) allocated for synchronizing the interface point cache file with Data Archive. By default, the interface synchronizes the point cache if running in the disconnected startup mode. To disable synchronization of the point cache file, specify /cachesync=0. The minimum is 50 milliseconds and the maximum 3000 milliseconds (3s). Values less than the minimum or greater than the maximum are adjusted accordingly. This value must be less than the smallest scan class period. If the value is greater than the smallest scan class value, input scans are missed while the point cache file is being synchronized. Default: 250 milliseconds
/cm=Option	(Optional) Configure cluster-related behavior for interface-level failover running on a cluster. Mode 0 (/cm=0) is preferred-primary mode. Mode 1 (/cm=1) is a non-preferential mode, where whichever interface instance is active stays active until the cluster resource fails over. Default: 0 Options: 0 or 1
/cn=tag_name	(Optional) For interface-level failover running on a cluster, this parameter specifies a string PI point that receives the node name of the interface instance that is currently gathering data. This feature enables you to track the cluster node that is the active interface node. Ensure that the point source of the specified point is not in use by any interface.
/credfilepath=path\to\cred\file.cred	(Optional) Path to credential file for OPC Private Security. Set by the ICU Control, it is recommended that the path not be modified manually. The ICU provides an OPC Security Credentials file with a .cred extension. This file must be populated

Parameter and syntax	Description
	manually with the OPCUser and OPCPassword.
/cs=tag_name	(Optional) PI point that tracks the currently-active OPC server for failover. Ensure that the point source of the specified point is not in use by any interface.
/ct=#	(Optional) Specifies the time in seconds to wait for a response from the OPC server before concluding that the connection to the server has failed and needs to be reset. In such a case, the interface attempts to reconnect every /CT second until it connects successfully, at which time it recreates all groups and re-subscribes for updates to advise points. The interface ignores this parameter when server-level failover is configured. Instead, it uses the /FT parameter to determine how long to wait before timing out and failing over to the alternate server.
/da=option	(Optional) Configures default authentication level, part of DCOM security settings for the interface. This parameter sets the interface-specific authentication level required to verify the identity of the OPC server during calls. Valid <i>option</i> values are as follows: • DEFAULT • NONE • CONNECT • CALL • PKT • PKT_INTEGRITY (default) • PKT_PRIVACY Use this setting with the <i>di</i> parameter. If you set /di and omit /da, PKT_INTEGRITY is used. If neither <i>da</i> nor <i>di</i> is configured, the interface uses the default permissions on the client machine. Default: PKT_INTEGRITY
/db=#	(Optional) Set level of debugging output to be logged. By default, debug logging is disabled. For valid settings, see Debug settings .
/dc	(Optional) Disable callbacks for polled groups, to reduce OPC server workload. No effect on advise groups. By default, callbacks are enabled.

Parameter and syntax	Description
/df=tag_name	(Optional) Configure a PI point that contains the debug level, which allows you to change debug level while the interface is running. Configure an Int32 output point for the interface, and set its value to 0, then configure the point using the <i>df</i> parameter. After starting the interface, you can change debug level by setting the point to the desired level. For valid settings, see Debug settings. For <i>instrumenttag</i>, you are required to enter a value, but the value is ignored and need not be a valid OPC ItemID.
/di=level	(Optional) Sets the interface-specific impersonation level to be granted to an OPC server to perform processing tasks on behalf of the interface. Default Impersonation level is one of the DCOM security settings for the interface. Valid authority levels are: • ANONYMOUS • IDENTIFY (default) • IMPERSONATE • DELEGATE Use with the <i>da</i> parameter. If you specify the <i>/da</i> parameter and omit <i>/di</i>, the default IDENTIFY is used. If neither parameter is set, the interface uses the computer's default DCOM security settings. Default: IDENTIFY
/dll=drive:\path\filename.dll	(Optional) Configure a post-processing DLL. The default path is the Plugins sub-directory of the interface installation directory. You cannot configure more than one plug-in.
/dllconfig=config_file_name or /dll_ini=config_file_name	(Optional) Specifies the directory path and file name of the configuration file for a post-processing DLL. (Some post-processing DLLs do not require a configuration file.)
/dt=tag_name	(Optional) Specify the point for which detailed information is to be logged when using the verbose debug level. (/DB=64). If you omit this setting, the interface uses the first point for which it receives a value.

Parameter and syntax	Description
/ec=#	(Optional) Specify a counter number for an I/O rate point. If you require multiple interface instances with event counters, specify a different counter number for each instance. Must correspond to a PI point in the iorates.dat file. If you specify /ec and omit the argument, the default counter is 1. Default: /ec=1
/eg=group_name	Optional. Used to configure custom event group names. To name a particular group, use /eg#=name. For example, /eg1=MyEvent and /eg2=NextEvent will instruct the interface to use the name “MyEvent” for the first event group and the name “NextEvent” for the second event group. To set a base name for event groups, use /eg=name. If a name is not specified for a particular group using /eg#=name, then that group’s name will be the base name with the group number appended to it. For example, if /eg=EventGroup and there is no /eg3 specified, the third event group will be called “EventGroup3”. The default base name for event groups is “Event”. If event group names are not configured, the event groups will be named “Event1”, “Event2”, “Event3”, and so on. Default: /eg=Event
/er=hh:mm:ss	(Optional) Specifies the requested update rate for the event scan class group. (All event-based points belong to the same group.) The default update rate for the group is one second. If the OPC server data cache for event-based tags does not need to be updated that frequently, reduce workload by specifying a lower rate. For v2.0 OPC servers, all event reads are done from the device, so set this rate high (/er=24:00:00) unless you require more frequent updates to the cache for other reasons. Default: 00:00:01
/es=option	(Optional) For V1.0a OPC server, specifies whether event tag reads come from the cache (CACHE) or the device (DEVICE). Device reads can adversely affect the performance of the OPC server. For details, refer to Event points .

Parameter and syntax	Description
	Default: CACHE
/f=frequency[,offset]	Define a scan class, specify how often the data in the class is scanned. Specify scan frequency and optional offset using the following format: <i>HH:MM:SS.##,HH:MM:SS.##</i> . For details, see Scan class attribute (location4) .
/fm=#	(Optional) Configure type of interface-level failover. Valid options are: • 1: Do not create groups on the server • 2: Create inactive groups and add tags • 3: Create active groups, but do not advise groups For details, see Failover configuration . Default: 3
/ft=#	(Optional) Specifies (in seconds) how long the interface tries to connect to the current server before failing over to the server specified by the <i>backup</i> parameter. If the specified value is less than 30, also sets how often the interface checks server state. By default (and, at a minimum), the interface checks server state every 60 seconds. Default: 60
/ga	(Optional) Staggers group activation to reduce OPC server workload. Use in conjunction with scan class offsets.
/gi	(Optional) To reduce load on the OPC server during startup, inactivates groups until they are built.
/gl=Option	(Optional) Fix for some early v1.0a servers. If the log contains the error message <i>OnDataChange: Invalid group ID</i> , try setting /gl=N. Default: Y Options: Y or N
/gs=Option	(Optional) Fix for older, non-compliant OPC servers that do not provide a valid GroupStatus on asynchronous reads. If the log file contains the error message <i>OnDataChange: Header status</i> , try setting /gs=N to tell the interface to ignore the group status parameters.

Parameter and syntax	Description
	Options: Y or N
/host=host:5450	(Required) Specifies the host and port of the Data Archive node to which the interface sends data. Host is the node name of the host node.
/hs=Option	(Obsolete) Request a cache update rate of one half of the scan rate for the scan class. Use <i>ur</i> instead. Options: Y or N
/hwps	(Optional) Check for Plantscape-specific Item error codes <i>0xE00483FD</i> or <i>0xE00483FC</i> and, if found, failover to alternate OPC server.
/id=#	(Optional) Specifies the ID of the interface instance. The interface uses this setting together with /ps to determine which points to load. If /id >0 the interface will load only tags with Location1 matching /id. If /id=0, the interface will load all points with PointSource matching /ps. Maximum nine digits. Optional, but highly recommended.
/if=Option	(Optional) Ignore the first value sent for each point. For use with OPC servers that send a response when the interface connects to a point, regardless of whether they have a valid value. Default: N Options: Y or N
/is=Option	(Optional) Ignore the state returned by the OPC server. Some OPC servers do not return <i>OPC_STATUS_RUNNING</i> when ready. If the interface hangs on startup and PI OPC Client tool displays an OPC server state other than <i>RUNNING</i> , set /is=Y and report the issue to your OPC server vendor. Default: N Options: Y or N

Parameter and syntax	Description
/isds =x	(Optional) Hide device status messages that are specific to the OPC DA Interface set x equal to the values below: 1: Suppress "One or more groups not getting data" 2: Suppress "One or more groups missing scan(s)" 4: Suppress "One or more groups hit output count warning" 8: Suppress "Error on Refresh calls(s)" 16: Suppress "Error on activate group(s)" 32: Suppress "Too many invalid ItemIDs" 64: Suppress "One or more output groups dropping data"
/it=Option	(Optional) To truncate the sub-second portion of the time stamps being passed to PI Data Archive and only send whole integer seconds, set to Y. Reduces CPU and disk consumption. Default: N Options: Y or N
/ma=Option	(Optional) By default, the OPC interface adds items to groups one at a time, because some OPC servers reject an entire group if one item is invalid. To add all the items in a class at the same time, set to Y. Recommended for efficiency if supported by your OPC server. Default: N Options: Y or N
/maxstoptime=#	(Optional) Specifies how many seconds are allocated for the interface to close its connections and exit cleanly. Default: 120 seconds
/ni=#	(Optional) Specifies the number of instances of the interface running on the node. Used in conjunction with the <i>ft</i> parameter to determine how long to wait before initiating server-level failover.

Parameter and syntax	Description
/nt=Option	(Optional) Write/do not write <i>I/O Timeout</i> to PI points when the connection to the OPC server is lost. Set to Y to disable writing. Default: N Options: Y or N
/oc=#	Maximum number of outstanding outputs per group. After issuing the specified number of writes, the interface waits for one or more to be acknowledged before issuing any additional writes.
/od=#	(Optional) Level at which to start dropping outputs. When the output queue for a group contains the specified number of outputs, the interface will start dropping queued data: • To drop the oldest queued outputs, specify a negative value. • To drop the newest queued outputs, specify a positive value. • To drop all data when the connection to the OPC Server is down, specify zero. Once the connection is restored, data will no longer be dropped.
/og=#	Number of output groups. Each group has its own queue. Default: 1
/opcstopstat=system_digital_state	(Optional) To indicate that data collection was stopped when the interface is shut down, writes digital state to each input tag. If digital state is omitted, "I/O Timeout" is written. If digital state is specified, it must be a valid state from the system digital state set. Caution: Do not use the <code>Unlnt stopstat</code> parameter with the OPC interface. The <i>stopstat</i> can cause invalid values to be stored in PI points. Default: "I/O Timeout"
/ot=#	Maximum number of point values to write at one time. Default: 36

Parameter and syntax	Description
/ou=group_name	Optional. Used to configure custom output group names. To name a particular group, use /ou#=name. For example, /ou1=MyOutput and /ou2=NextOutput will instruct the interface to use the name “MyOutput” for the first output group and the name “NextOutput” for the second output group. To set a base name for output groups, use /ou=name. If a name is not specified for a particular group using /ou#=name, then that group’s name will be the base name with the group number appended to it. For example, if /ou=OutputGroup and there is no /ou3 specified, the third output group will be called “OutputGroup3”. The default base name for output groups is “Write”. If output group names are not configured, the output groups will be named “Write1”, “Write2”, “Write3”, and so on. Default: /ou=Write
/outputsnaptime	(Optional). For output tags, use the timestamp from the original event that triggered the output. By default, the interface uses the time it receives a new value as the timestamp for storing the output after it has completed.
/outputacktime	(Optional) For output tag timestamps, use "acknowledge" time rather than the timestamp from the event that triggered the output. Uses the timestamp from the OPC server's acknowledgement of the write. Overrides <i>outputsnaptime</i>, if enabled.
/ow=#	(Optional) Number of pending outputs at which to set Device Status to warn that outputs are arriving faster than the OPC server can process them. If <i>ow</i> and <i>od</i> are both specified, <i>ow</i> must be less than <i>od</i>.

Parameter and syntax	Description
/pg=group_name	Optional. Used to configure custom polled group names. To name a particular group, use /pg#=name. For example, /pg1=MyPolled and /pg2=NextPolled will instruct the interface to use the name “MyPolled” for the first polled group and the name “NextPolled” for the second polled group. To set a base name for polled groups, use /pg=name. If a name is not specified for a particular group using /pg#=name, then that group’s name will be the base name with the group number appended to it. For example, if /pg=PolledGroup and there is no /pg3 specified, the third polled group will be called “PolledGroup3”. The default base name for polled groups is “Poll”. If polled group names are not configured, the polled groups will be named “Poll1”, “Poll2”, “Poll3”, and so on. Default: /pg=Poll
/pisdck=#	(Optional) Enable (1) or disable (0) the AF SDK. If required by a PI System interface, the AF SDK is enabled and cannot be disabled using this setting.
/pisdckcontimeout=#	(Optional) Set the number of seconds to wait before timing out on AF SDK calls. Default: 15
/pr=#	Configure cluster failover for the interface instance: • 0: No cluster failover (default) • 1: Primary interface instance • 2: Backup interface instance Default: 0
/ps=point_source	(Required) Specifies the point source for the interface instance. Not case sensitive. The interface instance uses this setting to determine which PI points to load and update.

Parameter and syntax	Description
/pw=password(DEPRECATED)	You must use the ICU to configure a password for private OPC security. The ICU provides an OPC Security Credentials file with a .cred extension. This file must be populated manually with the OPCUser and OPCPassword.
/rd=#	(Optional) Number of seconds to wait before trying to reconnect to the OPC server.
/rn=#	(Optional) Specifies the resource number of the APlonline service that the interface depends on. For example, /rn=1 configures the interface to depend on apionline1. Required if there are multiple instances of PI Interface for OPC DA running with different service names on the same machine. For configuring Windows Cluster failover.
/rp=#	Specifies the minimum percentage of points required to be accepted by the OPC server as valid. If less than the specified percentage is accepted, the interface sets its device status to <i>Connected/No Data</i> , which triggers Unilnt failover if configured. Default: 80
/rt=#	Polled scan classes only: The maximum number of scans that can return no data before the group is assumed to be non-responsive and the interface sets the Device Status of the interface to warn of the problem. Minimum is 2 scan periods. Default: 10
/sd=#	(Optional) Specifies how many seconds to wait after connecting before reading OPC points. By default, there is no delay after connecting. Default: 0

Parameter and syntax	Description
/sec or /sec=interfacename + interfaceid	(Optional) To enable the NT security option of the OPC standard, specify /sec (omit interfacename + interfaceid). If you are using the OPC private security option, this parameter is set by the ICU based on the interfacename and interfaceID. The ICU provides an OPC Security Credentials file with a .cred extension. This file must be populated manually with the OPCUser and OPCPassword.
/server=host::name	(Required) Configures the target OPC server. If the OPC server runs on the same machine as the interface, omit host name and colons. If your server name has embedded spaces, enclose the name in double quotes.
/sg=[S]	(Optional) Send only <i>GOOD</i> quality data. Questionable quality data and <i>BAD</i> quality data are ignored. To ignore substatus for values that have <i>GOOD</i> status, specify /sg=s. To treat OPC_QUALITY_LOCAL_OVERRIDE as <i>SUBSTITUTED</i> , specify /sg. To treat OPC_QUALITY_LOCAL_OVERRIDE as <i>GOOD</i> , specify /sg=s. If the /sq=I or /sq=y parameter is also set, questionable quality data is sent to Data Archive. <i>BAD</i> quality data is ignored. Quality information continues to be sent to points that are configured to store quality instead of values.
/sin=node	(Obsolete) Specifies the name of the secondary interface node for cluster failover.
/sq=Option	(Optional) By default, the interface stores and flags uncertain-quality values. To store quality data instead of value for data that is not <i>GOOD</i> , specify /sq=Y. To store questionable data, specify /sq=I. For <i>BAD</i> quality data, the interface sends a digital state code to the PI point. Default: N Options: Y or I

Parameter and syntax	Description
/startup_delay=#	(Optional) Configures a post-startup delay. The interface waits for the specified period before beginning operation. Intended for use if you have configured the interface for auto start and the network layer needs time to complete startup before it becomes available. If you specify /startup_delay and omit the delay, a thirty-second delay is configured. Default: 30
/sw=#	(Optional) Specifies how long (in seconds) the interface waits for the OPC server to enter the <i>OPC_STATUS_RUNNING</i> state. If the specified period elapses and the OPC server is not running, the interface fails over to the alternate OPC server.
/ta=#.#	(Optional) For advise PI points, send the value from the preceding scan if the new value's time stamp is greater than the specified number of scan periods. Enabling this setting causes advise tags to behave as if the <i>Step</i> attribute is enabled.
/tf=format	Specifies the format of time stamp strings. (Required) if PI points are configured with: • TIM= in the <i>exdesc</i> PI point attribute. • <i>location2</i> = 6 or 7, where the ItemID is either a string that contains a time stamp or a VT_DATE value. Note: If you do not specify /tf=format on the command line, but <i>do</i> have either of the above PI point configuration options then the message, "No time format specified on command line", will be written to the error log file for those PI points. Otherwise (Optional)
/to=HH:MM:SS	(Optional) Applies an offset to all time stamps coming from the server. Provided to deal with servers and installations that do not follow the OPC specifications (for example, where the time zone must be UTC regardless of the location of the server). The format is the same as the scan period (<i>f</i>) parameter. For negative offsets, precede the offset with a minus sign. Note: PI Interface for OPC DA checks the system time

Parameter and syntax	Description
	of a machine with running Data Archive every 10 minutes and updates the current offset.
/ts=Option	(Optional) Specifies whether time stamps come from the OPC server or are applied by the interface when the data arrives. By default, the interface provides time stamps (/ts=N). If the OPC server can provide valid time stamps, specify /ts=Y. If the OPC server can provide valid time stamps only for advised points, specify /ts=A. To only use time stamps provided by the OPC server and apply no time offset, specify /ts=U. Default: N Options: Y or N or A or U
/ufo_id=#	(Required for Unilnt interface failover phase 1 or 2) Specifies failover ID. Must be a unique, positive integer.
/ufo_interval=#	(Optional) Specifies in milliseconds how often the failover heartbeat points are updated and interface status is checked. Must be the same on both interface nodes. Minimum: 50 Maximum: 600000 milliseconds (10 minutes) Default for Phase 1 failover: 1000 Default for Phase 2 failover: 5000
/ufo_otherid=#	Failover ID of the other interface instance. Required for phase 1 or 2 interface-level failover.

Parameter and syntax	Description
/ufo_sync=path/[file_name]	(Required for phase 2 interface level failover) Path and, optionally, name of the shared file containing the failover state. The <i>path</i> can be a fully qualified node name and directory, a mapped drive letter, or a local path if the shared file is on an interface node. The <i>path</i> must be terminated by a slash or backslash character. The default filename is: <i>executablename_pointsource_interfaceID.dat</i> . If there are any spaces in the <i>path</i> or <i>filename</i> , the entire path and filename must be enclosed in quotes. If you enclose the path in double quotes, the final backslash must be a double backslash \\.
/ufo_type=type	(Required for phase 2 interface level failover) Specifies the type of failover configuration: HOT, WARM, or COLD. Note: "WARM" is not set in the /ufo_type switch through the command line, but rather is the failover mode employed when /ufo_type=HOT and the /FM switch are configured. The /FM switch instructs the interface to use "WARM", identifying the type of "WARM" failover mode to be used.
/uht_id=#	(Optional) Specifies a unique ID for interface instances that are run in a redundant mode without using the Unilnt failover mechanism. If the ID is specified, only health points with the specified value in <i>location3</i> are loaded.
/ur=HH:MM:SS.000	(Optional) Specifies the requested update rate for the group. By default, the update rate requested for a scan class is the same as the rate. The update rate is applied to the scan period that it follows. For example: • /f=00:00:02 - Update rate two seconds • /f=00:00:03 /ur=00:00:00.5 - Update rate 0.5 seconds • /f=00:00:01 - Update rate one second
/us	(Optional) If the current snapshot is a system digital state and the new value is older than the snapshot, the interface sends the new value to Data Archive one second after the snapshot time stamp of the system digital state. This check is not done if the current snapshot is a good value.

Parameter and syntax	Description
/uwq=#	Specifies the maximum number of watchdog PI points that can have an error or <i>BAD</i> quality before failover is triggered.
/vn=Option	(Optional) Specifies the OPC server version (v1.0a or V2.0). Default: 2 Options: 1 or 2
/wd=#	(Optional) For configuring failover using multiple watchdog points, trigger failover if the sum of the values of the points drops below the specified value.
/wd1=wd1tagname and /wd2=wd2tagname	(Optional) Configure watchdog points for failover. For more information, see Server status and OPC watchdog points .
/whitelist=path\filename	Enables allowlist file for output points. Specifies the path to the allowlist file. The allowlist file specifies a list of permitted output points and their attributes, which the interface uses to write output data to the data source.
/wq=#	(Optional) For configuring failover using multiple watchdog points. Directs the interface to fail over if the number of watchdog points with quality other than <i>GOOD</i> exceeds the specified value (and, for v2.0 servers, if there is an error reading the item).
/ws=Option	(Optional) For failover, set to 1 to configure the interface instance to disconnect from the server if the server leaves the OPC_STATUS_RUNNING state. By default, the interface stays connected. Default: 0 Options: 0 or 1

Parameters by function

The parameters are grouped by how they are used, and are specific to the OPC DA interface, except for the Unilnt parameters that are common to all AVEVA Unilnt-based interfaces.

Parameters are not case-sensitive.

Common Unilnt Parameters	Advanced Parameters	Data Handling
/CACHEMODE /CACHESYNC /EC /F /ID /MAXSTOPTIME /PISDK /PISDKCONTIMEOUT /PS /STARTUP_DELAY /WHITELIST	/AM /AR /AT /DC /ES /GA /GI /GL /GS /HWPS /IS /ISDS /MA /OC /OD /OG /OT /OUTPUTACKTIME /OUTPUTSNAPTIME /OW /RD /SD	/AF /AS /ER /HOST /IF /IT /NT /OPCSTOPSTAT /SG /SQ /TA /TF /TO /UR /US
DCOM Security	Plug-ins (Post-processing DLLs)	Debugging
/DA /DI	/DLL /DLLCONFIG	/DB /DF /DT
Server-level Failover	Unilnt Interface-level Failover	Interface-level Failover
/BACKUP /CS /FT /NI /SW /WS /WD	/UFO_ID /UFO_INTERVAL /UFO_OTHERID /UFO_SYNC /UFO_TYPE /UWQ /RP	/CM /CN /FM /PR /RN /UHT_ID

Server-level Failover	Unit Interface-level Failover	Interface-level Failover
<i>/WD1</i> <i>/WD2</i> <i>/WQ</i> <i>/WS</i>	<i>/RT</i>	
OPC Server	Miscellaneous	
<i>/Server</i> <i>/TS</i> <i>/VN</i>	<i>/CREDFILEPATH</i> <i>/PW</i> <i>/SEC</i>	

Review error and informational messages

Messages are logged as follows:

- During startup: Messages include the version of the interface, the version of UniInt, the command line parameters used, and the number of points.
- During point retrieval: Messages are sent to the log if there are problems with the configuration of the points.
- If debugging is enabled.

For more information about message logs, see the *PI Universal Interface (UniInt) User Guide*.

Logging

PI Interface for OPC DA logs messages about its operation in the local PI Message Log file.

The following information is logged:

- Startup and shutdown status messages
- The scan rate configured for each scan class and the actual update rate provided by the OPC server
- The number of PI points in each scan class, output points, and advise and event tags
- Misconfigured points
- Points rejected by the OPC server (and other error messages from the OPC server)
- OPC server connection attempts and results, including loss of connectivity

Message Logs

The message log contains messages from PI Interface for OPC DA, the UniInt framework, and the PI API.

The following table describes only messages from the interface. If any error message has a PI point number as well as a point name, use the point number to identify the problem point, because long point names are truncated to 12 characters.

Informational

Message	Meaning
<i>Shutdown Advise Failed</i> <i>No ConnectionPoint for OPCShutdown</i>	The OPC server does not implement the Shutdown interface or does not implement it properly. Does not prevent proper operation of the interface.
<i>QueryInterface:IID_IconnectionPointContainer failed, using v1.0a protocol</i>	The OPC server does not support OPC DA v2.0.
<i>GetStatus: Server has no current time.</i>	Indicates a non-OPC-standard server that does not send the time of day. The OPC specifications state that the server is supposed to include current time when it sends its status. The interface guesses time stamps, but its accuracy is likely to be questionable.
<i>Cleaning up connections</i> <i>Cleaned up connections</i>	Indicates that the interface is disconnecting and exiting.
<i>Server sent shutdown notice.</i>	Interface received a shutdown notification from the OPC server. The interface periodically tries to reconnect to the server, until it is shut down or succeeds in connecting.
<i>Got %d and cleared it</i> <i>ClearWrite: dwTransID mismatch, have %d, got %d</i> <i>Stashing transid %d</i> <i>Sending transid %d</i> <i>Writing transid %d</i>	Debug level 4 messages indicating that the server acknowledged the specified write and the interface is clear to send another write value. The %d variable is a placeholder for a decimal integer value.
<i>Can't find status tag, ignoring</i> <i>Can't find queue tag, ignoring</i> <i>Status tag is not Digital tag, ignoring</i> <i>Queue tag is not Integer tag, ignoring</i>	Status/queue point does not exist or its data type is incorrect.
<i>Can't connect to OPC Server, going into slow cycle wait</i>	Interface attempted to connect to OPC server. Check for other messages that contain details about the reason for the failed attempt. Interface periodically tries to reconnect.

Message	Meaning
<i>AddItems failed, server not in RUNNING state, will try later</i>	The interface is waiting for the OPC server to enter RUNNING state. You can use the PI OPCClient tool to see the state of the server (use the Get Status button). If the server does not enter the RUNNING state, investigate the cause.

Errors

Message	Cause	Resolution
<i>Out of Memory. Cannot allocate a list; fails. Unable to add tag.</i>	The system has run out of resources.	Use the Windows Task Manager to check the resources being used: press Ctrl+Shift+Esc to open the Task Manager. If you see high memory usage for opcint.exe , there might be a bottleneck between the interface and Data Archive. Look for related messages in the log (see also <i>Running low on memory, dropping data</i>).
<i>CLSIDFromProgID</i>	OPC Server Registry entries are not valid.	Check your server installation instructions.
<i>CoCreateInstanceEx</i>	Indicates a problem with your DCOM configuration.	Check your DCOM settings.
<i>IOPCServer</i>	The proxy stub files are not registered.	To register the opcproxy.dll and opccomn_ps.dll files, you must use an administrator account. Open a command prompt window, change to the directory where the DLLs are installed, and issue the following commands: >regsvr32 opcproxy.dll >regsvr32 opccomn_ps.dll

Message	Cause	Resolution
<i>AddRef</i>	Indicates that the OPC server does not let the interface perform the simplest function.	If you can read and write points using PI OPCClient but this error is logged, check your DCOM settings, check what user account the interface is running under, and try running the interface interactively.
<i>Advise returns error: 80040202(Unable to open the access token of the current thread)</i>	Advise errors indicate that the OPC Server could not initiate a connection back to the OPC Interface (or other OPC client like the OPC Client Tool). Advise error 80040202 can have many root causes: • Network/authentication problems prevent the OPC server from establishing an IP connection or Windows login back to the interface node. • An OPC Server that is not in a good state. The following are possible causes of connectivity/login issues: • There is a firewall between the OPC server and client and the connection is blocked. • There is a name resolution error and the OPC Server cannot resolve the name of the OPC Interface machine.	To verify network connectivity, on the interface node: obtain the interface node's hostname and IP address using IPConfig /all. On the OPC server, issue the command: tracertinterface_nodename (tracert will resolve the hostname to an IP address. Compare the IP address with the actual address(es) of the interface node. If they do not match, the problem is confirmed.) The following tests can be used to identify the root cause of a network issue: • issue the command nslookup for the hostname and each real IP address on the OPC server. Do they resolve? • Check the hosts file on the OPC server for bad entries. • Verify that two hosts with the same IP address do not exist on the network.

Message	Cause	Resolution
<i>Advise returns error: 80040202(Unable to open the access token of the current thread) (continued)</i>	The interface node cannot authenticate the user account associated with the OPC Server. Enable Windows security auditing on the OPC Interface machine to confirm. This might be caused by incorrect group policy settings. If using a local userid, make sure the userid and password matches on the OPC Interface machine and OPC Server machine. If you see this error after connecting to the server successfully, the wrong opcproxy.dll might be loaded.	Verify that there is no firewall between the server and interface. Firewalls can exist in the operating system or on the router itself. Note that simple connectivity tests may not be enough; if the firewall is performing network address translation (NAT), DCOM will fail even if other connections work. If network connectivity is OK, and authentication works, then the OPC Server might be in a bad state. Try restarting the OPC Server. If you have multiple copies of opcproxy.dll on your interface node (possibly because you have more than one OPC server on your machine), make sure that they are the same version. It is safest to have only one version around on your system (in the %windows%\system32 or %windows%\sysWOW64 directory). Check whether directories containing older versions are specified in the system path.
<i>AddGroup failed for scanclass %d</i> <i>AddItem failed for %s</i> <i>AddItems failed for tag %s</i> <i>Advise Group failed for %s</i> <i>Advise returns E_OUTOFMEMORY</i>	These are fatal errors returned by the OPC server.	Try the same operation using PI OPCClient. If successful, run the interface interactively to see if the same error occurs. If successful, check your DCOM configuration to make sure that you have granted required permissions to the INTERACTIVE

Message	Cause	Resolution
<i>Advise returns E_UNEXPECTED</i> <i>Advise returns error</i> <i>No ConnectionPoint for scanclass %d</i> <i>QueryInterface:IID_IdataObject failed for scanclass %d</i> <i>QueryInterface:IID_IOPCAsyncIO 2 failed for scanclass %d</i> <i>Read: (string)</i> <i>Refresh: (string)</i> <i>Unable to add to group</i> <i>Unable to add to OPC group.</i> <i>Unable to advise event group</i> <i>Unable to advise group</i> <i>Unable to advise output group</i> <i>Unable to create group</i> <i>Write error %X for tag</i> <i>Write failed</i>		account. <i>c0040004</i>: Indicates that the requested data type cannot be returned for this item. Use PI OPCClient to add that Item to a group, omitting data type. The server will send you values using the data type that it uses internally for that item. <i>c0040007</i>: Returned from AddItem, indicates that the server does not have the specified item. Verify that the <i>instrumenttag</i> field of the PI point matches the OPC item name exactly. Using PI OPCClient, try to add the Item to a group, or if your OPC server supports browsing, browse to the item and double-click on it to display its full name.
<i>Invalid read/write mode requested for tag %s</i>	The server is returning invalid information about read/write access.	To tell the interface to ignore this information, specify the /ar=N parameter.
<i>RemoveItem failed for tag %s</i> <i>dev_remove_tag: Unable to unadvise %s</i> <i>dev_remove_tag: Unable to remove group %s</i>	The server won't remove an item from a group or stop collecting data for a group.	Not a serious problem unless accompanied by lots of other messages, but indicates some problem with the OPC server.
<i>Write unable to get values: Getsnapshotx error %d</i>	The interface was unable to read a value from PI to write to the OPC server.	To verify that Data Archive is running, use apisnap (in the API directory). Verify that the source and target data types and values are compatible.

Message	Cause	Resolution
<i>Event Point has invalid scan class (!= 0)</i> <i>No Item name – instrumenttag and exdesc both empty</i> <i>Nonzero Totalcode requires nonzero Convers</i> <i>Point cannot be write and Read On Change</i> <i>Point has invalid scan class</i> <i>Point has invalid scan class (== 0)</i> <i>ROC Point has invalid scan class (= 0)</i> <i>Square root must be 0, 1 or 2</i> <i>This Totalcode requires Dzero to be specified.</i> <i>Total must be 0,1,2,3,4, or 5</i> <i>Unable to get point type</i> <i>Unable to get source point type.</i> <i>Unable to get square root</i> <i>Unable to get total specs</i>	Indicates that a PI point is improperly configured.	Check the point configuration, especially the attribute indicated.
<i>GetStatus</i>	The OPC server did not respond to a status query. It might be down or disconnected.	Use PI OPCClient to check the status.
<i>Interface failed to write some %s states</i>	When the OPC server shuts down, the interface sends a shutdown status to each point, if configured to do so (/opcstopstat). This error message indicates that the interface was unable to send some or all of them, owing to lack of connectivity to Data Archive or lack of buffering.	You may want to manually enter the digital state for the affected points to indicate that the interface shutdown.

Message	Cause	Resolution
<i>OnDataChange:Invalid group ID < 0</i> <i>OnDataChange:Invalid advise group ID:</i> <i>OnDataChange:Invalid group ID > 999</i> <i>OnDataChange: Header status: OnDataChange has format not Hglobal</i> <i>OnDataChange:Invalid group ID for write completion</i> <i>Unknown access type for group %s</i>	Indicates that the server is sending meaningless data.	To ignore the header status field in incoming data, set /gs=N. Contact your OPC server vendor. Use PI OPCClient to create groups and try AsyncReads and Advises. Check whether incoming data is valid: the meaningless data might not interfere with data collection.
<i>OnDataChange: Bad Timestamp</i>	The interface received an invalid time stamp with data from the OPC server.	Check your OPC server, and use PI OPCClient to display time stamps.
<i>Invalid timestamp for tag: %s, %d and %.36f</i>	The interface received an invalid time stamp with data from the OPC server. Indicates an issue in the OPC server.	Use PI OPCClient to display the item in question. Use Refresh or Advise or AsyncRead to see a time stamp.
<i>Putsnap system error %d, %d</i> <i>Putsnap no longer in system error %d, %d</i>	System error indicating a problem sending data to PI Data Archive.	No action required unless the error persists.
<i>Putsnapsx not implemented %d</i> <i>Getsnapshotx not implemented</i>	Indicates an outdated version of the PI API.	Update your version of the PI API.

Message	Cause	Resolution
<i>Unable to translate string.</i>	Attempt to translate an ASCII string value from a PI point to Unicode failed.	Check the value of the point.
<i>Unable to initialize server object</i>	Privileges are not granting access.	Verify that account has sufficient privileges.
<i>No OPC Server specified</i>	The <i>server</i> parameter specifies a non-existent server or the invocation is on multiple lines.	Verify that the interface batch startup file specifies valid <i>server</i> parameter and that the invocation is a single line.
<i>Unable to create clock drift timer</i>	Interface cannot create a timer to track drift.	Check system resources.
<i>Running low on memory, dropping data</i> <i>Memory load within acceptable limits, resuming data collection</i>	If Data Archive cannot accept data as quickly as the interface can send it, the interface buffers data in memory. To avoid consuming all available memory, the interface starts dropping data as the limits configured by <i>hq</i> and <i>lq</i> are approached.	Consider giving the PI System more resources, changing the exception parameters of your points, or changing the scan period of your points.

Message	Cause	Resolution
<i>Failed to open cluster: error #####. Intf-failover will not be supported.</i> <i>Failed to open cluster resource: error #####. Intf-failover will not be supported.</i>	Win32 error code indicating that an attempt to open the cluster service or resource failed.	Check the cluster settings.

Critical errors

Message	Cause	Resolution
<i>Error from Colnitalize:</i> <i>Error from ColnitalizeSecurity:</i>	COM might not be properly installed on your system.	If true, this is a serious problem. First check your COM setup. You may also need to contact AVEVA Technical Support.
<i>Cannot get PI Server time.</i>	Cannot connect to Data Archive.	For newly-installed systems, reboot and verify that you can connect to PI Data Archive. To verify connectivity, ping the PI Data Archive machine. To verify that PI Data Archive is running, use apisnap. For existing installations, contact AVEVA Technical Support.
<i>OnDataChange: VariantCopy</i>	Indicates that the OPC server sent meaningless data. Interface rejects the data and writes <i>BADSTAT</i> to the point. Time stamp is good.	Check the data with PI OPCClient.

System errors and PI system errors

System error numbers are positive. PI System error numbers are negative.

To display descriptions of system and PI System errors, use the **pidiag** utility:

```
\PI\adm\pidiag -e error_number
```

Unilnt failover-specific messages

Informational

Message	Meaning
<i>16-May-06 10:38:00 OPCInt 1> Unilnt failover: Interface in the "Backup" state.</i>	Upon interface startup, the initial transition is made to the backup state. In this state the interface monitors the status of the other interface participating in failover. When configured for hot failover, data received from the data source is queued and not sent to Data Archive while in this state. The amount of data queued while in this state is determined by the failover update interval. In any case, there will be typically no more than two update intervals of data in the queue at any given time. Some transition chains can cause the queue to hold up to five failover update intervals worth of data.
<i>16-May-06 10:38:05 OPCInt 1> Unilnt failover: Interface in the "Primary" state and actively sending data to PI. Backup interface not available.</i>	In primary state, the interface sends data to Data Archive as it is received. This message also indicates that there is not a backup interface participating in failover.
<i>16-May-06 16:37:21 OPCInt 1> Unilnt failover: Interface in the "Primary" state and actively sending data to PI. Backup interface available.</i>	While in this state, the interface sends data to Data Archive as it is received. This message also states that the other copy of the interface appears to be ready to take over the role of primary.

Error messages (Phase 1 & 2)

Message	Cause	Resolution
<i>16-May-06 17:29:06 OPCInt 1></i>	The Active ID tag is not	Check validity of point

Message	Cause	Resolution
<i>One of the required Failover Synchronization points was not loaded. Error = 0: The Active ID synchronization point was not loaded. The input PI tag was not loaded.</i>	configured properly.	attributes. Make sure the <i>location1</i> attribute is set to the id parameter of the interface. All failover tags must have the same <i>pointsource</i> and <i>location1</i> attributes. Modify point attributes as necessary and restart the interface.
<i>16-May-06 17:38:06 OPCInt 1> One of the required Failover Synchronization points was not loaded. Error = 0: The Heartbeat point for this copy of the interface was not loaded. The input PI tag was not loaded.</i>	The heartbeat tag is not configured properly.	Check validity of point attributes. Make sure the <i>location1</i> attribute is set to the id parameter of the interface. All failover tags must have the same <i>pointsource</i> and <i>location1</i> attributes. Modify point attributes as necessary and restart the interface.
<i>17-May-06 09:06:03 OPCint > The UInt FailOver ID (/UFO_ID) must be a positive integer.</i>	The <i>ufo_id</i> parameter has not been assigned a positive integer value.	Change and verify the parameter to a positive integer and restart the interface.
<i>17-May-06 09:06:03 OPCInt 1> The Failover ID parameter (/UFO_ID) was found but the ID for the redundant copy was not</i>	The <i>ufo_otherid</i> parameter is not defined or has not been assigned a positive integer value.	Change and verify the <i>ufo_otherid</i> parameter to a positive integer and restart the interface.

Message	Cause	Resolution
<i>found</i>		

Errors (Phase 1)

Message	Cause	Resolution
<i>17-May-06 09:06:03 OPCInt 1> Unilnt failover: Interface in an "Error" state. Could not read failover control points.</i>	The failover control points on the data source are returning an erroneous value to the interface. This error can be caused by creating a non-initialized control point on the data source. This message will only be received if the interface is configured to be synchronized through the data source (Phase 1).	Check validity of the value of the control points on the data source.
<i>16-May-06 17:29:06 OPCInt 1> Loading Failover Synchronization tag failed Error Number = 0: Description = [FailOver] or HeartBeat:n] was found in the exdesc for Tag Active_IN but the tag was not loaded by the interface. Failover will not be initialized unless another Active ID tag is successfully loaded by the interface.</i>	The Active ID or heartbeat tag is not configured properly. This message is received only if the interface is configured to be synchronized through the data source (Phase 1).	Check validity of point attributes. Make sure the <i>location1</i> attribute is set to the id parameter of the interface. All failover tags must have the same <i>pointsource</i> and <i>location1</i> attributes. Modify point attributes as necessary and restart the interface.
<i>17-May-06 09:05:39 OPCInt 1> Error reading Active ID point from Data source Active_IN (Point 29600) status = -255</i>	The Active ID point value on the data source caused an error when read by the interface. The value read from the data source must be valid. Upon receiving this error, the interface enters the "Backup in Error state."	Check validity of the value of the Active ID point on the data source.
<i>17-May-06 09:06:03 OPCInt 1> Error reading the value for the other copy's Heartbeat point from Data source HB2_IN (Point 29604) status = -255</i>	The heartbeat point value on the data source caused an error when read by the interface. The value read from the data source must be valid. Upon receiving this error, the interface enters	Check validity of the value of the heartbeat point on the data source

Message	Cause	Resolution
	the "Backup in Error state."	

Error Messages (Phase 2)

Message	Cause	Resolution
<i>27-Jun-08 17:27:17 PI Eight Track 1 1> Error 5: Unable to create file '\\georgiaking\GeorgiaKingStorage\UnIntFailover\PIEightTrack_eight_1.dat' Verify that interface has read/write/create access on file server machine. Initializing uniint library failed Stopping Interface</i>	The interface is unable to create a new failover synchronization file at startup. The creation of the file takes place the first time either copy of the interface is started and the file does not exist. The error number most commonly seen is error number 5. Error number 5 is an "access denied" error and is probably the result of a permissions problem.	Ensure the account that the interface is running under has read and write permissions for the directory. Set the access control list on the directory to read/write/create access to the account that runs the interface.
<i>Sun Jun 29 17:18:51 2008 PI Eight Track 1 2> WARNING> Failover Warning: Error = 64 Unable to open Failover Control File '\\georgiaking\GeorgiaKingStorage\Eight\PIEightTrack_eight_1.dat' The interface will not be able to change state if PI is not available</i>	The interface is unable to open the failover synchronization file. The interface failover continues to operate correctly if communication to Data Archive is not interrupted. If communication to Data Archive is interrupted while one or both interfaces cannot access the synchronization file, the interfaces remain in the state they were in at the time of the second failure: the primary interface remains primary and the backup interface remains backup.	Ensure the account that the interface is running under has read and write permissions for the directory. Set the access control list on the directory to read/write/create access to the account that runs the interface.

Appendix A. Set ExcMax on advise and event points

The PI Interface for OPC DA applies special rules for advise and event points in cases where ExcMax has elapsed since the last time a point updated. This custom exception handling mechanism is complicated, and the results it produces depend not only on other parameters the user has set, but potentially on the user's OPC server and network as well.

The interface applies the same rules for event (i.e., triggered) points as it does for advise points. The following information focuses on advise points only.

Why set ExcMax for an advise point?

The OPC server sends data to advise points whenever a new value is available for the following reasons.

1. The point typically updates slowly, but end users would like to see it update more frequently so that it can be included more easily in displays, rollup calculations, or other analyses.
2. The user wants to record a step trend in the archived data itself instead of setting the *step* parameter on the point. This way steps are visible even when the data is plotted in a program that is not part of the PI System.

In case (2), AVEVA strongly recommends setting the */TA* interface parameter to accomplish the same result without resorting to setting ExcMax on all advise points. For details, please see [Why not just use the /TA parameter?](#) below.

How does the interface handle ExcMax for advise points?

When the OPC server sends a new value to the interface for an advise point, the interface passes this value along to the Data Archive and stores it in memory along with its timestamp. If no new value arrives by the time ExcMax has elapsed, it sends the value again using a timestamp equal to the previous timestamp plus ExcMax.

Note that this exception value does not represent an actual interaction with the data source. This is important because in some circumstances, the value can be incorrect. For example, consider a case where scan class 1, reserved for advise points, has a scan time of 10 seconds. By default, the interface configures the update rate on the OPC server to be equal to this scan time. In this case, the OPC server would poll the PLC it reads this value from every 10 seconds.

Suppose an advise point that reads this value has ExcMax set to 5 seconds. The interface receives a value from the OPC server and then, five seconds later, sends the same value again even though it might have changed on the PLC. Because the OPC server only polls the PLC every 10 seconds, it would not have advised the interface of this change. Without ExcMax set, there would be no record of this value at this time, but with ExcMax set this way, there is a record with an inaccurate value. For this reason, it is extremely important to consider the update rate when setting ExcMax. In particular, the user should never configure ExcMax to be less than the update rate. If possible, the user should set it to at least twice the update rate. See [What happens if I set ExcMax equal to the update rate?](#) for a description of problems that can arise when the two quantities are equal.

Why not just use the /TA parameter?

In cases where the user only wants to create steps in the data and does not necessarily need frequent updates, AVEVA strongly recommends using the /TA interface parameter instead of configuring ExcMax on each advise point. With this parameter set, when a new value arrives for an advise point, the interface sends the previous value with a timestamp of one update interval prior to the new value's timestamp. For example, if the update rate is set to 1 second and a new value arrives 10 seconds after the previous one, the interface sends the previous value with a timestamp of one second less than the new value's timestamp (i.e., approximately 9 seconds after the previous timestamp) and then sends the new value with its current timestamp.

This approach is preferable to using ExcMax to accomplish the same goal because the exception values that are sent this way are guaranteed to be timestamped at or near an actual update time. The only assumption in this case is that the OPC server would have advised the interface at that time had the value been different. In contrast, it is possible to (mis)configure ExcMax so that exception values are sent with timestamps that are not at update times. As a result, one cannot necessarily assume that these values are actually accurate, since the true value might have changed without the OPC server noticing the change.

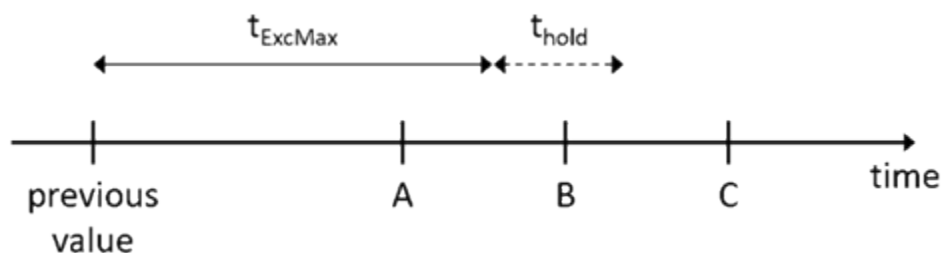
In cases where the user needs to see frequent updates to an advise point that changes infrequently, the /TA parameter does not suffice. Instead, AVEVA recommends two solutions: either configure the point to poll instead of advise (this is the preferred method), or if the resulting network traffic is not acceptable, set ExcMax equal to an integer multiple of the update interval. In the latter case, ExcMax should be at least twice-and ideally many times-the update interval.

Why is the data sometimes out of order?

Prior to version 2.6.3.5 of the interface, if the OPC server sent the interface a new value just as ExcMax elapsed, it was possible that the interface would send the exception value with a timestamp slightly after the new value's timestamp. Version 2.6.3.5 fixed this problem in the case that timestamps were provided by the interface (i.e., /TS=N), but it was still possible for exception values to appear out of order when the OPC server supplies the timestamp. This bug has been fixed in subsequent versions of the interface using a slightly more complicated mechanism than previous versions used.

Figure 1 shows how this mechanism works. The interface now holds on to each exception value for a short time, t_{hold} , before sending it to the Data Archive. If a new value arrives during this short time with a timestamp from before ExcMax elapsed, the interface sends the new value but discards the exception value. In version 2.6.4.x, t_{hold} is set to either 2 seconds or 90% of ExcMax, whichever is smaller.

Figure 1 - Mechanism of handling ExcMax in version 2.6.4 and later



A new value that arrives within ExcMax of the previous value (e.g., at time A) does not trigger an exception value to be sent. On the other hand, if both ExcMax and the extra holding time elapse before a new value is received (e.g., at time C), the interface sends the exception value at $t_{\text{ExcMax}} + t_{\text{hold}}$ with a timestamp equal to the previous

timestamp plus ExcMax. Finally, if a new value arrives after ExcMax has elapsed but before the extra holding time has elapsed (e.g., at time B), the interface will only send the exception value if its timestamp would occur before the timestamp on the new value. This way, the exception value never appears with a timestamp later than the new value, provided that the new value arrives within $t_{\text{ExcMax}} + t_{\text{hold}}$ of the previous value.

What happens if I ignore the fractional parts of timestamps?

The user can specify a command line parameter, `/IT=Y`, that causes the interface to truncate the fractional part of each timestamp and send only whole-second timestamps to the Data Archive. In this case, the interface applies the above mechanism for handling ExcMax before the timestamps are truncated. The only difference is that in cases where a new value arrives after ExcMax elapses and before the extra holding time elapses, the interface performs an additional check before sending the exception value. If truncating both the timestamp of the new value and that of the exception value would result in those timestamps being equal, it does not send the exception value.

What happens if I set ExcMax equal to the update rate?

Setting ExcMax equal to the update rate is almost always a bad idea because it introduces the possibility that the interface will send an exception value without the OPC server having updated at all. The precise timing of events moving from the OPC server to the interface is important in this case, and the interface's resulting behavior depends not just on the OPC server but also on the network.

Figure 2. Timing errors that affect the interface's behavior when ExcMax is equal to the update rate

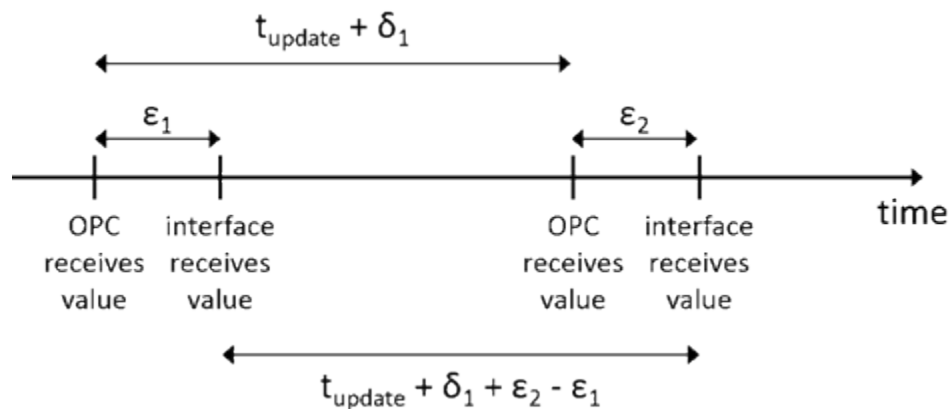


Figure 2 illustrates these timing issues. Specifically, it represents the case where two successive updates on the OPC server yield different values, so both are sent to the interface. There are two important cases to consider:

1. The interface is configured to use the timestamp sent by the OPC server (`/TS=Y`).

First notice that even if the user configures the update rate to some particular value, t_{update} , the OPC server might not update at exactly that interval. For example, the OPC server might wait for t_{update} to elapse, then poll the data source for a value, then reset the update timer, in which case the true time between updates is t_{update} plus the time required to poll for a new value.

In general, the difference between timestamps on successive values that the OPC server sends is $t_{\text{update}} + \delta$, where δ depends on the OPC server, the data source, and the network connecting the two machines. If $\delta > 0$, then setting ExcMax equal to the update interval would cause the interface to send an exception value after

every update, even if the update that immediately follows yields a different value. In other words, this configuration produces a step plot even if the underlying value on the data source is not stepped. The exception values that the interface reports in this case are wrong.

2. The interface is configured to apply its own timestamps as it receives values from the OPC server ($\sqrt{TS=N}$).

This case is similar to (1), but it is slightly more complicated because there is an additional timing error introduced by the network connecting the OPC server and the interface. This additional error is labeled ϵ in Figure 2. As shown in the diagram, the difference between the arrival times of successive values at the interface is $t_{\text{update}} + \delta_1 + \epsilon_2 - \epsilon_1$, where ϵ_1 and ϵ_2 depend on the network between the OPC server and the interface, and δ_1 depends on the factors described above. If the quantity $\delta_1 + \epsilon_2 - \epsilon_1$ is positive, then the same problems described in (1) would still occur. Because the difference $\epsilon_2 - \epsilon_1$ can be negative, however, the false exception values might be sent only intermittently.

As a general rule, it is much safer to set ExcMax significantly larger than the update interval. This ensures that the OPC server actually measures the value multiple times and verifies that it does not change.

Summary

- The PI Interface for OPC DA uses a custom exception reporting mechanism for advise and event points. Specifically, when it has been longer than ExcMax since the interface last received an update from the OPC server, it sends the previous value again with a new timestamp.
- This exception value does not represent a real interaction with the OPC server.
- Users who require true readings should configure polled rather than advise or event points. Alternatively, they should set ExcMax to zero to turn off exception reporting altogether.

To reduce the likelihood of recording inaccurate data, users should at the very least set ExcMax larger than the update rate.

Glossary of interface terms

This glossary defines technical terms related to PI interfaces and related products.

buffering

buffering

n. In the PI System, buffering is implemented as a service that stores and forwards events to a PI Data Archive server or collective. When the server or collective is unavailable for any reason, the buffering service temporarily stores the data from PI System applications running on the buffered computer. When the connection with PI Data Archive is restored, the buffering service sends all the stored data from the buffer to PI Data Archive in chronological order.

ICU

ICU

See [PI Interface Configuration Utility](#).

ICU Control

ICU Control

n. A plug-in to the [PI Interface Configuration Utility](#). Whereas the ICU handles functionality common to all interfaces, an ICU Control implements interface-specific behavior. Most PI interfaces have an associated ICU Control.

n-way buffering

n-way buffering

n. A configuration used when buffering data to a PI Data Archive collective. With n-way buffering, the buffering service sends the same data to each collective member. Of the two buffering services, AVEVA recommends PI Buffer Subsystem (pibufss) for most buffering scenarios, including n-way buffering. PI Buffer Subsystem automatically handles n-way buffering. API Buffer Server (bufserv) requires manual configuration whenever a collective changes, and buffered data is compressed at PI Data Archive, so archives at different collective members could contain different records.

OCS

OCS

n. OSIsoft Cloud Services. Cloud services support applications that require real-time capture, processing and

sharing of large numbers of assets and operational data.

OMF

OMF

n. Open Message Format. Defines a set of message headers and bodies that can be used to generate messages for ingestion into a compliant back-end system.

pigetmsg utility

pigetmsg utility

n. A command-line utility to view messages stored by [PI Message Subsystem](#). The utility can retrieve messages based on characteristics like time stamp, a search string, or the program name that generated the message.

PIHOME

PIHOME

n. PIHOME refers to the folder defined by the *PIHOME* environment variable and is the directory in which most 32-bit client applications are installed. By default, *PIHOME* is **c:\program files(x86)\pipc** on 64-bit machines and **c:\program files\pipc** on 32-bit machines. **pipc** does not need to be in the name, but it is in the name by default.

PIHOME64

PIHOME64

n. PIHOME64 refers to the folder defined by the *PIHOME64* environment variable and is the directory in which most 64-bit client applications are installed. By default, *PIHOME64* is **c:\program files\pipc** on 64-bit machines. **pipc** does not need to be in the name, but it is in the name by default.

pipc.log file

pipc.log file

The **pipc.log** file is a message file for AVEVA interfaces based on Unint versions earlier than 4.5.0.x. When these interfaces run, they write informational and error messages to the **pipc.log** file. View this log file with PI Interface Configuration Utility.

point

point

n. See [PI point](#).

Note: AVEVA often uses the terms point and tag interchangeably. Point is recommended, unless it refers directly

to the PI [tag attribute](#).

PI API

PI API

n. A C-based application programming interface (API) library of functions that enable programs to access a local or remote PI Data Archive server across a network.

PI Data Archive

PI Data Archive

n. The AVEVA product that stores time-series data from distributed data sources and serves this data to client applications in real-time. A major component of PI Server along with PI Asset Framework (AF).

PI Data Archive collective

PI Data Archive collective

n. A configuration of multiple servers that act as a logical PI Data Archive server in your PI System to provide high availability (HA), disaster recovery, load distribution, and increased scalability. Each server in a collective is called a member of the collective.

PI Data Archive computer

PI Data Archive computer

n. A computer hosting PI Data Archive software.

PI ICU

PI ICU

n. The acronym for [PI Interface Configuration Utility](#).

PI Interface Configuration Utility

PI Interface Configuration Utility

n. An AVEVA application that is generally installed on a PI interface node to manage interfaces and their configuration files. Also known by the acronym [PI ICU](#).

PI message log

PI message log

n. A log file that contains messages that [PI Message Subsystem](#) records. Use to troubleshoot components that write to PI Message Subsystem, such as PI Data Archive, client applications, or interfaces. View the log file with [PI SMT](#), the [pigetmsg utility](#), or the [PI SDK](#) utility.

PI Message Subsystem

PI Message Subsystem

n. The core PI Data Archive component that records informational and error messages from various PI Data Archive components in a series of log files. PI Message Subsystem can also serve these messages to various client applications.

PI point

PI point

n. A single data stream stored by PI Data Archive. For example, a PI point might store the flow rate from a meter, a controller's mode of operation, the batch number of a product, text comments from an operator, or the results of a calculation.

PI point attribute

PI point attribute

n. A collection of characteristics or parameters for a [PI point](#) that provides information to specify how an interface and Data Archive collect and process data values.

PI SDK

PI SDK

n. The COM-based software development kit for PI System applications. The PI SDK is a set of programming libraries for development of Microsoft Windows client programs or interfaces that can communicate with most PI Data Archive versions (PI Server 3.2.357 and up) on any supported operating system.

PI SMT

PI SMT

n. See [PI System Management Tools \(SMT\)](#).

PI System Management Tools

PI System Management Tools

n. A set of client-based programs to complete a wide variety of common administrative tasks on a Data Archive. Also known by the acronym [PI SMT](#).

tag attribute

tag attribute

n. The [PI point attribute](#) that assigns a name to uniquely identify a point.

Release notes

PI Interface for OPC DA

Version 2.7.2.20

Unilnt Version 4.7.2.1

Overview

PI Interface for OPC DA is an OSIsoft client application that reads and archives data from OPC data sources.

Fixes and enhancements

Fixes

This section lists items that were resolved in the 2.7.2.20 release:

Work Items	Description
40582	ICU Control now correctly adds /UWQ parameter to the .bat file even if is set to the default value of 0.
40584	Interface now sends "good quality" when /SG is parameter is set.
40593	Resolved duplicate data in PI DA resulting from duplicate connections to the OPC Server when interface gets disconnected from the OPC Server.
40602	Exceptions occurring in the On data changed are now caught and logged.
40608	Resolved incorrect file permissions on .cred file creation. The service user now has access to the credential file.
40619	Backup interface will only collect data from the OPC Server during hot failover. When the interface is assigned to "primary," it will begin sending data to PI.
40623	Interface now continues to update existing Advise tags according to their ExcMax parameter when a new Advise tag is added to a separate empty scan class.
40633	Timestamp plug-in now works with event (triggered) PI points.

Work Items	Description
40636	Startup Delay (/SD) can now be greater than 2 seconds when connecting to a DeltaV OPC Server.
40641	Resolved interface not sending correct values for VT_DATE types when using a certain time format (/TF) parameter. Both 'ccyy' and 'yy' are now properly parsed when sending outputs.
40651	Resolved interface incorrectly reporting opc quality 0xFF as a "Bad" quality when quality should be reported as "Good." Interface now accepts quality range 0xC0 - 0xFF as a "Good" range.
40652	Resolved the interface stopping unexpectedly when adding data to queue.
40668	Resolved interface stopping unexpectedly from misconfigured location codes. Interface now reports an error message with invalid scan classes and then exits.
40670	Interface now starts, collects data, and utilizes specified digital states when Generic Plug-in DLL is used.
40672	When the /CredFilePath is left empty, the warning message now displays the correct parameter name.
40675	Resolved the PI OPC DA Interface falsely reporting the device status "One or more groups not getting data" under certain configurations
40676	Resolved PI String tag output event not being correct if day part of date is not specified in time format (/TF).
40705	OPC DA Server VT_DATE tags with millisecond timestamp values are now written to PI string tags (location2=7) with the correct millisecond values as per the time format specified on the interface, and as decimal second values written to PI Float tags.
43675	Resolved explicit logging for output tags assigned to an output group that does not exist.
72454	Generic and Metso plugins now read digital states properly.

Enhancements

This section lists the enhancements in the 2.7.2.20 release:

Work Items	Description
41277	Updated default authentication level to 'Packet Integrity' in order to be compatible with the requirements of Windows DCOM Server Security. Notes: Interface: The DCOM security authentication level is defined by the /da parameter. OPC servers: The PI OPC Servers will continue to work using that setting. It is recommended to use that setting in the DCOM hardening guidelines. By default, the PI OPC Servers will use an authentication level of 'Default', which will use the machine-wide setting defined at the Computer level. If the machine-wide authentication level is set to Packet Integrity or greater, no change is required. Third-Party Applications: Some OPC applications will fail to establish communication even though the system-wide authentication level is set to Packet Integrity because they hard-code the authentication level to None, Connect, Call, or Packet. Contact the specific vendor of the application for details on how to enable an authentication level of Packet Integrity or greater. Performance: Application performance could be impacted. It is recommended to test with Packet Integrity to ensure performance remains adequate in your environment. To mitigate performance issues, the following should be considered: • use advise tags instead of polled tags • assess data density requirements and adjust scan classes • split tags into multiple interface instances/OPC servers • install the interface on the OPC server machine • upgrade old hardware
43132	Updated dependencies included in install kit provided by OPC Foundation to latest version.

Work Items	Description
72453	Removed bundled executable tools for security purposes including: • APIOnline.exe • DaVinciBackUpExe.exe • makedumpset.exe • opcrefresh.exe • opcresponse.exe • opcsan.exe In the rare case one of these tools are needed, they can still be obtained from the last previously released version of the interface.

Known Issues

This section lists problems and/or enhancements that have been deferred until a future release

- There are no known issues with this release or its ICU control.

System Requirements

Operating Systems

This release is supported on the following operating systems:

- Windows 11
- Windows 10
- Windows Server 2022
- Windows Server 2019
- Windows Server 2016
- Windows Server 2012

Distribution Kit Files

Product	Software Version
PI Interface for OPC DA (OPCInt)	2.7.2.20
PI Interface for OPC DA – Plug-Ins	2.7.2.20
PI Interface for OPC DA (OPCInt) – ICU Control	2.7.2.20

Product	Software Version
OSIsoft MS VB Runtime Redistributables	1.0.1
Microsoft Visual C++ 2015-2019 Redistributable (x86 and x64)	14.21.27702.2
PI Software Development Kit (PI SDK) 2018 SP1 Patch 3 (x86 and x64)	1.4.7.620
PI GenericNames DLL (PIGenericNames)	1.3.5.25
PI Spt DLL (PISpt)	1.2.4.0
PI Interface Configuration Utility	1.5.1.10
PI Buffer Subsystem 2018 SP2 Patch 2	4.9.0.37

Installation and Upgrade

The PI Interface for OPC DA can be installed or upgraded using the desired option of the PI Interface for OPC DA installation kit.

- For the read-only option: OPCInt_ReadOnly_2.7.2.20_.exe
- For the read-write option: OPCInt_ReadWrite_2.7.2.20_.exe

This installation kit can be obtained via the OSIsoft Customer Portal. For detailed instructions on downloading the installation kit, please see the How to Download Products link located on the [OSIsoft Customer Portal](#) site.

For additional information regarding the PI Interface for OPC DA installation, please see the Installation instructions portion of the User Guide. This user guide is located in the PI Interfaces section of the [AVEVA Documentation Library](#).

Uninstalling PI Interface for OPC DA

The OPC DA interface can be uninstalled using the Programs and Features list accessible from the Windows Control Panel. After accessing the Programs and Features list, perform the following steps to uninstall the interface and its ICU control:

- For the read-only option:
 - Select **PI Interface for OPC DA (OPCInt) - Read-Only** and then select **Uninstall**.
 - Select **PI Interface for OPC DA (OPCInt) - ICU Control** and then select **Uninstall** to uninstall the interface's ICU control.
- For the read-write option:
 - Select **PI Interface for OPC DA (OPCInt)** and then select **Uninstall**.
 - Select **PI Interface for OPC DA (OPCInt) - ICU Control** and then select **Uninstall** button to uninstall the ICU control.

Security information and guidance

We are [committed to releasing secure products](#). This section is intended to provide relevant security-related information to guide your installation or upgrade decision.

We [proactively disclose](#) aggregate information about the number and severity of security vulnerabilities addressed in each release. The tables below provide an overview of security issues addressed and their relative severity based on [standard scoring](#).



AVEVA Group Limited

High Cross
Maddingley Road
Cambridge
CB3 0HB
UK

Tel +44 (0)1223 556655

www.aveva.com

To find your local AVEVA office, visit

www.aveva.com/en/about/about-aveva/locations/

AVEVA believes the information in this publication is correct as of its publication date. As part of continued product development, such information is subject to change without prior notice and is related to the current software release. AVEVA is not responsible for any inadvertent errors. All product names mentioned are the trademarks of their respective holders.